

Szymon Galutowski's NEA

**Hospital/GP Scheduling and
Management System**

GPQueue



Contents

Analysis-----	5
Introduction-----	5
Current system and Comparisons -----	5
Priority Level -----	8
End users-----	8
Hospital workers-----	8
Patients -----	8
Requirements gathering-----	9
Interview with GP staff member -----	9
Survey-----	10
Research -----	13
Symptoms-----	13
Level 1 -----	13
Level 2 -----	14
Level 3 -----	14
PostgreSQL -----	14
Website vs. Executable-----	17
React -----	19
API -----	21
REST-----	21
Flask -----	22
Fetch API -----	23
Async/Await -----	23
CORS-----	24
JWT -----	24
Other Security-----	25
Password hashing -----	25
Role-based Access Control -----	25
.env files -----	26

Refresh Token Rotation-----	26
Hiding Passwords in Inputs-----	26
Objectives-----	26
Design-----	28
Wireframes-----	28
UML Sequence Diagrams -----	34
Registration/Login -----	35
Home-----	36
Flowcharts -----	39
Database diagrams-----	44
Technology Stack -----	45
Role-based Access Control -----	45
JWT storage -----	45
Contexts-----	46
Back Button-----	46
Technical Solution -----	48
Frontend-----	48
Written Files -----	48
1 - App.js -----	48
2 – home.js-----	49
3 – login.js-----	56
4 – TopBar.js -----	61
5 – appointments.js-----	62
ApiCalls.js -----	74
smallComponents.js -----	83
stack.js -----	85
userContexts.js -----	87
testData.js -----	88
App.css -----	88
Favicon.png-----	88
test.js -----	88
Auto-Generated Files -----	88

Generated files meant for use, but gone unused: -----	88
Generated files, not meant/required to be edited: -----	88
Backend -----	89
pycache -----	89
.env -----	89
Backend.py -----	89
fetch_dates.py -----	97
booking_algorithm.py -----	99
MyHash.py -----	104
Cronjob.py -----	105
PostgreSQL Scripts -----	108
DB.pqsql -----	108
InsertStuff.pgsql -----	109
InsertTestUsers.pgsql -----	109
populate.pgsql -----	110
WIPE.pgsql -----	111
Testing -----	113
Modular Testing -----	113
Testing -----	113
Evidence -----	117
Fixes -----	170
Acceptance Testing -----	171
Evaluation -----	176
Second interview -----	176
Completeness of Objectives -----	177
Final Thoughts -----	180
Bibliography -----	182
Appendix -----	184

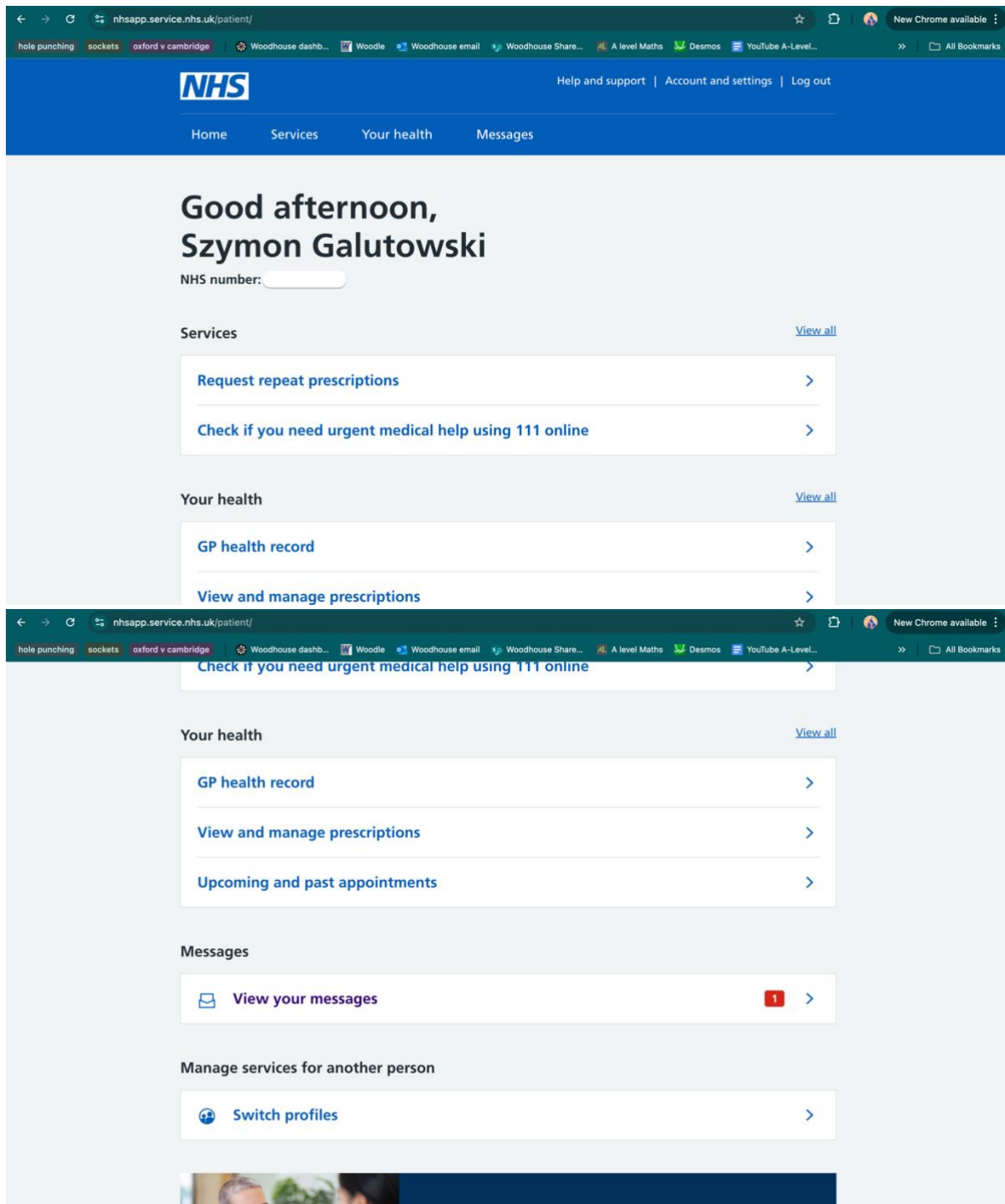
Analysis

Introduction

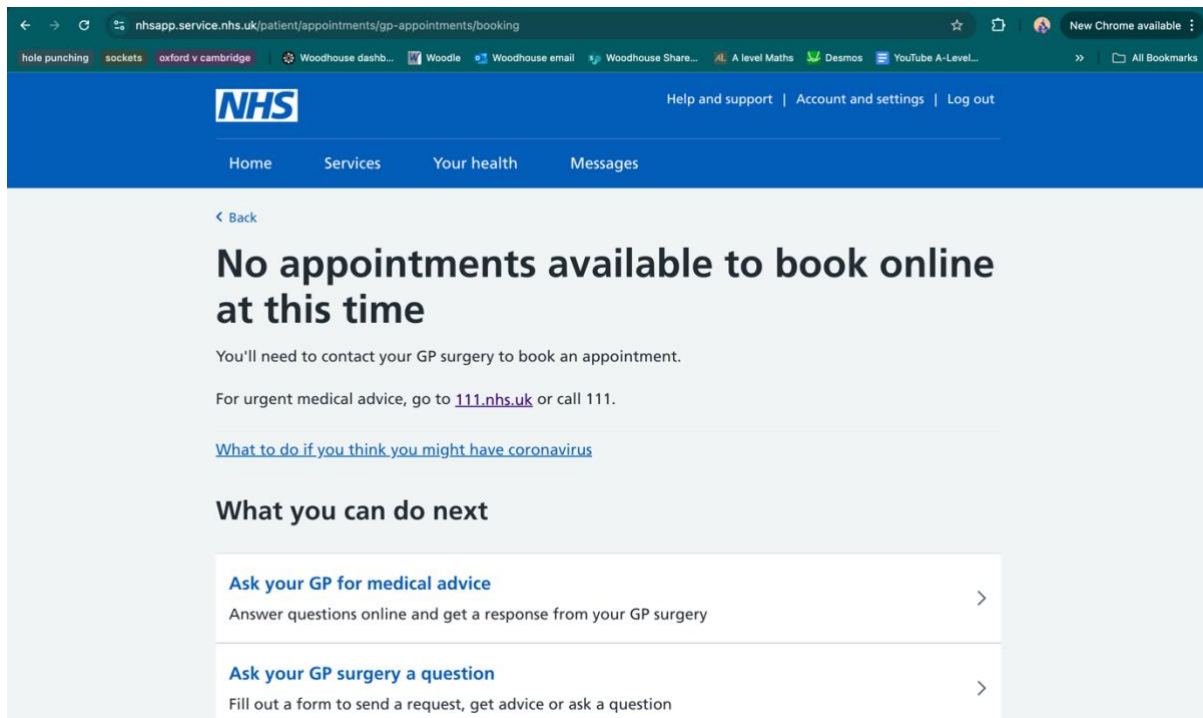
When you book an appointment at your local GP or hospital, there is a process where they take your details and store it in a system. They also must be able to schedule your appointment so that it fits among others and your requirements. This is quite tough to do, as a hospital can be quite chaotic, and all appointments are always booked. That is why my project is going to be a Hospital Management and Scheduling System. Building a scheduling system is rather easy, but its main functionality makes it harder to build. The system can reschedule patients based on priority level – for instance if a person booking a checkup will be assigned priority level 3, but a person with a more serious condition like a sprained ankle will have priority level 1 and will override the priority level 3 appointment. The other person will be rescheduled to another date and their priority level will increase, so that they aren't constantly shifted around.

Current system and Comparisons

The current system is what the NHS uses. As we all know, the NHS is not very good at scheduling appointments, with them often being at inconvenient times. Additionally, you then must wait a long time upon arrival, since there are always some delays. In comparison, my scheduling system will allow you to choose from open timeslots, and it will not involve any unnecessary phone calls. The NHS appointment website looks like this:



This is the home page, where you can click on different tabs to look at your health record, past and future appointments, prescriptions, and messages.



Above is the tab that opens if you click on 'Check for available GP appointments'. This is why I am building my own app – clearly, their scheduling system must be flawed, as there are no appointments available to book online.

A very small percentage of the population enjoys calling or going in person to book, so making it all online will appease those people and be accessible to people with disabilities or mobility issues. Another merit of the system being a website is the time saved by people booking – due to the simplicity it will be very quick and easy to book an appointment, which is much better than being on hold for half an hour or waiting in a queue at a busy GP.

The NHS booking system for most GPs is very simple: appointments in the morning are reserved for emergencies, and all other appointments are for normal, day to day worries. However, the patient may not be educated or informed well enough to know if their symptom is an emergency or not. The timing of emergency appointments is also a bit flawed, as early in the morning is when people will be feeling at their worst and a diagnosis made by a staff member may be over-exaggerated.

The reason why my system will be better in this sense is because this allows the patients to choose their symptom from an extensively researched list of symptoms, which are directly linked to a priority level. This priority level will directly correlate to the importance of the appointment, and depending on how high it is, it will automatically remove the possibility of the appointment being rescheduled.

By automating appointment scheduling and reminders, my system reduces the workload for healthcare staff. This means that they can focus on more important tasks

that require their individual attention, such as providing patient care and planning prescriptions.

To summarise, the difference between the NHS system and mine is timing and better freedom of choice in terms of appointment time, with the added complexity of priority levels which ensures that serious symptoms will always be seen to as soon as possible.

Priority Level

As I have already mentioned, my scheduling algorithm will priority based. The lower the priority integer, the better appointment availability the patient receives. In this document I will be saying that a higher priority is equal to a smaller integer, and vice versa. However, determining priority level based on symptoms can be quite tough. Some reasons for booking, such as a checkup, will be easy to book as they will most likely have a priority level of 3 – the lowest. Some symptoms may appear minor at first, but they often can indicate a more serious condition behind the scenes. This makes it important to assess them accurately. As such, I have done some research on the most common symptoms and given them a rated priority level. By using this priority system, the scheduling algorithm will ultimately ensure that patients with urgent conditions receive prompt attention, while patients with less severe symptoms are scheduled for appointments at a later time.

End users

Hospital workers

The hospital workers will be using the website to check which patients they have to treat/check, and the details of those patients. There will be a functionality where a time slot is assigned to a worker, so that there will be multiple appointments at once (based on the number of active workers). The workers will be able to see which time slots are assigned to them, and which patients they see in that time.

Patients

The patients will be using the app to book their own appointments. To start using it, they have to make an account and enter their details such as their phone number, email address, and name. Upon account creation they will also receive a patient ID. They will also see the times of their existing appointments, or possible appointment times for booking. The app will also send them emails as their appointment approaches, or if their appointment has been moved (which may happen).

Requirements gathering

Interview with GP staff member

I performed an interview with a GP staff member. These were my questions and their answers:

Question	Answer
What do you think of the current system your workplace uses?	I don't really have an opinion on it – it would be nice to have it upgraded though.
Would you prefer a command line interface or a graphical interface?	I want to have buttons and pictures that I can interact with.
Would you prefer a website or a desktop app?	A website.
Would you like a calendar display for the days or a listed display?	A calendar display.
What should the length of the timeslots be, and when should they start and end?	The day starts at 9:00 and ends at 21:00, and each slot should be 30min long.
Would you like email reminders for every appointment with a patient?	Yes, 30 minutes before would be cool.
Would you like a lot of information on each page, or for it to be hidden and simplified?	Simplified.
Would you like to be able to see staff schedules?	I need to see my own schedule, I don't need to see the schedule of other staff though.
Do you require an appointment log?	Yes.
Do you require the ability to create appointments with a patient?	No, the patient will create them
What is the maximum number of appointments a patient should have booked in one time?	3.
Would you like to be able to change the language?	No, people working here will know English.
Would you like to delete appointment log history?	No.
Should there be two-factor-authentication?	No.
How should users log in?	With their email and password.
What information should be gathered upon registration apart from name and email?	Date of birth.
How should staff register?	They fill in their details as a normal user would, but their registration is checked by

	an IT admin who then can manually verify them.
--	--

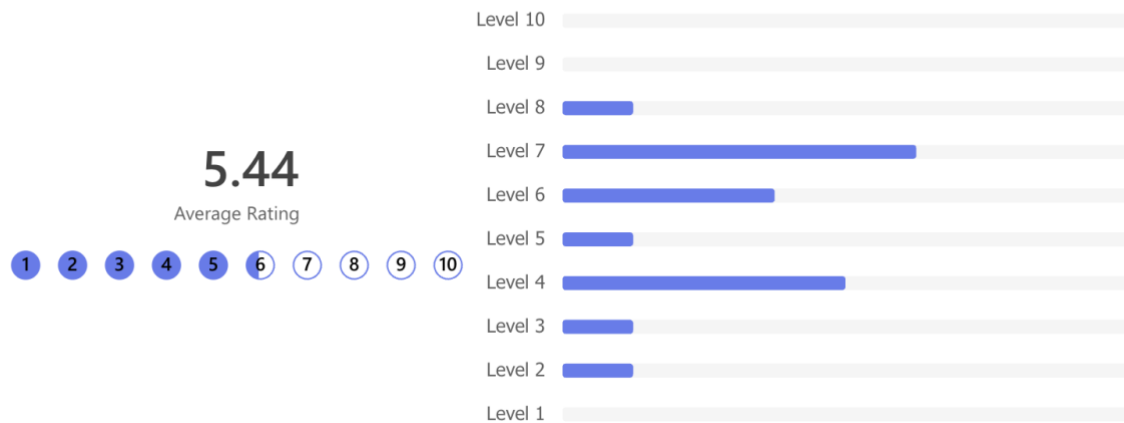
Survey

I made a survey to gather client opinions. I then sent the survey out to communities I was a part of, making sure to only gather opinions from British people, or people who have dealt with the NHS.

These are the results:

1. On a scale of 1-10, how easy do you find it to book an appointment with the NHS? (1 = hard, 10 = easy)

16 Responses



2. If you were going to use an app to book an appointment, would you like it to be a downloadable phone/desktop app or a website?

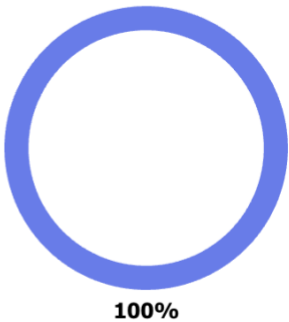
16 Responses



4. Would you like to be able to view your own appointment history?

16 Responses

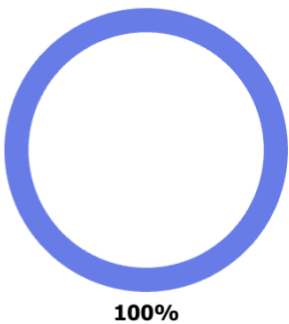
- Yes 16
- No 0



5. Would you like to receive email reminders about your appointment?

16 Responses

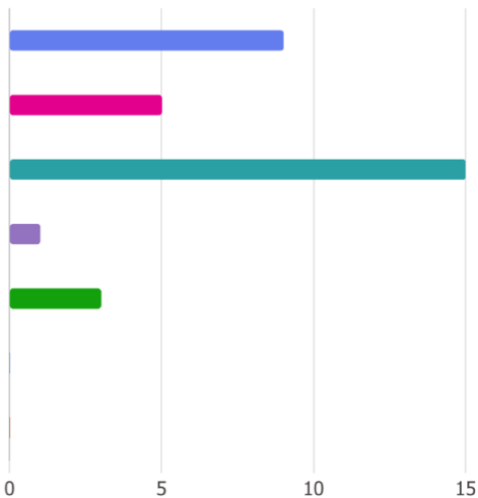
- Yes 16
- No 0



6. When would you rather receive appointment reminders?

16 Responses

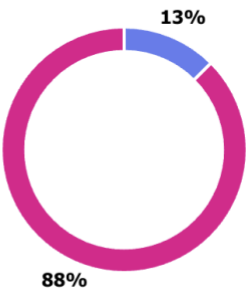
- A week before 9
- 2 days before 5
- 1 day before 15
- 12 hours before 1
- 1 hour before 3
- 30 minutes before 0
- I do not want to receive email reminders. 0



7. Would getting rescheduled anger you if the person who took your spot had a more serious condition than you?

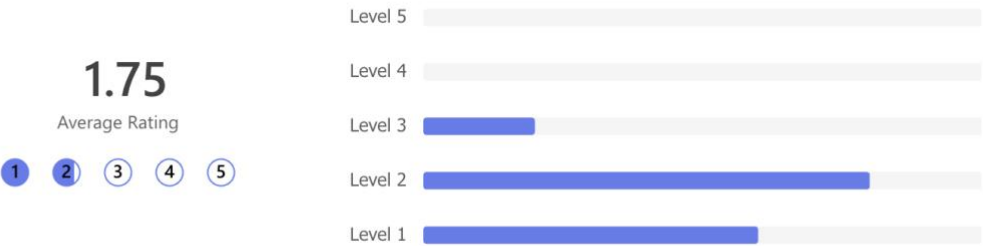
16 Responses

- Yes 2
- No 14



8. What is the maximum amount of times a patient should be rescheduled before they gain priority and have a guaranteed appointment at a set time?

16 Responses



9. Would you prefer a command line interface or a graphical interface?

[More Details](#)

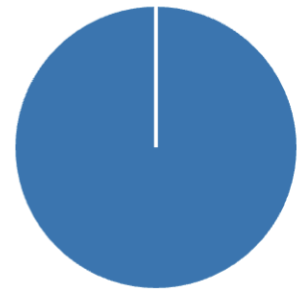
[Insights](#)

- Command line (typing things in) 0
- Graphical (pictures and buttons) 16



10. Would you like a calendar display for the timeslots or a listed display?

[More Details](#)



11. Would you like a lot of information on each page, or for it to be hidden and simplified?

[More Details](#)

 Insights



Research

Symptoms

To help gather a list of symptoms, I utilised the NHS Inform [1] website. I did some research on the symptoms to see which conditions they could correlate to, such as Dizziness correlating to a possible ear infection, migraines, or dehydration. This allowed me to make my own decisions on how to rank the symptoms. I was able to split them all into 3 priority levels – the most serious for Level 1, medium for Level 2, and minor for Level 3.

Since I am not a medical professional, to ensure I was being accurate, I used Claude.ai [2], an AI model which excels in analysing and structuring information. Claude took my unordered list and split them into 3 levels, in a very similar way to my own results. After verifying my list, and identifying the differences, I ended up with a mix of our 2 lists, shown below:

Level 1

- Persistent fever
- Severe sore throat
- Ear pain

- Abdominal pain
- Concerning lumps
- Anxiety/Depression (worsening)
- Unexplained weight loss
- UTI or STI symptoms
- Worsening chest infection

Level 2

- Sprained joint
- Ongoing migraines
- Skin rashes
- Chronic pain needing review
- Digestive issues
- Sleep problems
- Medication side effects
- Minor infection
- Persistent cough

Level 3

- Check up
- Medication review
- Blood pressure monitoring
- Routine vaccinations
- Minor skin conditions
- Mild joint pain
- General health advice
- Travel vaccinations
- Test results review
- Other

PostgreSQL

I looked at 4 main Relational Database Management Systems:

- MySQL (recommended by a friend)
- SQLite3 (a very RDBMS for python)
- Oracle (a slightly more niche database system)
- PostgreSQL (a well-known and well-admired RDBMS)

During my research, I found a very interesting article [3] which referenced and grouped comparisons between different RDBMS. In one of the tests performed, Oracle was the fastest, partly due to the test being conducted in a virtual environment, where Oracle's performance is optimised. However, in a different test, not located within a virtual environment, it was ascertained that SQLite was the most at retrieval and insertion operations, and it was deemed to be very scalable. In both tests, MySQL came out at

the bottom, and PostgreSQL came second. There were more tests referenced, and multiple tests conducted by the writer, but overall, the hypothesis that PostgreSQL is suitable for an application with a small load was proven. This is one of the reasons why I ended up using it.

I did attempt to use MySQL and SQLite3. I did not attempt to use Oracle, because after visiting the documentation and making some quick google searches, I disliked the amount of support available for it as opposed to the other RDBMS. SQLite3 was quite a popular choice for python, however it didn't have any good tools for viewing the database structure in something like an ERD diagram or quickly viewing the contents of the tables, or none that I could locate. I installed MySQL and followed the setup instructions, however due to technical difficulties it would not work properly, and the server kept on shutting down. When I installed PostgreSQL, the server ran properly, and I was able to host multiple databases, hence it would be useful for a test database as well. Additionally, my editor (VSCode) had an existing extension for viewing the tables in a database, and their contents. I was also able to locate a tool known as PGAdmin4 [4], which provided a lot of extra methods for working with the database. Running PostgreSQL in python was proving to be a problem, but I found a python library known as psycopg2 which provided an interface from the python code to the database.

To summarise:

- MySQL – Overly complex to set up, research paper provided evidence for it not being optimal.
- SQLite3 – Lightweight and simple, however there is a lack of tools.
- Oracle – Very good performance, however it's nicheness means there is a low amount of support with learning how to use it.
- PostgreSQL – Good support, useful tools, optimal for applications with a low data load.

Overall, due to conclusions withdrawn from the research article [3], and other experimentation conducted by myself, I chose PostgreSQL to be my RDBMS. Therefore, I created a quick login prototype to make sure I would be able to use it, which was successful.

```

1 import psycpg2
2
3 class User():
4
5     def __init__(self):
6         self.conn = self.connect()
7         self.cur = self.cursor()
8         self.role = None
9
10
11     def connect(self):
12         with open('/Users/szymonator1625/PythonPrograms/NEA_Prototyping/FullPrototype/hosp_config.txt', 'r') as f:
13             config = []
14             for i in f:
15                 config.append(i[:-1])
16
17             try:
18                 conn = psycpg2.connect(host=config[0],
19                                       dbname=config[1],
20                                       user=config[2],
21                                       password=config[3],
22                                       port=config[4])
23             except Exception as error:
24                 print(error)
25
26         return conn
27
28     def cursor(self):
29         return self.conn.cursor()
30
31
32     def register(self):
33         forename = input('Enter your forename:\n').title()
34         surname = input('Enter your surname:\n').title()
35         password = input('Enter a password:\n')
36         print('Are you a patient or a doctor? Type P for patient and D for doctor:\n')
37         while True:
38             role = input()
39             if role == 'd' or role == 'D':
40                 role = 'Doctor'
41                 break
42             elif role == 'p' or role == 'P':
43                 role = 'Patient'
44                 break
45             else:
46                 print('Invalid entry, try again.\n')
47
48
49 class User():
50     def register(self):
51         self.role = role
52         try:
53             self.cur.execute('SELECT id FROM logins')
54             id = self.cur.fetchall()[-1][0] + 1
55         except IndexError as error:
56             id = 0
57         self.cur.execute('INSERT INTO logins(id, forename, surname, role, password)
58                           VALUES (%s,%s,%s,%s,%s)', (id, forename, surname, role, password))
59         self.conn.commit()
60
61     def login(self):
62         p_flag = False
63         forename = input("Please enter your forename:\n").title()
64         surname = input("Please enter your surname:\n").title()
65         password = input("Please enter your password:\n")
66         print('Do you work at the hospital as a medical professional? Y/N')
67         while True:
68             role = input()
69             if role == 'Y' or role == 'y':
70                 role = 'Doctor'
71                 break
72             elif role == 'N' or role == 'n':
73                 role = 'Patient'
74                 break
75             else:
76                 print('Invalid entry, try again.\n')
77                 continue
78
79         try:
80             self.cur.execute('SELECT password FROM logins
81                               WHERE %s=forename AND %s=surname AND %s=role', (forename, surname, role))
82             p = self.cur.fetchall()[0]
83             for i in p:
84                 if i == password:
85                     p = password
86                     p_flag = True
87             if not p_flag:
88                 print('Invalid name, password, or role, please try again.')
89                 return False
90         except IndexError as error:
91             print('Invalid name, password, or role, please try again.')
92             return False

```

```

3 class User():
59     def login(self):
74
92         self.cur.execute('SELECT id FROM logins WHERE %s=forename AND %s=surname AND %s=role AND %s=password',(forename, surname, role, p))
93         id = self.cur.fetchall()[0][0]
94         print("Welcome {} {}, you are now logged in. (ID {})".format(forename, surname, id))
95         return True
96
97
98     def rate(self):
99         severity = input("What is the severity of your ailment?")
100
101     def terminate(self):
102         pass
103
104
105
106 user = User()
107 while True:
108     q = input('To log in, press L, to register press R:\n')
109     if q == 'l' or q == 'L':
110         if user.login() == True:
111             break
112     elif q == 'r' or q == 'R':
113         user.register()
114         if user.role == 'Patient':
115             user.rate()
116             break
117
118 q = input('Would you like to create an appointment? Y/N')
119 user.conn.close()

```

The prototype is command line based and takes in inputs to register a person, giving them an entry in the logins table. A person can then also login, and when they input their details it checks them against the logins table once more. Of course, this is just a prototype to see how PostgreSQL can be used in python, and I am going to be making a small relational database, using pgAdmin4 to manage the connections.

Website vs. Executable

I have decided to build a website instead of coding an executable. If I were to build an executable, it would most definitely be in python, since it is my most practised language. A well-known Python library for GUIs is Tkinter, which I have worked with before. In my opinion, the widgets all look archaic, and are extremely tedious to work with. For one, every widget has to be initialised and then packed properly so that it is displayed on the page in the correct position. As you can see below, this is very ugly,

and easy to lose track of:

```
welcome_m = tk.Label(homepage, text='Welcome Callum P!')
welcome_m.place(x=1,y=1)
quit_b2 = tk.Button(homepage, text='Quit', command=exit)
quit_b2.place(x=950,y=1)
arm_b = tk.Button(homepage, text='Arm System',height = 1, width = 20, bg='green')
arm_b.place(x=110,y=30)
sys_m = tk.Label(homepage, text='The system is currently disarmed.')
sys_m.place(x=300, y=30)
disarm_b = tk.Button(homepage, text='Disarm System', height=1, width=20, bg='red')
disarm_b.place(x=510,y=30)
live_b = tk.Button(homepage, text='Live Feed', height=1, width=20, bg='cyan')
live_b.place(x=710, y=30)
search_e = tk.Entry(homepage)
search_e.place(x=300, y=400)
search_b = tk.Button(homepage, text='Search by Video ID', height=1, width=15)
search_b.place(x=500, y=400)
refresh_b = tk.Button(homepage, text='Refresh', height=1, width=15, bg='mediumslateblue')
refresh_b.place(x=400, y=500)
treeview = tk.Frame(homepage)
treeview.place(x=30,y=110)
scroll = ttk.Scrollbar(treeview)
scroll.pack(side='right', fill='y')
```

The way that the Tkinter main loop runs means that it is also quite difficult to have multiple windows open at once, or different processes running at the same time. Therefore, if I end up using an executable, every time I attempt to run a process, the rendering loop of the current page, and the user will be locked until the process finishes. The consequence of not coding an executable is having to learn web development, and new languages. These will be HTML, CSS, and JavaScript. HTML is much 'nicer' in terms of how the code looks, and it is easier to follow the page structure. The CSS styling available is much better than what is available to Tkinter, so it will prevent the website from looking like the Windows XP OS interface. Another benefit of using a website is the accessibility. The user does not have to install anything, and the workload will be directed to a backend, where there will be dedicated hardware to perform many intensive operations. This is good since it means that any user can access the system, regardless of the performance of their technology, since all they need is a browser. Additionally, in the surveys conducted above, the conclusion was that users would prefer to use a website as opposed to an executable. As a result of these opinions and my own, I will be building a website because I want to, and everyone wants me to.

React

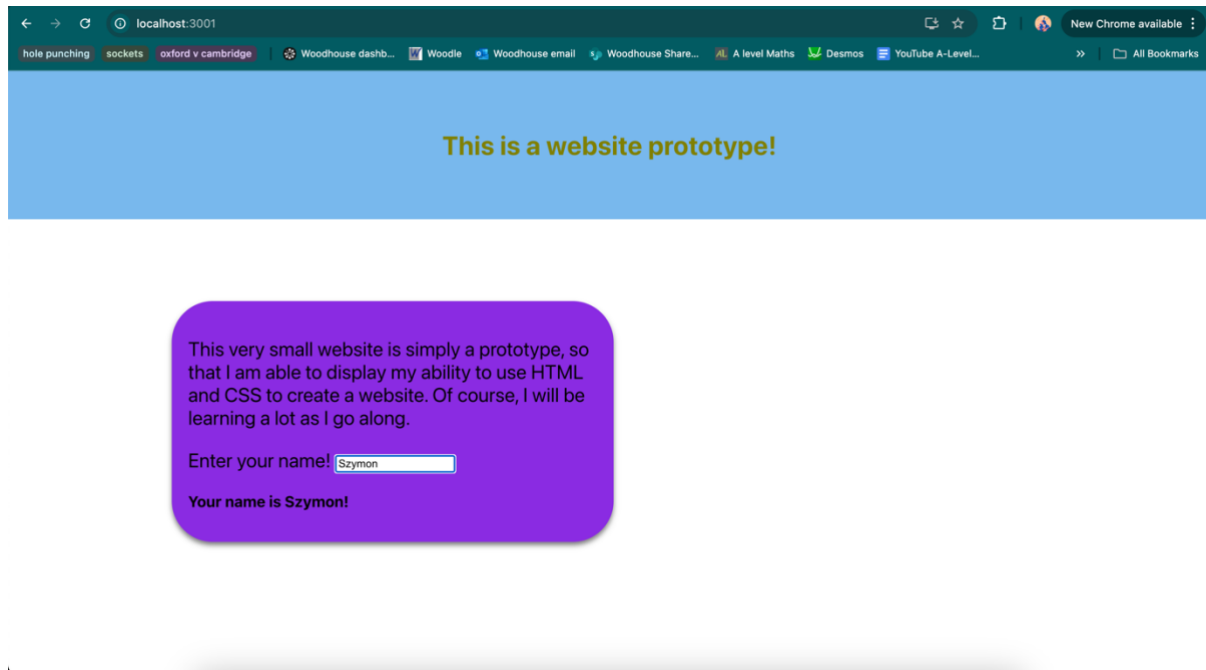
I decided that I would like to use a proper frontend, made with JavaScript, and HTML and CSS for the interface. This will be quite tough, unless I use a dedicated JavaScript framework like React.js. Luckily, there is a software existing already that allows you to very easily create a web app with React, called Create React App [5]. This automatically installs the necessary packages and creates a website (locally hosted, of course) for you to work on and practice your skills. React also helps you nest HTML and CSS elements, and since React is a JavaScript framework, it is very easy to use. JavaScript can be a difficult language, but there are a few similarities between it and python, so it will be easy enough to pick up, especially since I have some prior knowledge.

React is very suitable for my needs due to ease of use and availability and has several excellent features that make front-end web development faster and more responsive. For instance, React uses a virtual DOM (Document Object Model); where a DOM is a representation of a page or HTML document resources into objects. [6]DOM can be manipulated so that web pages become more interactive and attractive. React also has many components – where a component is a small UI element that can pass data to a View as props that can change over a period. Components are reusable, they separate the UI into different parts, they are like JavaScript functions, and take inputs known as props. There are also 2 types of components: class components and functional components. There is also conditional rendering, where you can choose to display different components based on if statements. This is very useful, as I am going to have different components load based on if the user is a patient or a staff member. Additionally, React allows for components to be nested, meaning I can preload each page with its own components. What may present a bit of a challenge is taking inputs from text boxes and using them in code to fetch data, but there are provided functions such as `useState` and `useEffect` that can aid in this. `useState` can be used to change the state of a variable upon a change in input, while `useEffect` is a very useful hook which allows us to run side-effects and cleanup code upon the updating of a state – where a state is a variable stored in our `useState` hook.

React's method for routing is very simple to use and only requires some setup. In the main `App.js` file, you can import the React Router library and write a few lines which will initialise the routes. You then add a line of code for each page, specifying the name of the page as a React component which can be imported from the relevant file, and the name of the page (the part of the URL that goes after the '/'). To then navigate between pages, you use a hook called `useNavigate`, which is very simple to use. You initialise the hook by assigning it to a variable such as one called `'navigate'`, which stores the function, and then you call your `'navigate'` function while providing the name of the page/component you want to navigate to. All processes cease in your current component, and the new component renders.

One final very useful feature which convinced me to use React is Contexts. Contexts are a storage of a value which is accessible throughout the entire program. The value which can be stored can also be an object, meaning you can store a lot of items. I am planning on storing user data which will be required for all pages, but it will be tedious to call the API for the data every time a page renders.

Here is a very small website prototype:



```
Js App.js  X  # index.css
my-app > src > Js App.js > [?] default
1  import './App.css';
2  import { useState } from 'react';
3
4  function EntryBox() {
5
6      const [name, setName] = useState('unknown');
7
8      const nameChangeHandler = (evnt) =>{
9          const nameValue = evnt.target.value;
10         setName(nameValue);
11     }
12
13     return (
14         <>
15         <label>Enter your name! <input onChange={nameChangeHandler}></input></label>
16         <p></p>
17         <h3>Your name is {name}!</h3>
18         </>
19     );
20 }
21
22 function App() {
23     return (
24         <body>
25         <div>
26             <div className='topBar'>
27                 <h1>This is a website prototype!</h1>
28             </div>
29             <div className='explain'>
30                 <p>This very small website is simply a prototype, so that I am able to display my ability to use HTML and CSS to create a website. 01
31                 <EntryBox></EntryBox>
32             </div>
33         </div>
34         </body>
35     );
36 }
37 export default App;
```

```
Js App.js  # index.css  X
my-app > src > # index.css > ...
1  h1 {
2      font-style: bold;
3      color: olive;
4      text-align: center;
5  }
6
7  .topBar {
8      background-color: #77b8ed;
9      padding-top: 50px;
10     padding-bottom: 50px;
11 }
12
13 .explain {
14     background-color: blueviolet;
15     border-radius: 50px;
16     border-color: red;
17     padding: 20px;
18     margin-top: 100px;
19     margin-bottom: 20px;
20     box-shadow: 0 4px 6px rgba(0, 0, 0, 0.5);
21     transition: transform 0.2s ease-in-out;
22     width: 500px;
23     padding: 20px;
24     margin-left: 200px;
25     margin-right: auto;
26 }
27
28
29 p, label {
30     font-size: x-large;
31 }
```

API

REST

To make the website display data, I will need an API (Application Programming Interface). This is because all the data is stored in an SQL database in the backend, but

it is going to be displayed in the frontend. There are multiple types of API structures; perhaps the most well-known is REST. REST was chosen to be the standard for APIs in the world because of its flexibility, scalability, ease of building and reliability. [7] RESTful API principles include using JSON or XML formats when sending the data, and what HTTP requests to use. The most common requests are POST for sending data and GET for requesting data, but PUT and DELETE are also used for updating and deleting data. These commands are a one to one mapping of CRUD: CREATE, RETRIEVE, UPDATE, DELETE.

There are also 2 other API structures: GraphQL and SOAP. [7]

Features of GraphQL:

- One query is enough to retrieve data of any complexity
- Availability of a typed query data schema.
- Works effectively with linked data.
- Detailed error log.
- Flexible customization of queries (format definition, portioning, single version).
- Low performance if there are too many nested fields.
- Difficulties with caching.
- And perhaps the most deciding factor; a high learning curve.

Features of SOAP:

- The ability to work with various transport protocols.
- Implementation of built-in error handling.
- Security enhancements.
- Detailed message format (lots of metadata).
- Support for only a single XML format
- Requirement for large server bandwidth due to the use of XML
- Requires deep knowledge of protocols and data transfer rules.

In conclusion, I have decided to use a RESTful API. This is not only due to the fact that it is an industry standard, but also because JSON can be sent over (as opposed to XML, which is not syntactically sugary at all). It is possibly the simplest and abstracted API structure out there, and this is good enough for me.

Flask

To receive requests in the Python backend, we use a Python micro-framework known as Flask. Flask is built for web application building in Python. It also includes the ability to send and receive HTTP requests. Django and FastAPI are other libraries, but I chose to use Flask. This is because Django is commonly used for much heavier payloads, and

FastAPI doesn't have very good documentation, which is very important for me, since I am doing this for the first time. Flask is much more lightweight, and has easy to understand documentation, is very simple to set up and use, so it is therefore much better suited for my needs. Flask can also deliver payloads of JSON, which is important for me, since I am using React with JavaScript, and JSON text directly maps to JavaScript objects, and can be sent over a RESTful API.

Fetch API

During my research, I found two main promising choices for making HTTP requests from the frontend. The first one is Fetch API, a built-in JavaScript library, and the other is Axios. There are several similarities and differences between the two: [8]

- Axios has better protection compared to Fetch.
- Axios can intercept requests and responses, which makes it more flexible than Fetch.
- Fetch is native and does not need to be installed, while Axios does.
- This also means Fetch is more future proof compared to Axios.
- Axios is marginally faster than Fetch, but this difference in performance is negligible

Both libraries essentially have the same functionality – they provide an interface to send and receive requests to the Flask web server I will set up. Axios is slightly more abstracted than Fetch, making it more favourable, but this abstraction can make it harder to understand as a first-time web developer. Therefore, since it is my first time, I will be using Fetch API.

Async/Await

Async and Await are part of a feature in JavaScript that doesn't necessarily need to be used in conjunction with an API, but they are very useful when doing so. Async and Await are used with promises to implement asynchronous computations. [9] For instance, if I want to make a certain function wait while another function completes its task, I will make the second function an asynchronous function by using the `async` keyword, and signal this to the main function by using the `await` keyword before calling it. Asynchronous functions return what is called a promise; a promise being an object that represents the value computed by an asynchronous computation, and is in one of three states: pending, fulfilled, or rejected. Using `async/await` also has many benefits, such as reducing boilerplate code (code that is repeated, such as promise chains), and it is easier to implement exception handling with the use of a single `catch` statement. However, one needs to be careful when utilising these methods, as using them can slow down execution and cause memory leaks. Sequential execution can be slower when asynchronous functions are being called one after the other and they do not depend on one another, meaning it would be much more efficient to run them in parallel. Memory

leaks are caused when `async/await` is used within loops, and those processes could queue and take up a large amount of memory. If an error occurs and it isn't handled properly, the functions will not release the memory and cause the program to slow or crash. Given that precautions are taken against these dangers, asynchronous computation is a very useful way of making sure functions 'wait' for each other, while also making the code look cleaner.

CORS

CORS (Cross-Origin Resource Sharing) is an HTTP-based header which allows the web server to indicate any origins, other than its own, which a browser should permit loading resources. [10] To be clear, an origin is defined by the protocol (HTTP or HTTPS), domain, and port number for a specific URL. An example of a Cross-Origin request is `localhost:3000` sending a request to `localhost:5000/endpoint1`, the location of a specific endpoint in a web server. Notice that the origins are not the same. For security reasons, browsers always restrict cross-origin HTTP requests, that is why unless the API is fetching from the same domain, CORS headers need to be present in the request. CORS provides extra security for cross-origin requests, since it can be used to whitelist certain origins. Since I can only run my React app on another port, I will be using CORS to whitelist that origin of request and keep my API secure.

JWT

JWTs (JSON Web Tokens) are a very good way of maintaining security between the frontend and the Flask web server. There are two types of token which I will be using: Access Tokens and Refresh Tokens. An access token is a digital asset, typically a JWT, allowing for access to resources without the need for a password. JWTs are created from a unique key that is related to the specific user, which is very useful when the data that requests return depends on the identity of the user. Instead of sending the user id in the body of a GET request every time, the access token can be created from the user id, and if the token is accepted and the endpoint process starts running, it can extract the id from the token. To maintain security, access tokens are made to expire. The default expiration of a JWT token from Flask's JWT-Extended library is 15 minutes. Therefore, once it has expired, to save the user the hassle of providing credentials every 15 minutes, refresh tokens can be generated which last 24 hours by default. When the browser detects that its access token has expired, it can use the refresh token to request a new access token. This is to maintain that extra layer of security. Therefore, JWTs provide safety when requesting data from an API, and can be used to make sure the user is properly authorised to access the data.

Other Security

To maintain security through methods other than JWT and CORS, I will be using these techniques:

- Password hashing
- Role-Based Access Control
- A .env file
- Refresh Token Rotation
- Hiding Passwords in Inputs

Password hashing

Storing passwords in a database is unwise, since if the database contents get leaked or hacked, the accounts of thousands of people, and their sensitive information, could get released. That is why password hashes are stored instead. When a user registers, their password will be hashed using an algorithm which takes in the password and a salt and returns the hash. A salt is a unique, completely random byte object which will be generated with something like `os.urandom()`, and it will be used in the hashing algorithm. The hash and the salt will be stored in the database, alongside attributes that identify the user. When the user logs in, the password they enter will be hashed using the salt that their email corresponds to, and if the computed hash matches the hash stored in the database, the user is let in. To be extra secure, I will be using an inbuilt hashing library in Python called `hashlib`. In particular, I will be using one of `hashlib`'s functions known as `pbkdf2_hmac`, which can be configured to use SHA256, a cryptographic hashing function which returns a 256-byte object. [11] The reason I will be using `pbkdf2_hmac` is because it utilises something called iterative hashing, which is when the result of a hash is hashed again, and the result of this hash is hashed again, and this goes on for a large number of iterations. Around 100000 iterations is usually deemed secure enough. To summarise, using an inbuilt hashing algorithm will be the smartest choice, since it will be much more secure than anything I could create.

Role-based Access Control

Role-based Access Control is a security model which restricts system access based on predefined user roles. Instead of granting permissions to specific users, the permissions are granted to roles, which can be provided as attributes to each user. In my website, I am going to have two types of users: a staff member and a patient. By setting this type, I am able to determine if the current user can perform certain actions, such as being assigned an appointment with a patient (reserved for staff members), or being able to book an appointment (reserved for patients). By using this model, it simplifies access management and security, as it is all done automatically.

.env files

A .env file is a method for secure storage of things like keys and passwords specific to the application. For instance, to connect to my PostgreSQL database, I need to provide the database name, my username, my password, the host, and the port number. All of this is sensitive information, and it is best if it is all stored in one spot instead of being provided as raw values in every `psycopg2.connect()` function. By storing these values in a .env file, I will be able to ensure that this data is not accessible to the public, and that it is not used with malicious intent. Another benefit of storing all the values in one spot is that if I need to make a change, I just change it in the .env, and this change is reflected in the whole program when it extracts the values from the .env.

Refresh Token Rotation

While Refresh Tokens are already in use, it can be unsafe to use just one per session. This is because an attacker could potentially steal this refresh token and use it for their own session and therefore access the user's account and data. To stop this, we can use something called Refresh Token Rotation, which is where every time an access token expires, during the retrieval of a new one, we check that the refresh token sent over matches the previous one assigned, which was stored in the database as an attribute of the user. If they match, then we generate a new refresh token and access token, sending them back and updating the database so that the new refresh token is stored there correctly. However, if they do not match, we can direct the user back to the login page for them to sign in again. This feature is a vital line of defence which is often overlooked, so implementing this feature means that attackers will not be able to use old tokens to gain access.

Hiding Passwords in Inputs

To ensure that no one can steal a password by looking over someone's shoulder, or seeing it during a screen share/recording, inputting the password will cause it to show up as dots. With React, this is very simple. All we need to do is to set the type of the `<input></input>` element to "password". This hides the password for us.

Objectives

After the completing my research on the topics I deem necessary, I now know that my application should:

1. Use a graphical interface.
2. Be a website, accessible through a web browser.
3. Use React for creation of all the website components and content.
4. Make sure that the list of appointment dates is given in a calendar-like (grid instead of list) format.
5. Send email reminders about the appointment a day before and a week before to patients, and 30 minutes before for doctors/nurses.

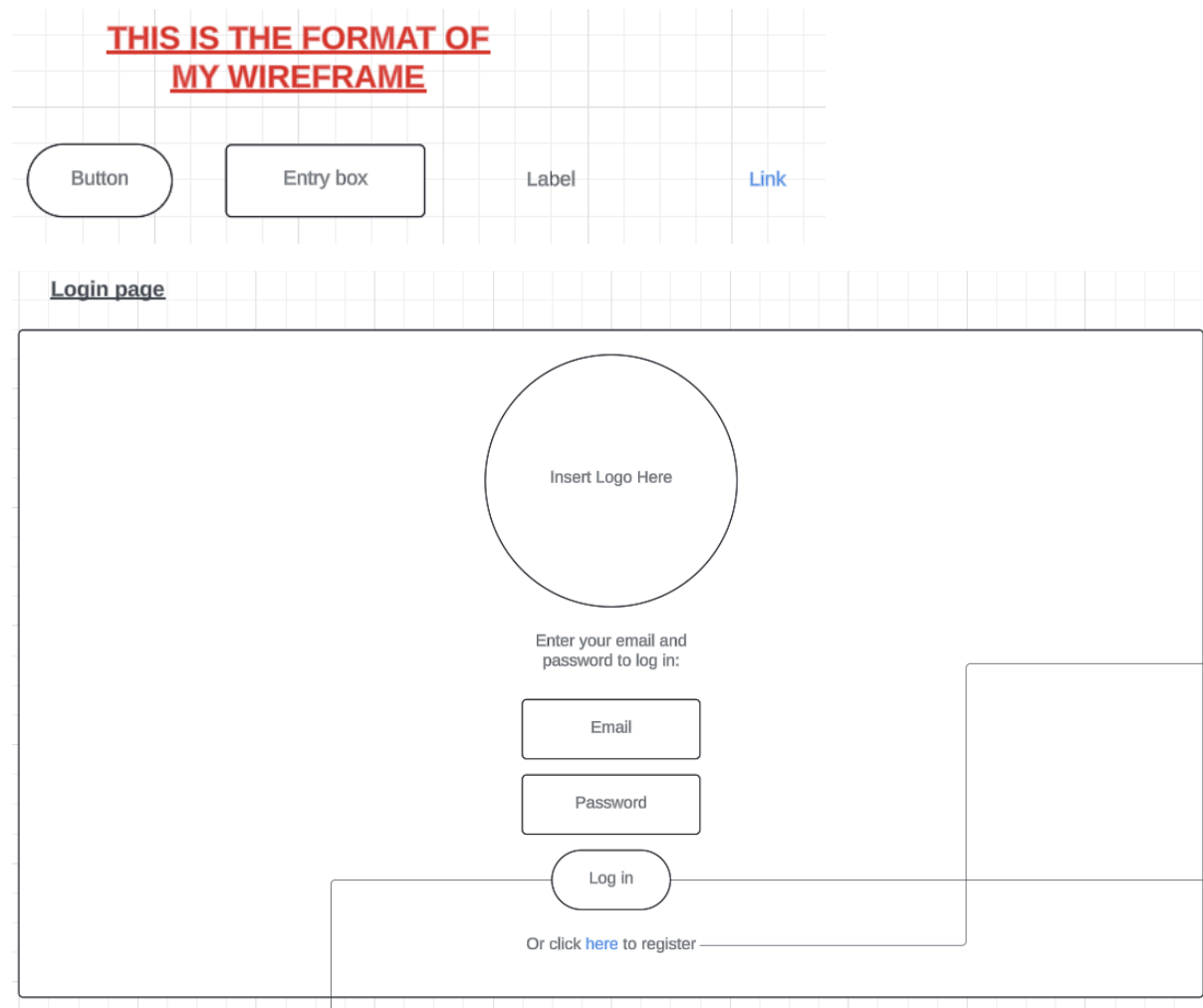
6. Be very simple to use and understand.
7. Make sure staff cannot see other staff information/appointments.
8. Make sure there is an appointment log associated with the patient and staff that were present.
9. Give patients the ability to create appointments.
10. Make sure that staff cannot create appointments.
11. Make sure that patients should be able to register with their full name, email address, and their date of birth.
12. Make sure that staff should be able to register the same way, but before they gain access an IT admin has to manually verify the member.
13. Make sure that the scheduling system should use a priority-based method.
14. Make sure that patients should be rescheduled a maximum of 2 times and gain a priority level every time they are.
15. Make sure that patients should have a maximum of 3 appointments booked in the future at once.
16. Make sure that appointment timeslots must start at 9:00 and end at 21:00, and each appointment must be 30min long.
17. Use JWT Access Tokens and Refresh Tokens.
18. Use Refresh Token Rotation.
19. Use Fetch API in conjunction with async/await.
20. Use a Flask Web Server for the backend.
21. Have an established connection with a dedicated PostgreSQL database.
22. Utilise password hashing.
23. Utilise a .env file for secure storage of important and confidential values.

Design

For all of my designing, I utilised the software known as Lucidchart [12].

Wireframes

I have made a lot of wireframes for every single page that will exist. Some of these pages are exclusive to staff or patients, and some will be shared.



The login page is where patients and staff both log in. It is very simple. There is also a link leading to the registration page below.

Registration page

The registration page wireframe features a central logo placeholder at the top. Below it, there are two columns of input fields. The left column includes fields for 'Email', 'Password', and 'Confirm password', with a note specifying password requirements. The right column includes fields for 'First name', 'Surname', 'Date of birth' (with a dropdown menu), and 'Phone number'. At the bottom, there is a 'Register' button, a link to 'Or click here to log in', and a 'Register as staff' button.

Insert Logo Here

Enter your email:

Enter your first name:

Enter your password:

Enter your surname:

(Your password must be larger than 10 characters and include capitals, numbers, and special characters)

Enter your date of birth:

Confirm password:

Enter your number: (optional)

Or click [here](#) to log in

Here, you provide more details than in the login page, such as your name, date of birth, and your number. A normal patient will then click register, while a staff member will click the register as staff button. To adhere to the objectives and make sure patients cannot register as staff, they are then brought to this page:

Staff confirmation page

The staff confirmation page wireframe features a central logo placeholder at the top. Below it, there is a large text area with a message: 'Please wait for your supervisor to approve your registration. When it is approved, you will be able to log in by going back to the login page.'

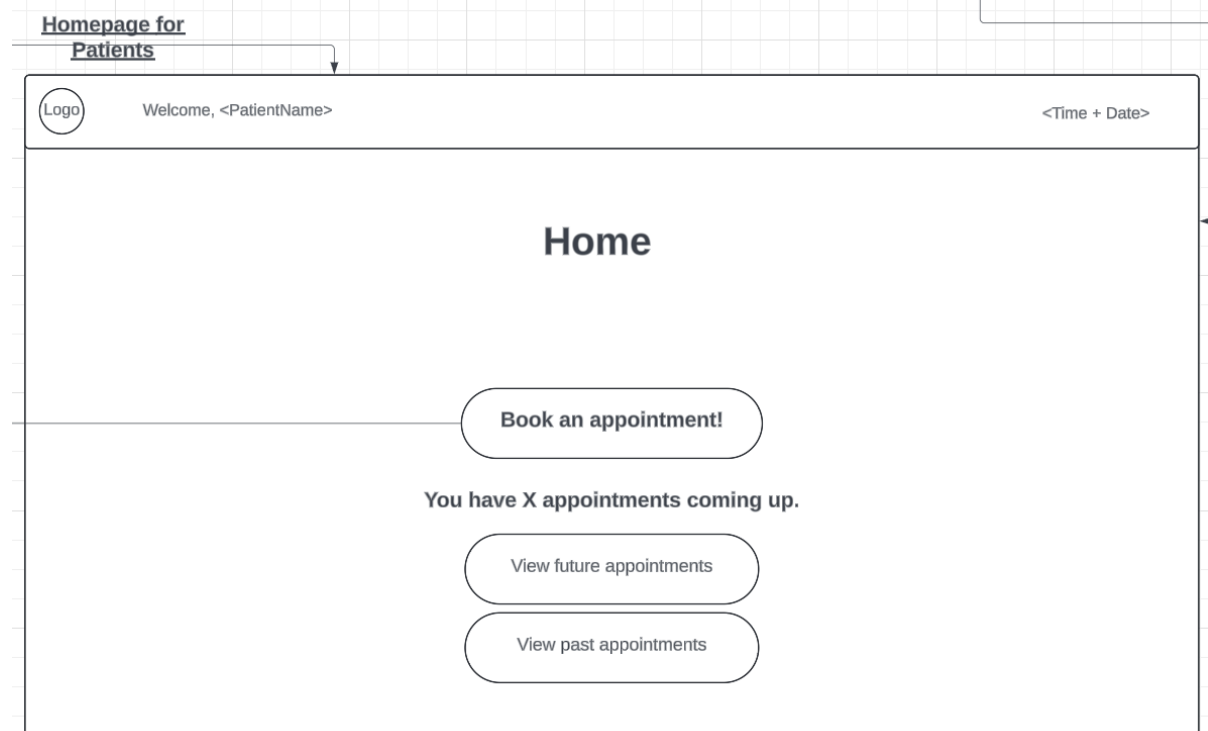
Insert Logo Here

Please wait for your supervisor to approve your registration.

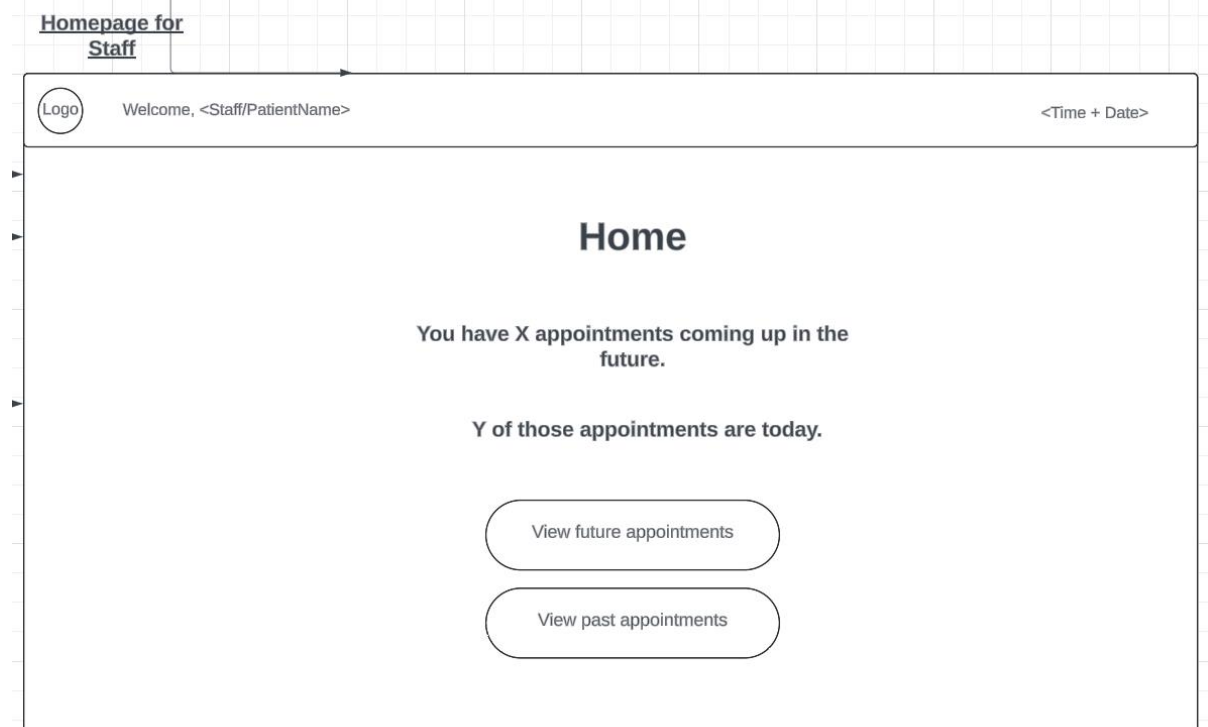
When it is approved, you will be able to log in by going back to the login page.

An IT administrator in charge of their server will manually use a .pgsql script to approve the new staff member once their identity has been verified.

Then there are the homepages for staff and patients. A patient homepage looks like this:



A staff homepage looks like this:



As you can see, both users have a label at the bottom giving the current time. Staff also have a timer till their next appointment with a patient. The difference is that staff cannot

book appointments, they can only view upcoming appointments. Viewing future and past appointments will show a page like this:

Past Appointments
for Patients

Logo

Welcome, <PatientName>

Appointments

Prescriptions

Message

Medical History

Settings/Account

<Past Appointment Details>

<Past Appointment Details>

<Past Appointment Details>

Previous Page

Page X of N

Next Page

<Time + Date>

This looks the same for future appointments, for staff and patients alike, with very minor changes. Pressing next/previous page will cause the frontend to request data from the backend to display the next 3 appointments.

The booking of an actual appointment has multiple sections. Upon pressing book, the patient is brought to this page:

Szymon Galutowski

Candidate No. 6528

Centre No. 12290

31

Appointment
booking page for
patients 1

Back

This is to
determine
priority
level

Reason for
booking

Why do you want to
book this appointment?

sympton 1	sympton 7
sympton 2	sympton 8
sympton 3	sympton 9
sympton 4	sympton 10
sympton 5	sympton 11
sympton 6	sympton 12

Continue

<Time + Date>

This is just a selection of possible reasons, and I have done some research in my analysis on which other reasons I could include and their priority levels.

The next section is date selection:

Appointment booking
page for patients 2

If a date it
completely full it
should be greyed
out or have a
different colour.

1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31	1	2	3	4

<Current month view>
PrevNext

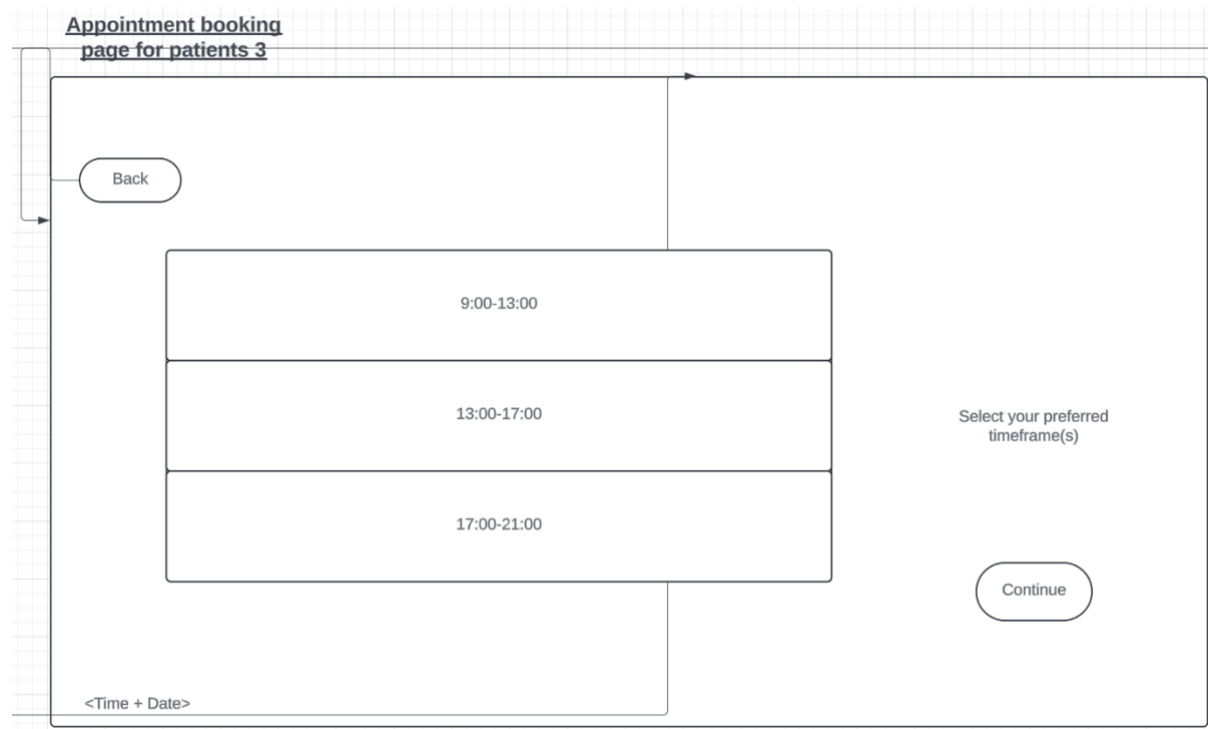
Select your preferred
days

Continue

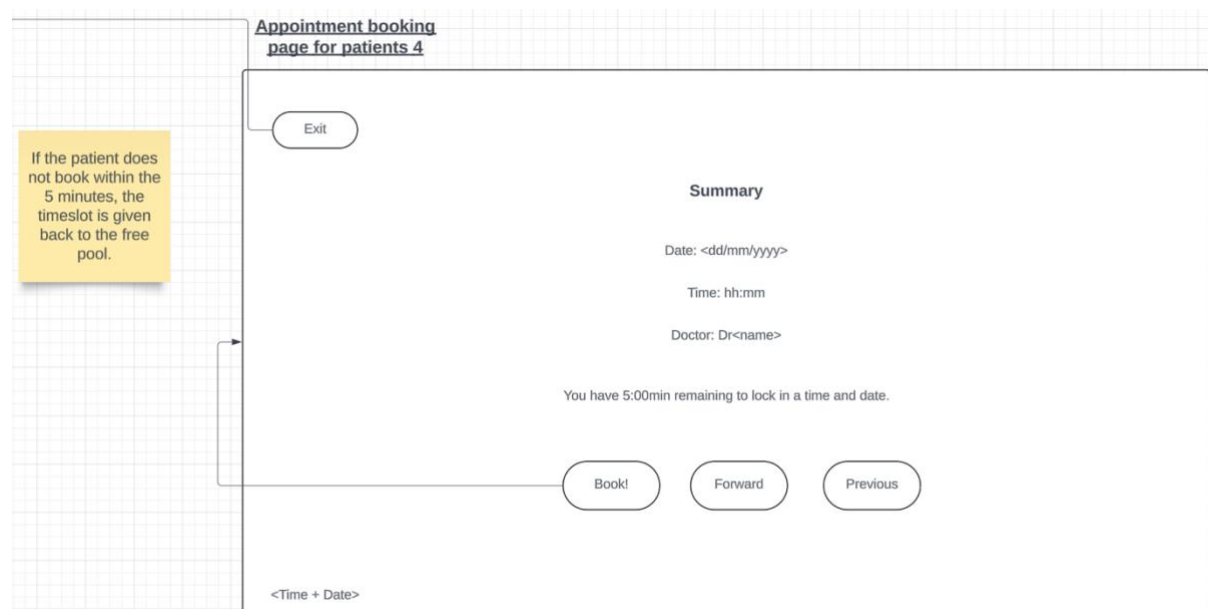
<Time + Date>

Patients can select multiple dates in different months. Then they select their preferred

timeframes:



This is so that they do not get assigned an appointment during a time that does not work for them and gives them a bit more freedom of choice. Once they have selected their times, they press continue and after the backend algorithm has done it's job, the patient is brought to this page:



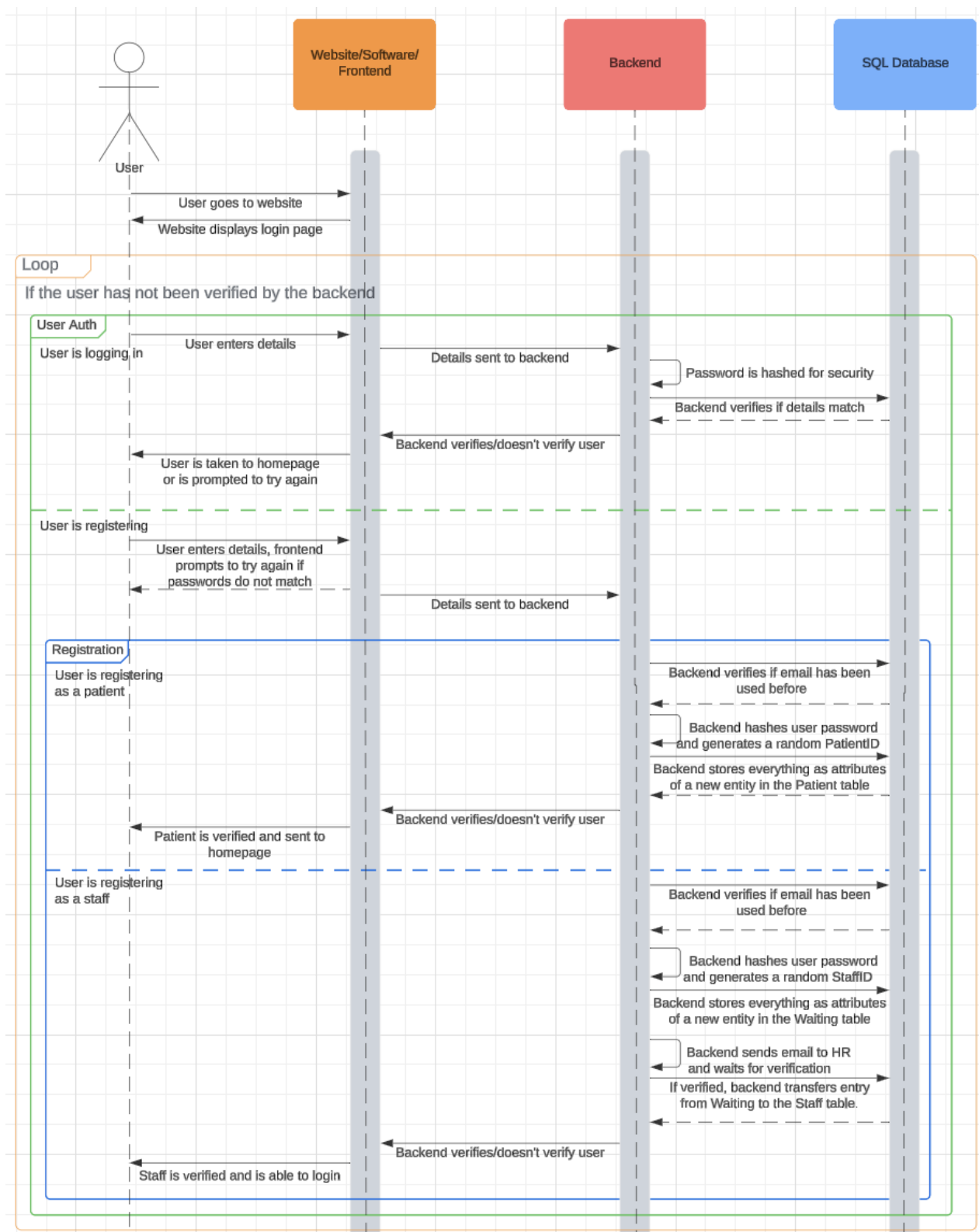
As you can see in the notes, if the patient does not book within 5 minutes, their booking session is cancelled, and their appointments are returned to the free pool. They can also press Exit to go back to the homepage, and this action also cancels the session and makes sure the appointments are returned to the free pool in the backend. Once

they press book, the database is cleaned up, (unchosen appointments are updated/deleted,) their appointments is booked, and they are taken to the homepage.

UML Sequence Diagrams

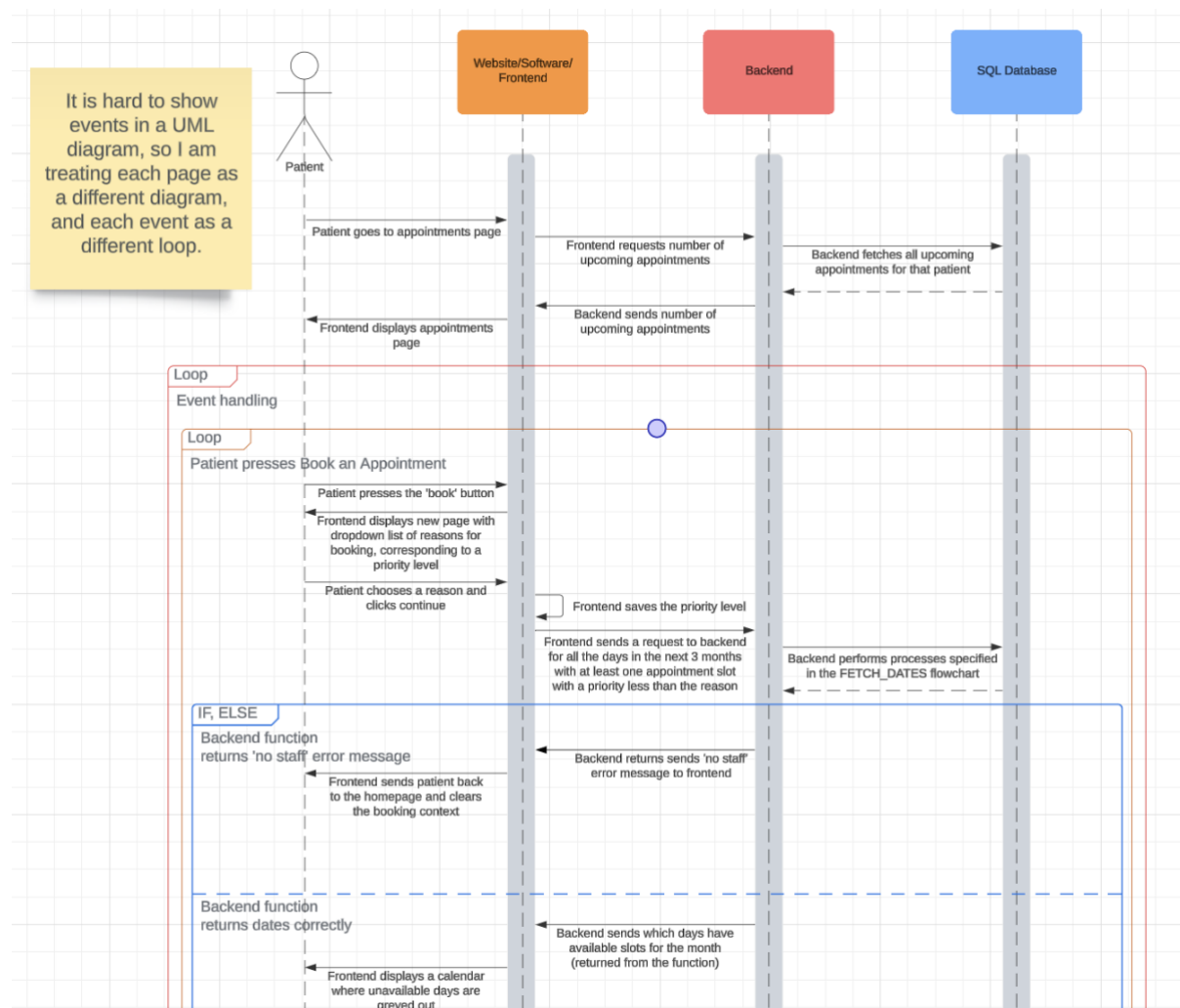
I have created a few UML Sequence Diagrams to illustrate the flow of data throughout the program. Some of the processes I deemed important were the registration/login, the appointments page, the patient lookup system, and the changing of details in account settings. The reason I chose to use sequence diagrams instead of something like a flowchart was because sequence diagrams are more effective in showing the flow of data, not just the processes. Event handling was a little tough to express with the UML sequence diagram symbols, but I used the loop/conditional containers to show this, used them together to show different actions and conditions. Where a loop finished, I specified, so that it would be easy to understand the processes. I also colour coded the different containers for added simplicity, as the diagrams can get quite complex, and so I used colours to distinguish between the 'layer' of container.

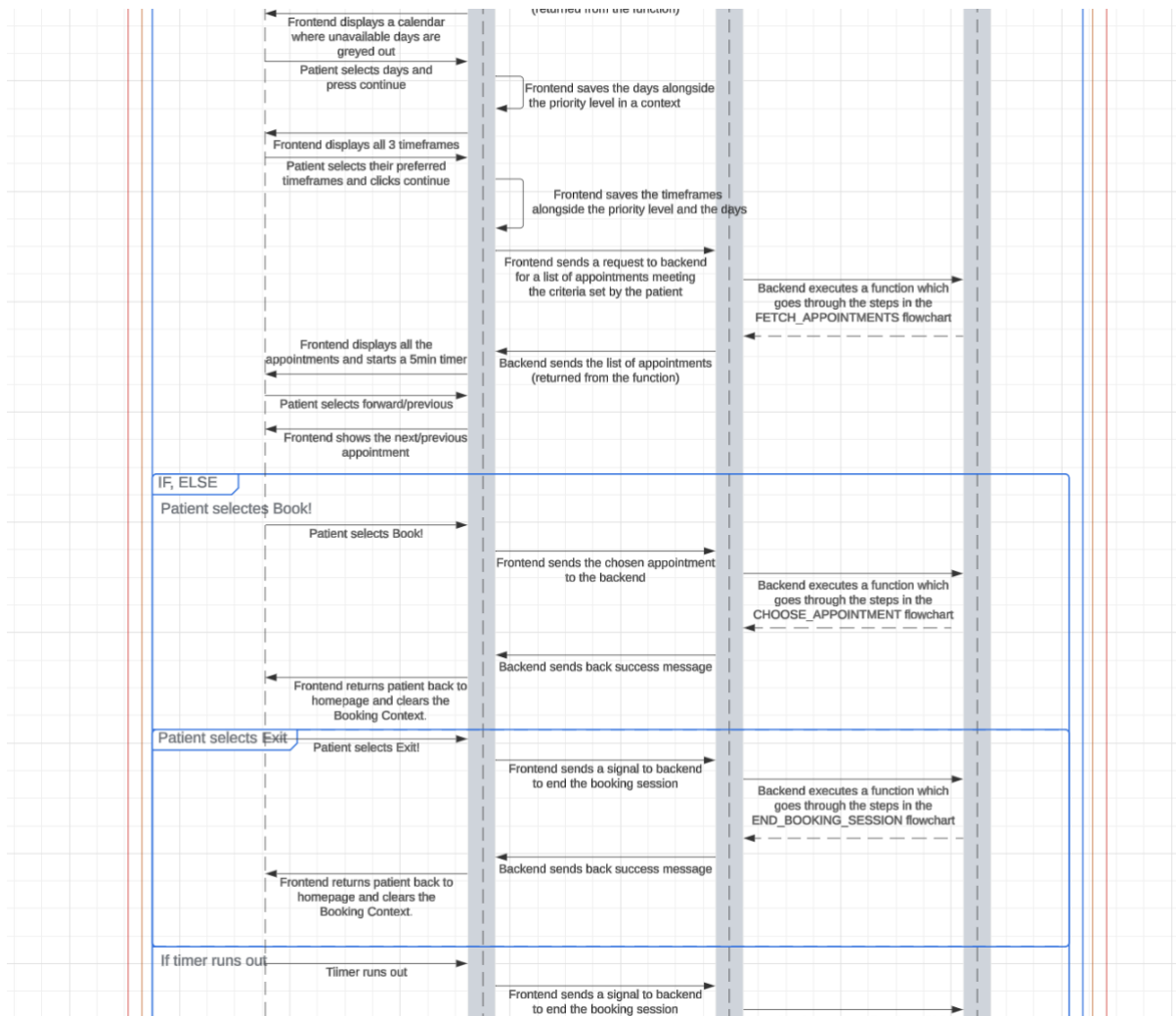
Registration/Login

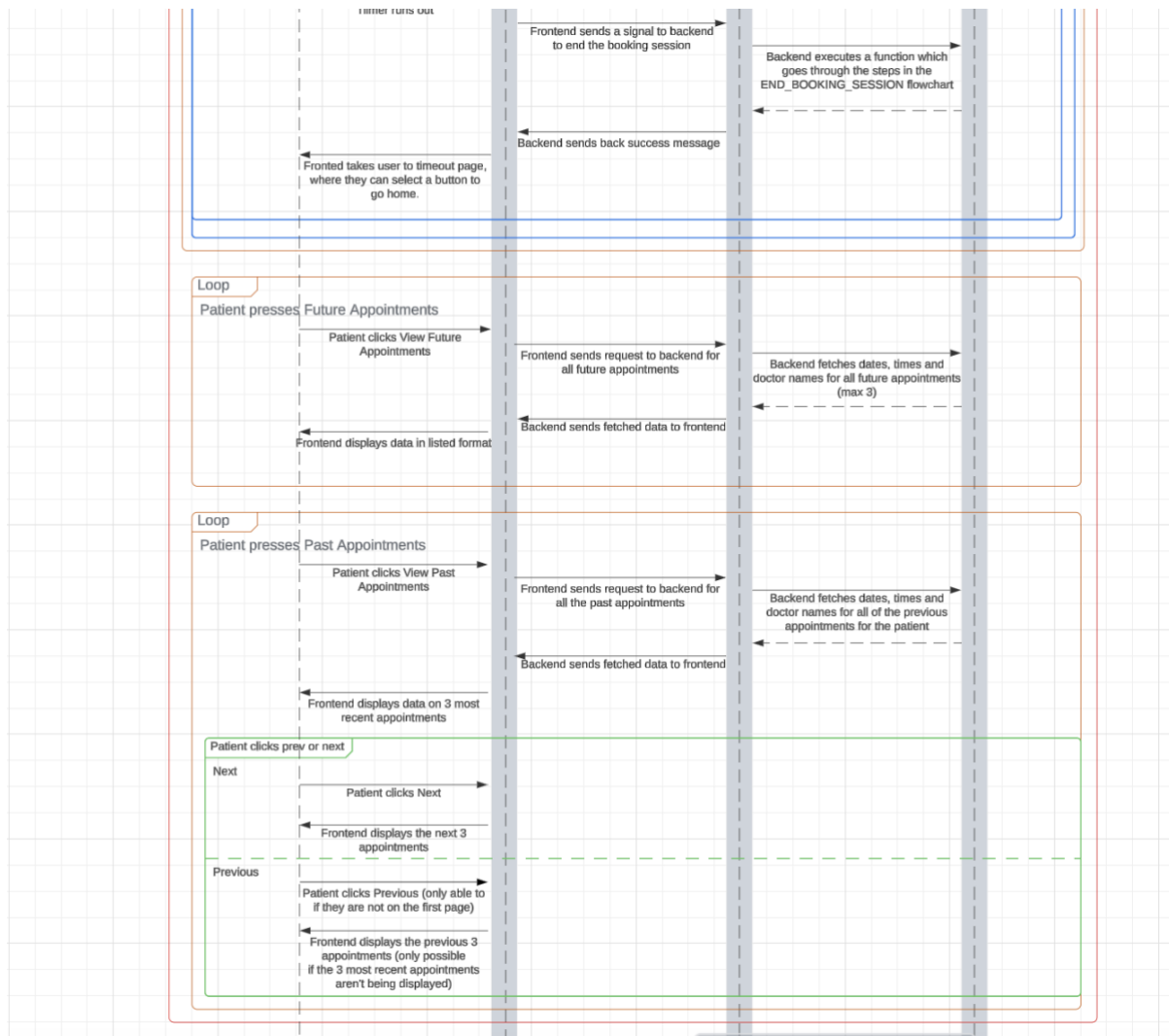


Home

This is my main UML diagram which shows all the possible processes that occur from the user entering the homepage.

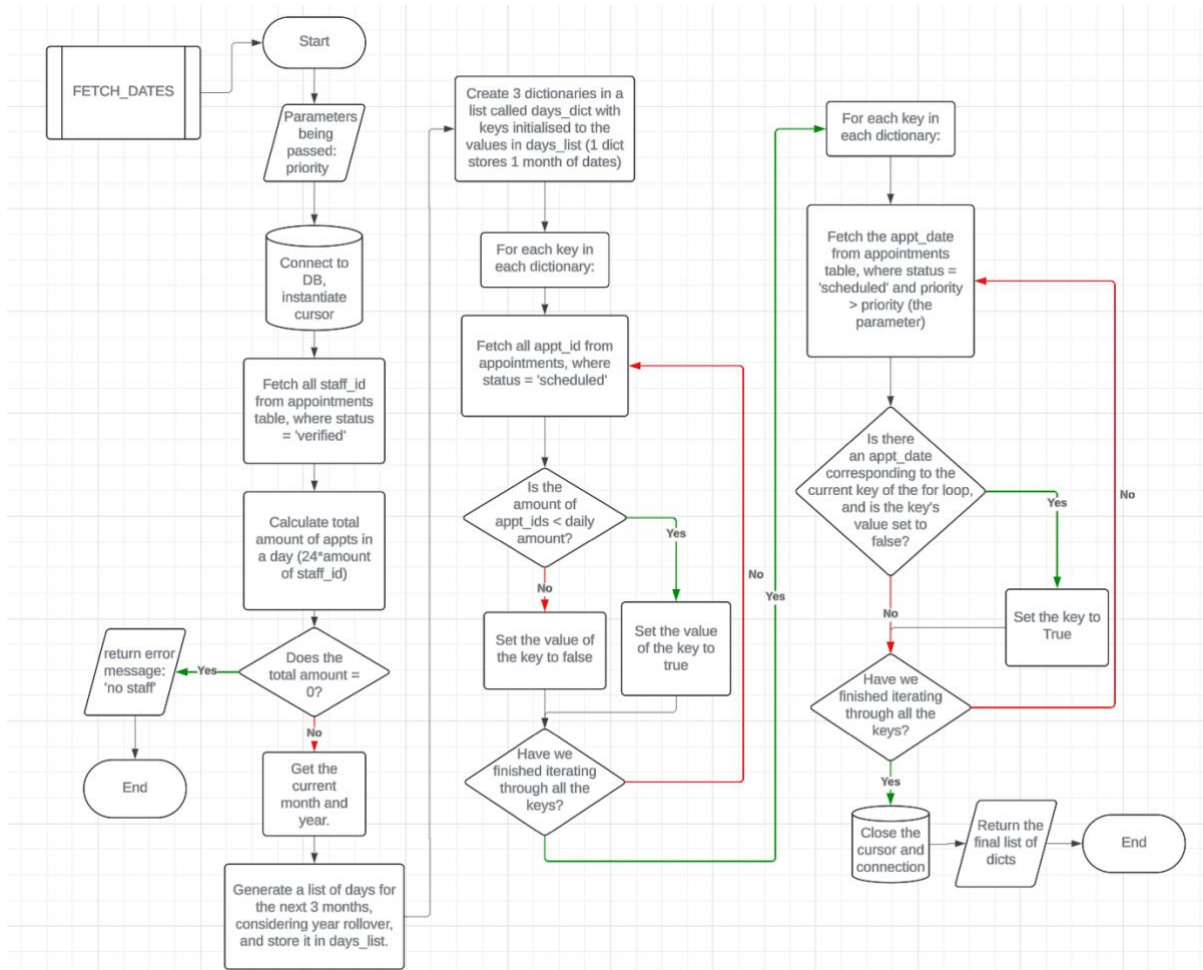


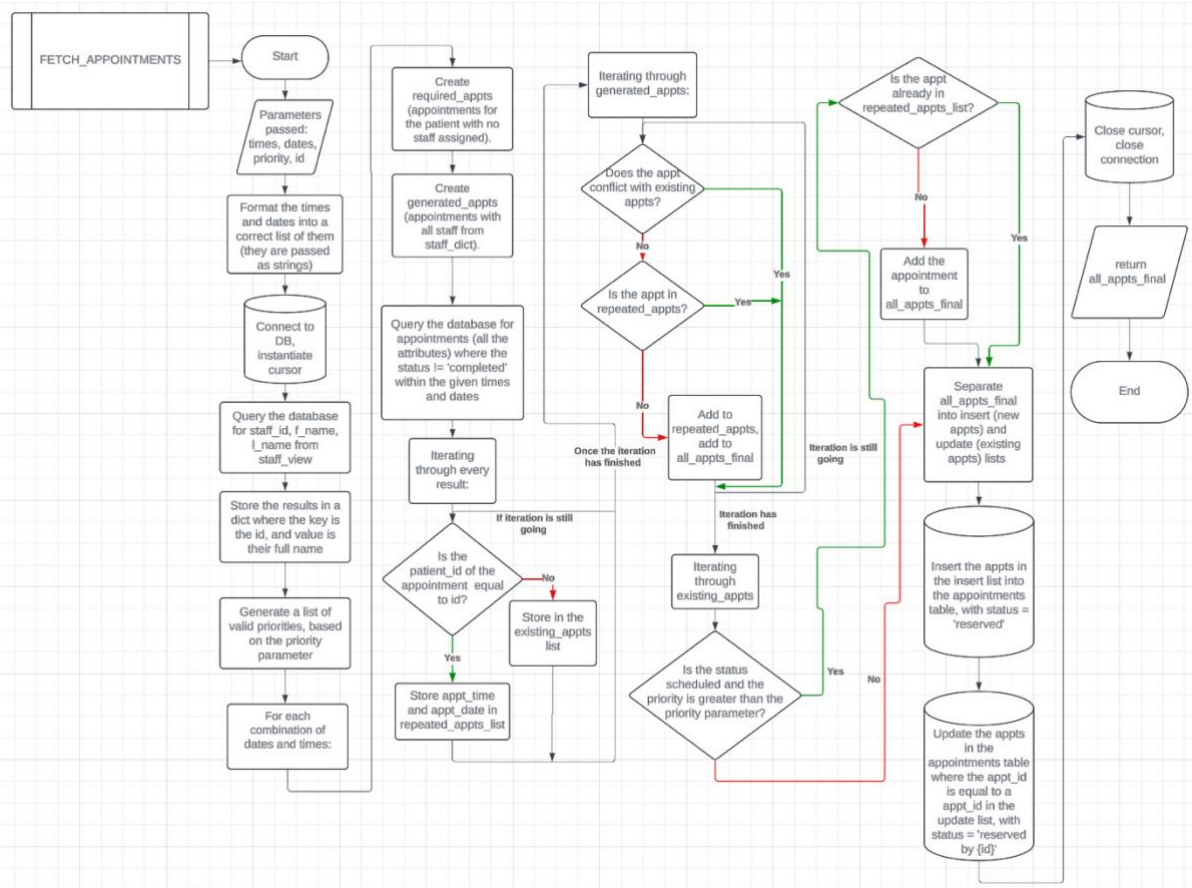


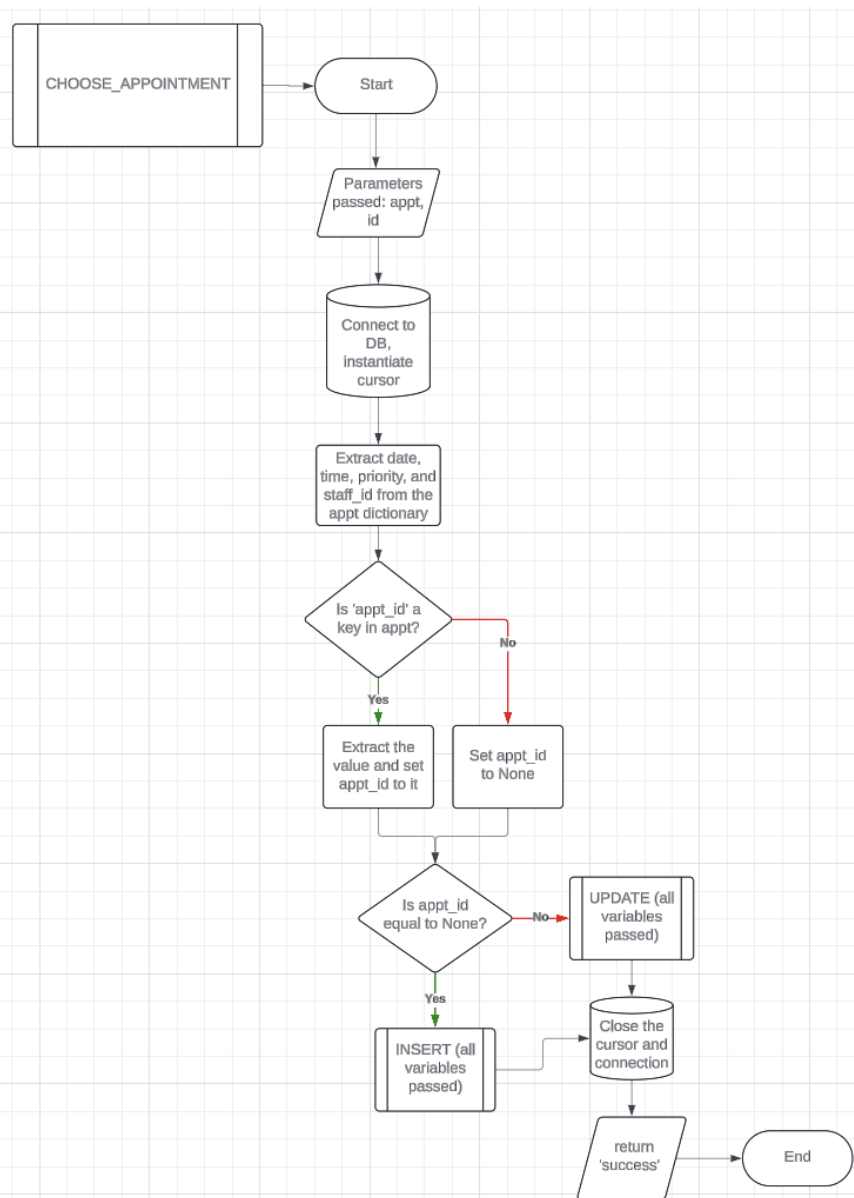


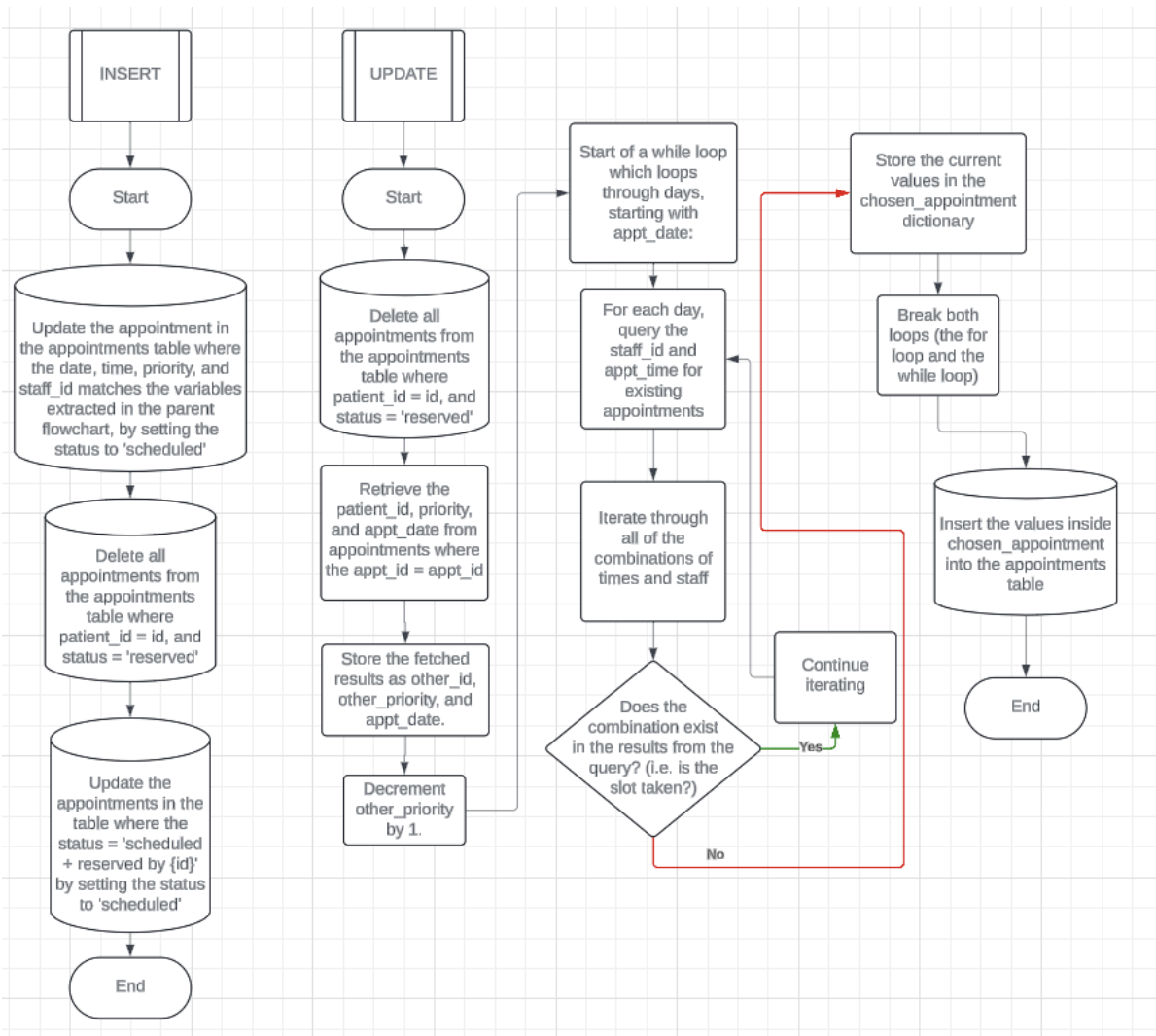
Flowcharts

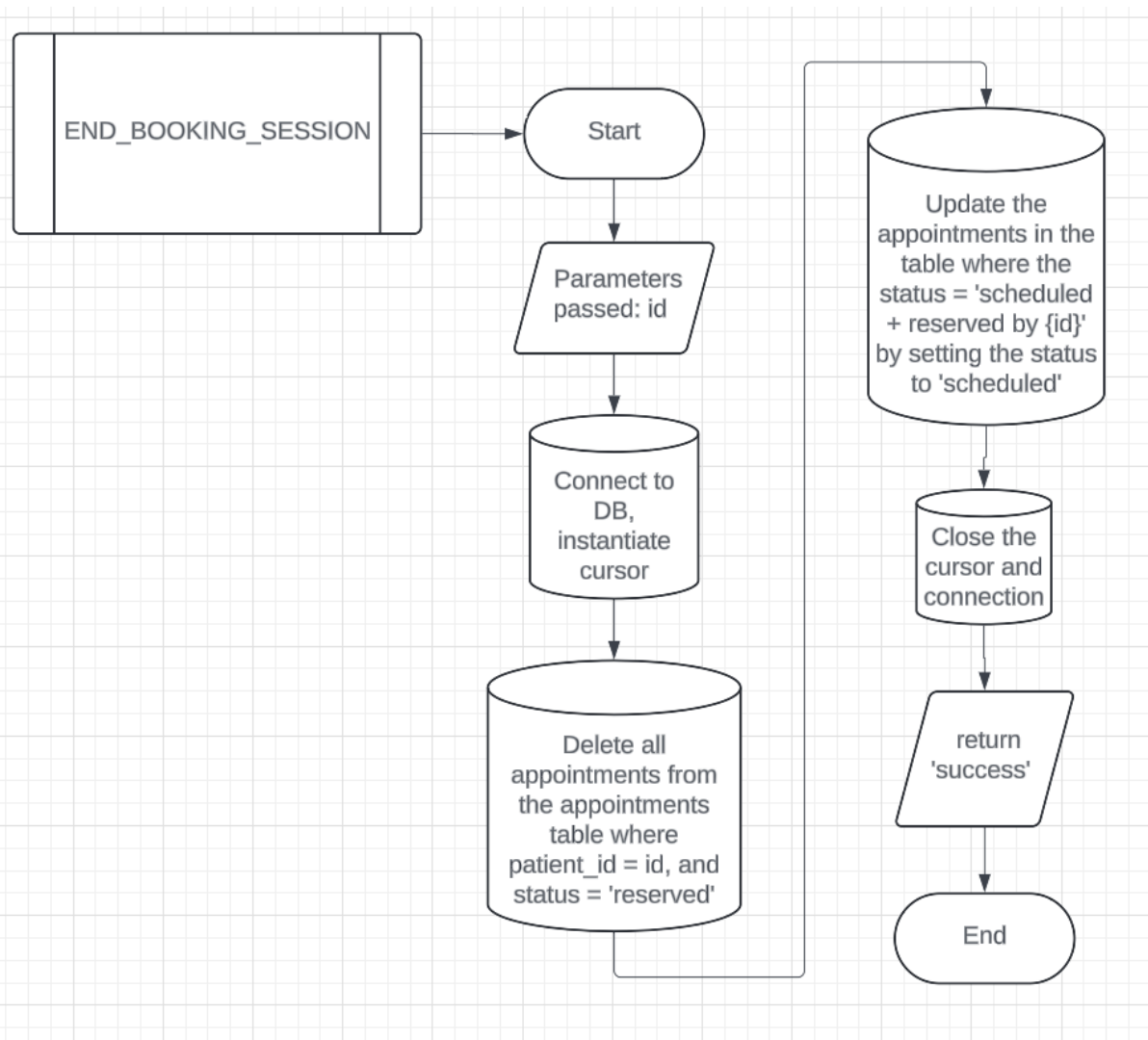
These are all my flowcharts. They are referenced in the UML Sequence Diagrams above and show how some of the more complex backend processes occur. By using flowcharts, I have been able to decompose these processes into smaller and more manageable steps. This has helped me to identify potential issues in my processes, which were then fixed accordingly.











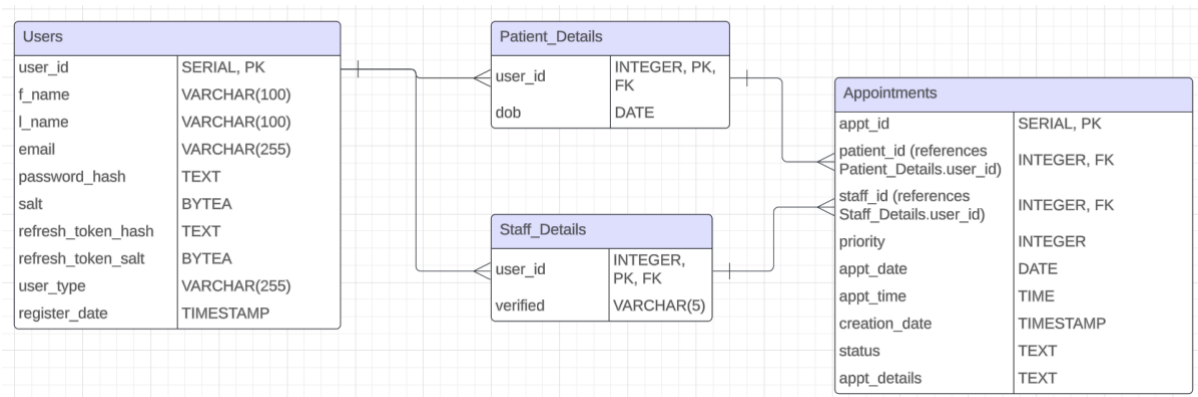
Database diagrams

In my opinion, the best way to explain the inner workings of a database is through an Entity Relationship Diagram (ERD). This shows the relationships between entities and attributes, and can display the use of primary/foreign keys very well. There are two main notation types, Chen's and Crow's Foot. Chen's notation is very widely used; however it can be very confusing due to the large volume of shapes and lines. I chose to use Crow's Foot notation as it is much easier to understand – you have 1 column tables with each row representing a different attribute, symbols in those rows for primary and foreign keys, and simple lines connecting attributes together. PGAdmin4, the software I am using for database management, can automatically create ERD diagrams based on the SQL scripts I write. However, I dislike the format they are in, and I would prefer to use Crow's Foot, which is why I am redrawing it in a simpler format.

The SQL script I used to create my database is in my technical solution, under the [PostgreSQL Scripts](#) header.

It creates my 4 tables, as well as two views for easier recollection of data. The schema is normalised in 3rd Normal Form (3NF). This is because atomicity is maintained, there are no partial dependencies, and all non-key attributes depend on nothing but the entire key, thus meaning the schema is in 3NF. However, this makes it a bit more challenging to select all the data on a specific user with one select statement. If I wanted all the data on a patient, this means I need two select statements – one for the data in the Users table, and one for the data in the Patient_Details table. However, in this case I have created patient and staff views, which essentially group all the data into a virtual table. This is not violating any of the requirements for BCNF, and it allows me to retrieve data more efficiently. Another method for doing this is by using INNER JOIN during a SELECT query. By using INNER JOIN, you temporarily combine attributes from two tables based on whether certain attributes match, e.g. the primary and foreign key. Some attributes in my tables have predefined values they can take. I am using a Check statement to make sure this is kept the same.

When I ran the script, the all the tables were created. PgAdmin4 automatically created an ERD diagram, but due to the reason specified earlier, I then created my own:



Technology Stack

My tech stack will consist of:

- React.js
- JavaScript
- Python
- PostgreSQL

React.js is the JavaScript framework which I will be using to make the GUI. JavaScript is the language I'll be using to code some functions for the frontend, and to write the middleware. Python is the language for my entire backend, and I'll be using it to process data and fulfil requests. PostgreSQL will be my database, and it will be accessed using psycopg2, a python library.

Role-based Access Control

As per my objectives, there are two types of user; a staff member and a patient. Certain resources are restricted to only user type. For instance, as shown in the previous wireframes, and as specified in my objectives, the patients are the only people able to book an appointment. They are also able to look at their future appointments and their past appointments, which staff members are also able to do, but the booking of an appointment is restricted to patients only.

JWT storage

I will be storing the JWTs in Session Storage, which is a browser feature which allows us to store data for the duration of a single browsing session. The Session Storage can be viewed using Chrome Dev Tools, which will be useful for developments purposes and debugging, however this means it is very unsecure and vulnerable to attacks. I would like to be able to store the JWTs in cookies, as Flask has methods for this, but doing so would result in a requirement for setting up an HTTPS connection with certificates, which is beyond the scope of this project. Doing so would result in a deviation from the

current objectives, and there are other security features in place which could combat this vulnerability, such as a short access token lifetime and refresh token rotation.

Contexts

I am going to have two contexts used in my website. Contexts, as mentioned earlier, are a React.js feature which allows one to store a value, or a dictionary of values across multiple pages. My two contexts will be:

- Cookies: Holds a dictionary which holds general information:
 - The user type
 - The user's name
 - The prevPageStack list
 - The name of the current page
 - The backUsed bool (For checking if the user has used a back button)
 - The authBool bool (For checking if the user is authenticated)
- Booking: Holds a dictionary which holds:
 - The priority level
 - The reason for booking
 - A string of all the chosen dates
 - A string of the chosen timeframes

Back Button

My website will have a back button on certain pages for easier navigation. To make this work, I have designed two processes that occur. These two processes utilise a stack implementation, which I have designed myself using simple OOP.

The stack attributes include:

- Items: all the items the stack contains, stored in a list.

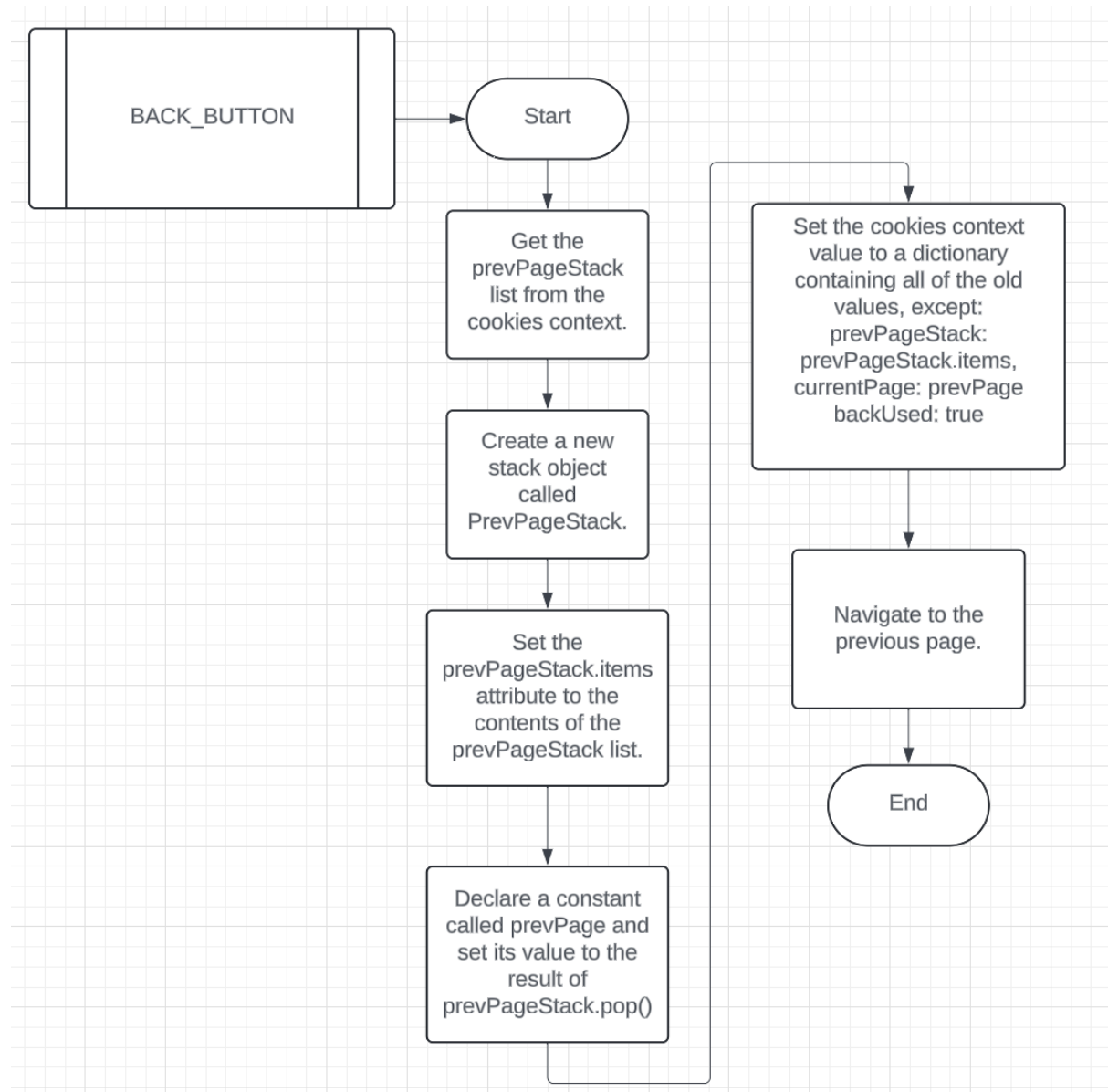
The stack methods include:

- isEmpty(): checks whether the list/stack is empty.
- Push(val): Pushes the value stored in val to the top of the list/stack.
- Pop(): Removes the value stored in the top of the list/stack, and returns it.
- Peek(): Returns the value stored in the top of the list/stack, without removing it.
- Purge(): Removes all of the values stored in the list/stack.

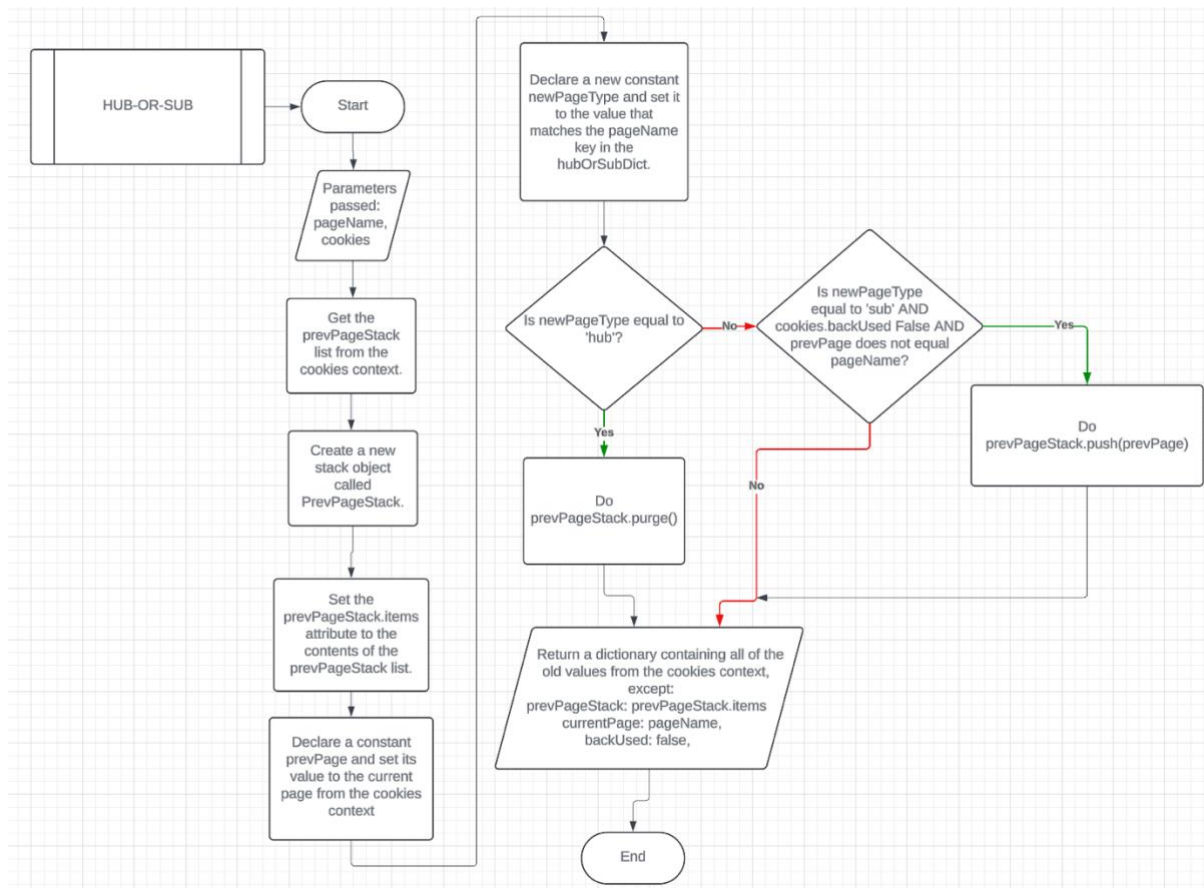
Since there is no structure with multiple classes, I will not draw a class diagram for this as there are no relationships to show.

The reason I have used a stack implementation is because stacks are perfect for undo operations, or back buttons. This is because as a LIFO data structure, I can check what the previous page was by peeking/popping, and I can add the next page by pushing.

The actual component is the back button. This is the flowchart detailing its process:



The second process is a function called hubOrSub. This is a function which is called upon the opening of every single page where the user requires authentication to visit. When calling the function, the name of the current page and the current value of the Cookies context is passed. The output of the function is set as the new value of the Cookies context. This flowchart goes into detail about its process:



Technical Solution

Frontend

This is my entire frontend, which in my case is everything that makes up the webpage. This will also include my Fetch API calls, which are a JavaScript built in function, so I will be putting them in here instead of making a dedicated middleware section.

Written Files

1 - App.js

This file contains all the routing information, the context routing, and is where all the page components are imported into. It is the file that is ran upon a user typing in the URL, which in my case is localhost:3000.

```

import './App.css';
import { BrowserRouter as Router, Routes, Route, Navigate } from "react-router-dom";
import { CookiesProvider, BookingProvider } from "./userContexts";
import {Login, Register, StaffApproval} from "./3 - login"
import { PatientHome, StaffHome, FutureAppts, PastAppts } from './2 - home';
import {BookAppointment1, BookAppointment2, BookAppointment3, BookAppointment4, Timeout } from './5 - appointments';
  
```

```

export default function App() {
  return (
    <CookiesProvider>
      <BookingProvider>
        <Router>
          <Routes>
            <Route path="/" element={<Navigate to="/login" />} /> />
            <Route path="/null" element={<Navigate to="/login" />} />

            <Route path="/login" element={<Login />} />
            <Route path="/register" element={<Register />} />
            <Route path="/staffApproval" element={<StaffApproval />} />
            <Route path="/home" element={<PatientHome />} />
            <Route path="/StaffHome" element={<StaffHome />} />
            <Route path="/BookAppointment1" element={<BookAppointment1 />} />
            <Route path="/BookAppointment2" element={<BookAppointment2 />} />
            <Route path="/BookAppointment3" element={<BookAppointment3 />} />
            <Route path="/BookAppointment4" element={<BookAppointment4 />} />
            <Route path="/FutureAppointments" element={<FutureAppts />} />
            <Route path="/PastAppointments" element={<PastAppts />} />
            <Route path="/timeout" element={<Timeout />} />

          </Routes>
        </Router>
      </BookingProvider>
    </CookiesProvider>
  );
}

```

2 – home.js

This file contains both homepages, one of which is for patients, and the other is for staff members. There is a need for different pages, since staff members shouldn't be able to book appointments. It contains a function call to an imported function known as `hubOrSub()`; in fact, all pages do, and this function will be covered in `smallComponents.js`.

```

import './App.css';
import './smallComponents.js'
import React from "react";
import {useState, useEffect, useRef} from "react";
// eslint-disable-next-line
import { BrowserRouter as Router, Routes, Route, Navigate, Link, useNavigate } from "react-router-dom";
import { Back, hubOrSub } from './smallComponents.js';
import { TopBar } from './4 - topBar.js'
import { useCookies } from './userContexts.js';
import { fetchFuture, fetchPast, fetchApptAmount } from './ApiCalls.js';

export{
  PatientHome,
  StaffHome,

```

```

    PastAppts,
    FutureAppts
  }

function PatientHome() {

  const navigate = useNavigate();
  const hasRun = useRef(false);
  const [ appointmentAmount, setAppointmentAmount ] = useState('X')

  let { cookies, setCookies } = useCookies();
  useEffect(() => {
    if (!hasRun.current) {
      if (!cookies.authBool | !sessionStorage.getItem('jwt')) {
        console.log('going back to login')
        navigate('/login')
      };
      setCookies(hubOrSub('home', cookies));
      hasRun.current = true
      fetchApptAmount(cookies.type, (callback) =>{
        if (callback === 'login again') {
          setCookies()
          navigate('/login')
        }
        setAppointmentAmount(callback)
        console.log(callback)
      })
    }, [cookies, setCookies, navigate])

    //API STUFF TO GET APPOINTMENT AMOUNT WOW! (in a useEffect)

    const sendToBook = (event) => {
      if (appointmentAmount < 3){
        console.log('sendToBook')
        navigate('/BookAppointment1');}
    }

    const sendToFuture = (event) => {
      console.log('sendToFuture')
      navigate('/FutureAppointments')
    }

    const sendToPast = (event) => {
      console.log('sendToPast')
      navigate('/PastAppointments')
    }

    return(
      <>
      <TopBar/>

```

```

        <div style={{ position: "relative", width: "100%" }}>
          <div className="centerPage">
            <h1>Home</h1>
            <div className="buttonDiv" onClick={sendToBook} style={{ 'pointerEvents': appointmentAmount<3 ?
"auto" : "none", 'cursor': appointmentAmount<3 ? "pointer" : "not-allowed" }}>
              <h2>Book an Appointment</h2>
            </div>
            <h3>You have {appointmentAmount} appointment(s) coming up in the future.</h3>
            <div className="buttonDiv" onClick={sendToFuture}>
              <h2>View Future Appointments</h2>
            </div>
            <div className="buttonDiv" onClick={sendToPast} >
              <h2>View Past Appointments</h2>
            </div>
          </div>
        </div>
      </>

    );
  }

function StaffHome() {

  const navigate = useNavigate();
  const hasRun = useRef(false);
  const { cookies, setCookies } = useCookies();
  const [ appointmentAmount, setAppointmentAmount ] = useState(0);
  const [ amountToday, setAmountToday ] = useState(0);

  useEffect(() => {
    if (!hasRun.current) {
      if (!cookies.authBool | !sessionStorage.getItem('jwt')) {navigate('/login')};
      setCookies(hubOrSub('StaffHome', cookies));
      hasRun.current = true
    }
    fetchApptAmount(cookies.type, (callback) =>{
      if (callback === 'login again') {
        setCookies()
        navigate('/login')
      }
      setAppointmentAmount(callback[0])
      setAmountToday(callback[1])
    })
  }, [cookies, setCookies, navigate])

  //API STUFF TO GET APPOINTMENT AMOUNT WOW!

  const sendToFuture = (event) => {
    console.log('sendToFuture', cookies.type)
  }

```

```

        navigate('/FutureAppointments')
    }

    const sendToPast = (event) => {
        console.log('sendToPast', cookies.type)

        navigate('/PastAppointments')
    }

    return(
        <>
        <TopBar/>
        <div style={{ position: "relative", width: "100%" }}>
            <div className="centerPage">
                <h1>Home</h1>
                <h3>You have {appointmentAmount} appointment(s) to tend to in the future.</h3>
                <h3>{amountToday} of those appointments are today.</h3>
                <div className="buttonDiv" onClick={sendToFuture}>
                    <h2>View Future Appointments</h2>
                </div>
                <div className="buttonDiv" onClick={sendToPast}>
                    <h2>View Past Appointments</h2>
                </div>
            </div>
        </div>
        </>
    );
}

function FutureAppts(){

    const navigate = useNavigate();
    const hasRun = useRef(false);
    const { cookies, setCookies } = useCookies();
    const [ fullFutureAppts, setFullFutureAppts ] = useState([]);
    const [ index, setIndex ] = useState(0)
    const [ currentFutureAppts, setCurrentFutureAppts ] = useState([]);
    let buttons = <></>

    useEffect(() => {
        if (!hasRun.current) {
            if (!cookies.authBool | !sessionStorage.getItem('jwt')) {navigate('/login')};
            setCookies(hubOrSub('FutureAppointments', cookies));
            hasRun.current = true
            fetchFuture(cookies.type, (callback) => {
                if (callback !== 'error'){
                    let temp = []

```

```

        for (let n = 0; n < (callback.length/3); n++) {
            temp.push(callback.slice(n*3, 3*n+3))
        }
        setFullFutureAppts(temp)
        setCurrentFutureAppts(temp[0])
    } else if (callback === 'login again') {
        setCookies()
        navigate('/login')
    }
})

}, [cookies, setCookies, navigate])

}

const handleNext = (event) => {
    if (index < fullFutureAppts.length-1){
        setIndex((index) => index + 1)
        setCurrentFutureAppts(fullFutureAppts[index+1])
    }
}

const handlePrevious = (event) => {
    if (index > 0){
        setIndex((index) => index - 1)
        setCurrentFutureAppts(fullFutureAppts[index-1])
    }
}

if (fullFutureAppts.length <= 1){
    buttons = <></>
} else {
    buttons = <div style={{'display':'flex', 'flexDirection':'row', 'gap':'200px'}}>
        <div className={'buttonDiv'} onClick={handlePrevious}><p>Previous</p></div>
        <div className={'buttonDiv'} onClick={handleNext}><p>Next</p></div>
    </div>
}

if (!currentFutureAppts.length) {
    return(<>
        <TopBar/>
        <div style={{ position: "relative", width: "100%" }}>
            <Back/>
            <div className="centerPage">
                <h1>Future Appointments</h1>
                <h2>You have no appointments in the future.</h2>
            </div>
        </div>
    </>);
}

```



```

        setCookies()
        navigate('/login')
    }
  })
}

}, [cookies, setCookies, navigate])

const handleNext = (event) => {
  if (index < fullPastAppts.length-1){
    setIndex((index) => index + 1)
    setCurrentPastAppts(fullPastAppts[index+1])
  }
}

const handlePrevious = (event) => {
  if (index > 0){
    setIndex((index) => index - 1)
    setCurrentPastAppts(fullPastAppts[index-1])
  }
}

if (fullPastAppts.length <= 1){
  buttons = <></>
} else {
  buttons = <div style={{'display':'flex', 'flexDirection':'row', 'gap':'200px'}}>
    <div className={'buttonDiv'} onClick={handlePrevious}><p>Previous</p></div>
    <div className={'buttonDiv'} onClick={handleNext}><p>Next</p></div>
  </div>
}

if (!currentPastAppts.length) {
  return(<>
    <TopBar/>
    <div style={{ position: "relative", width: "100%" }}>
      <Back/>
      <div className="centerPage">
        <h1>Past Appointments</h1>
        <h2>You have no previous appointments.</h2>
      </div>
    </div>
  </>);
}

return(
  <>
    <TopBar/>
    <div style={{ position: "relative", width: "100%" }}>
      <Back/>

```

```
<div className="centerPage">  
    <h1>Past Appointments</h1>  
    <div style={{'display':'flex', 'flexDirection':'column', 'gap':'10px', 'width':'1200px'}}>  
        {currentPastAppts.map((appt, index) => (  
            <div key={index} style={{'borderStyle':'solid'}}>  
                <h3>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;& Appointment Time: {appt['appt_time']}&nbsp; &nbsp;&nbsp;&nbsp;&nbsp;& Appointment Date: {appt['appt_date']}                <h3>&nbsp;&nbsp;&~{cookies.type === 'patient' ? 'Staff: Dr ' +  
appt['staff_name']: 'Patient: ' + appt['patient_name']}&nbsp;& Details:&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;& Details:&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;& Details:&nbsp;&nbsp;&~{appt['appt_details']}            </div>  
        ))}  
        {buttons}  
    </div>  
</div>  
</div>
```

3 – login.js

This file contains all the code related to logging in and registration. It also contains the page that is displayed upon the registration of a staff member. It does make some Fetch API calls; however those are callback function calls which are imported from another file for simplicity and abstraction.

```
import './App.css';

import React from "react";
import {useState} from "react";
// eslint-disable-next-line
import { BrowserRouter as Router, Routes, Route, Navigate, Link, useNavigate, data, resolvePath } from "react-router-dom";

import { useCookies } from './userContexts';
import { uponLogin, uponRegister } from './ApiCalls';

export{
    Login,
    Register,
    StaffApproval,
}

function Login() {

    const navigate = useNavigate();

    const { setCookies } = useCookies();

    const [ opacity, setOpacity ] = useState(0);
```

```

function handleSubmit(event) {

    event.preventDefault()
    const form = event.target;
    const email = form.elements.email.value;
    const password = form.elements.password.value;

    if (!email || !password){
        setOpacity(1)
    } else {
        setOpacity(0)
    }

    console.log('Form submitted', { Email: email, Password: password });

    uponLogin(email, password, (callback) => {
        if (callback === 'failed') {
            setOpacity(1)
        } else {
            setOpacity(0)
            console.log('User name received:', callback['name']);
            setCookies({
                type: callback['type'],
                name: callback['name'],
                prevPageStack: [],
                currentPage: null,
                backUsed : false,
                authBool: true,
            });

            if (callback['type'] === 'patient') {
                navigate('/home')
            } else if (callback['type'] === 'staff') {
                navigate('/StaffHome')
            }
        }
    });
}

return(
    <div className={'App-header'}>
        <h1>Welcome to GPQueue!</h1>
        <nav>
            <p>If you do not have an account, register <Link to="/register">here</Link>!</p>
        </nav>
        <h2>Login:</h2>
        <form onSubmit={handleSubmit}>
            <div>
                <p>Email:</p><input name="email"></input>
                <p>Password:</p><input name="password" type="password"></input>
            </div>

```

```

        <button type='submit'>Login!</button>
      </form>
      <h2 style={{'color':'red', 'opacity':opacity}}>Invalid email or password, please try again.</h2>
    </div>
  );
}

function Register() {

  const [opacity1, setOpacity1] = useState(0);
  const [opacity2, setOpacity2] = useState(0);
  const [opacity3, setOpacity3] = useState(0);
  const { setCookies } = useCookies();
  const navigate = useNavigate();

  function handleSubmit(event) {

    event.preventDefault()
    const form = event.target;
    const clickedButton = event.nativeEvent.submitter; // Identify the clicked button
    const type = clickedButton.getAttribute('data-type');

    // Set the user type based on the button clicked
    let password = form.elements.password.value;
    let cpassword = form.elements.cpassword.value;
    if (password === cpassword){
      setOpacity1(0);
      setOpacity2(0);
      let email = form.elements.email.value;
      let fname = form.elements.fname.value;
      let sname = form.elements.sname.value;
      let dob = form.elements.dob.value;
      console.log(fname)
      let temp_details = {
        'fname': fname,
        'sname': sname,
        'email': email,
        'password': password,
        'dob': dob,
        'type': type
      };

      if (fname !== '' && sname !== '' && email !== '' && password !== '' && validateDate(dob) && cpassword !== ''
&& validateEmail(email) && validatePassword(password)) {
        console.log("Registration submitted", temp_details)

        uponRegister(fname, sname, email, type, password, dob, (error) => {

          if (!error) {

```

```

        setOpacity3(1)
        return
    }

    setOpacity3(0)

    setCookies({
        type: type,
        name: fname+' '+sname,
        prevPageStack: [],
        currentPage: null,
        backUsed : false,
        authBool: true,
    });

    console.log(type)
    if (type === 'patient') {
        navigate("/home");
    } else {
        navigate("/staffApproval")
    }
    }) //API STUFF TO REGISTER USER

    } else {
        setOpacity2(1);
    }
    } else {
        setOpacity1(1);
    }
}

return(
    <>
    <div className={'App-header'}>
        <h1>Welcome to GPQueue!</h1>
        <nav>
            <p>If you do not have an account, login <Link to="/login">here</Link>!</p>
        </nav>
        <h2>Register:</h2>
    </div>
    <form onSubmit={handleSubmit} className={'App'}>
        <div className={'Row'}>
            <div className={'Column'}>
                <p>Email:</p><input name="email"></input>
                <p>Password:</p><input name="password" type="password"></input>
                <p>(Your password must be larger than 10 characters and include capitals, numbers, and special
characters)</p>
                <p>Confirm Password:</p><input name="cpassword" type='password'></input>

```

```

    </div>
    <div className={'Column'}>
        <p>First Name:</p><input name='fname'></input>
        <p>Surname:</p><input name='sname'></input>
        <p>Date of Birth (DD/MM/YYYY):</p><input name='dob'></input>
    </div>
</div>

<button type='submit' data-type='patient' >Register!</button>
<button style={{'position':'absolute', 'right':0, 'bottom':0, 'zIndex': 10}} type='submit' data-type='staff'
>Register as Staff</button>
</form>

<h3 style={{color: 'red', opacity: opacity1, textAlign:'center'}}>Passwords do not match, try again.</h3>
<h3 style={{color: 'red', opacity: opacity2, textAlign:'center'}}>Please make sure all fields are filled in
correctly.</h3>
<h3 style={{color: 'red', opacity: opacity3, textAlign:'center'}}>Email already in use.</h3>
</>
);
}

function StaffApproval() {
    return(
        <div className={'centerPage'}>
            <img src='https://i.imgur.com/v2qKKW0.png' alt='Logo' style={{width:'400px', height:'400px',
marginTop:'50px'}} />
            <h1>Please wait for your supervisor to approve your registration.</h1>
            <h1>When it is approved, you will be able to log in by going back to the login page.</h1>
        </div>
    );
}

function validDate(dateString) { //returns true/false

    const dateRegex = /^(\\d{2})\\/(\\d{2})\\/(\\d{4})$/; // #iHateRegex
    const match = dateString.match(dateRegex);
    console.log(match) // match = ["32/01/2007","32","01","2007"] when i input match[0]

    if (!match) {
        return false;
    }

    const day = parseInt(match[1], 10); // just converts the value, currently a str, into a base 10 int
    const month = parseInt(match[2], 10);
    const year = parseInt(match[3], 10);

    const date = new Date(year, month - 1, day); // checks if date is valid, 0 based indexing for months
    return (
        date.getFullYear() === year && date.getMonth() === month - 1 && date.getDate() === day
    );
}

function validEmail(emailString){

```

```

const emailRegex = /^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$/i
const match = emailString.match(emailRegex)
console.log(match)

return match
}

function validPassword(passwordString){

const specialRegex = /[!@#$%^&*(){}'";:~_+=`',.<>?/]/
const capitalRegex = /[A-Z]/
const lowerRegex = /[a-z]/
const numberRegex = /[0-9]/
const match1 = passwordString.match(specialRegex)
const match2 = passwordString.match(capitalRegex)
const match3 = passwordString.match(lowerRegex)
const match4 = passwordString.match(numberRegex)
return match1 && match2 && match3 && match4 && passwordString.length > 10
}

```

4 – TopBar.js

This file contains the component for the top bar, which is imported into other files, so that it can be used in other components (pages).

```

import './App.css';
import './smallComponents.js'
import React from "react";
// eslint-disable-next-line
import { BrowserRouter as Router, Routes, Route, Navigate, Link, useNavigate } from "react-router-dom";
import { useCookies } from "./userContexts.js";
import { Timer } from './smallComponents.js';

export{
  TopBar
}

function TopBar() {

  let { cookies } = useCookies();
  const name = cookies.name;
  const navigate = useNavigate();

  const sendHome = (event) => {
    if (cookies.type === 'patient'){
      navigate('/home');
    } else {
      navigate('/StaffHome')
    }
  }
}

```

```

    }

    return(
      <>
        <div className={'topBar'}>
          <img src='https://i.imgur.com/v2qKKW0.png' className={'icon'} onClick={sendHome} alt={'Logo'}/>
          <h2>Welcome, {name}!</h2>
          <Timer/>

        </div>
      </>
    );
  }
}

```

5 – appointments.js

This is the largest frontend file I have created, it contains 5 page components. The pages make up what is seen in the wireframes, as well as a timeout page which is displayed if the user exceeds their 5min time limit. This file is also where most of the fetch API call functions are called from, in some cases utilising callback functions. Most of the JSX (React.js's version of HTML) is very simple as in previous cases, however in the 2nd page component (BookAppointment2) there is some complexity, where we iterate through an object and create a div for every key-value pair, with different properties based on the key and value.

```

import './App.css';
import './smallComponents.js'
import React from "react";
import {useEffect, useState, useRef} from "react";
// eslint-disable-next-line
import { BrowserRouter as Router, Routes, Route, Navigate, Link, useNavigate } from "react-router-dom";
import { Timer, Back, hubOrSub } from './smallComponents.js';
import { useCookies, useBooking } from './userContexts.js';
import { fetchAppointments, fetchDates, chooseAppointment, endBookingSession } from './ApiCalls.js';

export{
  BookAppointment1,
  BookAppointment2,
  BookAppointment3,
  BookAppointment4,
  Timeout
}

function BookAppointment1() {

  const [ priority, setPriority ] = useState(3);
  const [ reason, setReason ] = useState('');
  const { cookies, setCookies } = useCookies();
  const { bookingData, setBookingData } = useBooking();

```

```

const tempData = bookingData
const [ opacity, setOpacity ] = useState(0);
const navigate = useNavigate();

const hasRun = useRef(false);

if (!cookies.authBool | !sessionStorage.getItem('jwt')) {navigate('/login')};
useEffect(() => {
  if (!hasRun.current) {
    setCookies(hubOrSub('BookAppointment1', cookies));
    if (tempData.reason !== '') {
      setReason(tempData.reason)
    }
    hasRun.current = true
  }
}, [setCookies, cookies, tempData.reason])

const sendToNext = (event) => {

  if (reason === '') {
    setOpacity(100)
  } else {

    setBookingData({
      priority: priority,
      reason: reason,
      dates: tempData.dates,
      times: tempData.times
    })
    navigate('/BookAppointment2')
  }
}

return(
  <>
    <div style={{ position: "relative", width: "100%", 'textAlign':'center' }}>
      <Back />
      <h1>Booking (Part 1)</h1>
      <h2>Choose your main reason for making an appointment.</h2>
      <h3>Reason: {reason}</h3>
      <h3 style={{'color':'red', opacity:opacity}}>Please choose a reason before proceeding!</h3>
      <div className="centerPage" style={{'height':'65vh'}}>
        <div className={ 'Row' }>
          <div className={ 'reasonList' }>
            <div className={ 'reason' } onClick={() => {setPriority(1); setReason('Persistent fever')}}>Persistent fever</div>
            <div className={ 'reason' } onClick={() => {setPriority(1); setReason('Severe sore throat')}}>Severe sore throat</div>

```

```

        <div className={'reason'} onClick={() => {setPriority(1); setReason('Ear pain')}}>Ear
pain</div>

        <div className={'reason'} onClick={() => {setPriority(1); setReason('Abdominal
pain')}}>Abdominal pain</div>

        <div className={'reason'} onClick={() => {setPriority(1); setReason('Concerning
lumps')}}>Concerning lumps</div>

        <div className={'reason'} onClick={() => {setPriority(1); setReason('Anxiety/Depression
(worsening')}}>Anxiety/Depression (worsening)</div>

        <div className={'reason'} onClick={() => {setPriority(1); setReason('Unexplained weight
loss')}}>Unexplained weight loss</div>

        <div className={'reason'} onClick={() => {setPriority(1); setReason('UTI or STI
symptoms')}}>UTI or STI symptoms</div>

        <div className={'reason'} onClick={() => {setPriority(1); setReason('Worsening chest
infection')}}>Worsening chest infection</div>

        <div className={'reason'} onClick={() => {setPriority(2); setReason('Sprained
joint')}}>Sprained joint</div>

        <div className={'reason'} onClick={() => {setPriority(2); setReason('Ongoing
migraines')}}>Ongoing migraines</div>

        <div className={'reason'} onClick={() => {setPriority(2); setReason('Skin rashes')}}>Skin
rashes</div>

        <div className={'reason'} onClick={() => {setPriority(2); setReason('Chronic pain needing
review')}}>Chronic pain needing review</div>

        <div className={'reason'} onClick={() => {setPriority(2); setReason('Digestive
issues')}}>Digestive issues</div>
    </div>

    <div className={'reasonList'}>
        <div className={'reason'} onClick={() => { setPriority(2); setReason('Sleep problems');
}}>Sleep problems</div>

        <div className={'reason'} onClick={() => { setPriority(2); setReason('Medication side
effects'); }}>Medication side effects</div>

        <div className={'reason'} onClick={() => { setPriority(2); setReason('Minor infection');
}}>Minor infection</div>

        <div className={'reason'} onClick={() => { setPriority(2); setReason('Persistent cough');
}}>Persistent cough</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Check up'); }}>Check
up</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Medication review');
}}>Medication review</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Blood pressure
monitoring'); }}>Blood pressure monitoring</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Routine vaccinations');
}}>Routine vaccinations</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Minor skin
conditions'); }}>Minor skin conditions</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Mild joint pain');
}}>Mild joint pain</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('General health
advice'); }}>General health advice</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Travel vaccinations');
}}>Travel vaccinations</div>

```

```

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Test results review');
}}>Test results review</div>

        <div className={'reason'} onClick={() => { setPriority(3); setReason('Other'); }}>Other</div>
    </div>
</div>

    <div className={'buttonDiv'} onClick={sendToNext}>
        <h3>Continue</h3>
    </div>
</div>
</div>
<Timer />
</>
);
}

```

```

function BookAppointment2() {
    const navigate = useNavigate();
    const hasRun = useRef(false);
    let { cookies, setCookies } = useCookies();
    const { bookingData, setBookingData } = useBooking();
    const tempData = bookingData;

    const [selection, setSelection] = useState('');
    const [opacity, setOpacity] = useState(0);
    const [fullDates, setFullDates] = useState(null);
    const [currentDates, setCurrentDates] = useState(null);
    const [loading, setLoading] = useState(true);

    if (!cookies.authBool | !sessionStorage.getItem('jwt')) {
        navigate('/login');
    }

    useEffect(() => {
        if (!hasRun.current) {
            setCookies(hubOrSub('BookAppointment2', cookies));
            if (tempData.dates !== '') {
                setSelection(tempData.dates);
            }

            fetchDates(bookingData.priority, (callback) => {
                console.log(callback)
                if (callback === 'error'){
                    setBookingData({
                        priority: null,
                        reason: '',
                        dates: '',
                        times: '',

```

```

        })
        navigate('/home')
      } else if (callback === 'login again') {
        setBookingData()
        setCookies()
        navigate('/login')
      } else {
        setFullDates(callback['dates']);
        setCurrentDates(callback['dates'][0]);
        setLoading(false);
      }
    });

    hasRun.current = true;
  }
}, [setCookies, cookies, tempData.dates, bookingData.priority, navigate, setBookingData]);

const now = new Date();
const months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October',
'November', 'December'];
const [month, setMonth] = useState(months[now.getMonth()]);

const handleSelect = (date) => {
  if (!selection.includes(date)) {
    setSelection(selection + date + ', ');
  } else {
    setSelection(selection.replace(date + ', ', ''));
  }
};

const group = useRef(0);

const handlePrevious = () => {
  if (group.current > 0) {
    setCurrentDates(fullDates[group.current - 1]);
    setMonth(months[Object.entries(fullDates[group.current - 1])[0][0].slice(3, 5) - 1]);
    group.current -= 1;
  }
};

const handleNext = () => {
  if (group.current < 2) {
    setCurrentDates(fullDates[group.current + 1]);
    setMonth(months[Object.entries(fullDates[group.current + 1])[0][0].slice(3, 5) - 1]);
    group.current += 1;
  }
};

const handleContinue = () => {
  if (selection === '') {
    setOpacity(100);
  }
};

```

```

    } else {
      setBookingData({
        priority: tempData.priority,
        reason: tempData.reason,
        dates: selection,
        times: tempData.times,
      });

      navigate('/BookAppointment3');
    }
  };

  // show loading until data is fetched
  if (loading || !currentDates) {
    return <div>Loading...</div>
  }

  return (
    <>
      <div style={{ position: "relative", width: "100%" }}>
        <Back />
        <h1 style={{ 'textAlign': 'center' }}>Booking (part 2)</h1>
        <h2 style={{ 'textAlign': 'center' }}>Select your preferred days</h2>
        <div className="centerPage" style={{ 'flexDirection': 'row', 'justifyContent': 'left', 'gap': '200px'
      }}>

          <div className={'Calendar'}>
            {Object.entries(currentDates).map(([date, dateBool]) => (
              <div key={date} className='day'
                onClick={dateBool ? () => handleSelect(date) : undefined} // Only call if available
                style={{
                  'backgroundColor': dateBool ? "#90ee90" : "#d3d3d3",
                  'pointerEvents': dateBool ? "auto" : "none",
                  'cursor': dateBool ? "pointer" : "not-allowed",
                  'borderRadius': '5px'
                }}>
                <p style={{ 'marginTop': '5px' }}>{date}</p>
              </div>
            ))}
          </div>
          <div style={{ 'display': 'flex', 'flexDirection': 'column', 'justifyContent': 'center', 'gap':
'10px', 'flexWrap': 'wrap', 'alignItems': 'center' }}>
            <h3>Current month: {month}</h3>
            <div style={{ 'maxWidth': '505px', 'textAlign': 'center' }}>
              <p>Days selected: {selection}</p>
            </div>
            <p style={{ 'color': 'red', 'opacity': opacity }}>Please select at least one day before
continuing!</p>
            <div className={'buttonDiv'} onClick={handlePrevious}>
              <p>Previous Month</p>
            </div>
            <div className={'buttonDiv'} onClick={handleNext}>

```

```

        <p>Next Month</p>
      </div>
      <div className={'buttonDiv'} style={{ 'width': '200px' }} onClick={handleContinue}>
        <p>Continue</p>
      </div>
    </div>
  </div>
  <Timer />
</>
);
}

```

```

function BookAppointment3() {

  const hasRun = useRef(false);
  let { cookies, setCookies } = useCookies();
  const navigate = useNavigate();
  const { bookingData, setBookingData } = useBooking();
  const tempData = bookingData
  const [ selection, setSelection ] = useState('');
  const [ opacity, setOpacity ] = useState(0);

  if (!cookies.authBool | !sessionStorage.getItem('jwt')) {navigate('/login')};
  useEffect(() => {
    if (!hasRun.current) {
      setCookies(hubOrSub('BookAppointment3', cookies));
      hasRun.current = true
    }
  })

  const handleSelect = (time) => {
    if (!selection.includes(time)) {
      setSelection(selection + time+', ')
    } else {
      setSelection(selection.replace(time+', ', ''))
    }
  }

  const handleContinue = (event) => {
    if (selection === '') {
      setOpacity(100);
    } else {

      setBookingData({
        priority: tempData.priority,
        reason: tempData.reason,
        dates: tempData.dates,
        times: selection,

```

```

    })

    navigate('/BookAppointment4')
  })

  return(
    <>
      <div style={{ position: "relative", width: "100%" }}>
        <Back />
        <h1 style={{'textAlign':'center'}}>Booking (part 3)</h1>
        <h2 style={{'textAlign':'center'}}>Select your preferred timeframes</h2>
        <div className="centerPage" style={{'flexDirection':'row', 'justifyContent':'left',
'alignItems':'center'}}>
          <div className='Column' style={{'display':'flex', 'alignItems':'center',
'flexDirection':'column', 'gap':'20px'}}>
            <div className='buttonDiv' style={{'backgroundColor':'white'}} onClick={() =>
{handleSelect('09:00-13:00')}}>
              <h3>9:00 - 13:00</h3>
            </div>
            <div className='buttonDiv' style={{'backgroundColor':'white'}} onClick={() =>
{handleSelect('13:00-17:00')}}>
              <h3>13:00 - 17:00</h3>
            </div>
            <div className='buttonDiv' style={{'backgroundColor':'white'}} onClick={() =>
{handleSelect('17:00-21:00')}}>
              <h3>17:00 - 21:00</h3>
            </div>
          </div>
          <div className='Column' style={{'display':'flex', 'alignItems':'center',
'flexDirection':'column', 'gap':'20px'}}>
            <p>Selection: {selection}</p>
            <p style={{'color':'red', 'opacity':opacity}}>Please select at least one timeframe!</p>
            <div className='buttonDiv' onClick={handleContinue}>
              <h3>Continue</h3>
            </div>
          </div>
        </div>
        </div>
        <Timer />
      </>
    );
  }

function BookAppointment4() {

  const hasRun = useRef(false);
  let { cookies, setCookies } = useCookies();

```

```

const navigate = useNavigate();
const { bookingData, setBookingData } = useBooking();
const tempData = bookingData

if (!cookies.authBool | !sessionStorage.getItem('jwt')) {
  navigate('/login');
}

const [ apptsList, setApptsList ] = useState([])
const [ currentAppt, setCurrentAppt ] = useState()
const [ countdown, setCountdown ] = useState(300);
const [ index, setIndex ] = useState(0);
const [ loading, setLoading ] = useState(true);
const [ noAppts, setNoAppts ] = useState(false);
const [ opacity, setOpacity ] = useState(0)

useEffect(() => {
  if (!hasRun.current) {
    hasRun.current = true;
    setCookies(hubOrSub('BookAppointment4', cookies));
    // Fetch data in useEffect
    fetchAppointments(tempData.priority, tempData.dates, tempData.times, (callback) => {
      console.log(callback)
      if (!callback.length){
        setNoAppts(true)
        console.log('no appts is true')
      } else if (callback === 'login again') {
        setBookingData()
        setCookies()
        navigate('/login')
      } else {

        setApptsList(callback)
        setCurrentAppt(callback[0])
        setLoading(false)

        try {
          if (callback[index]['appt_id']){
            setOpacity(1)
          }
        } catch(error) {
          console.log(error)
          setOpacity(0)
        }
      }
    });
  }
}, [cookies, currentAppt, setCookies, tempData, navigate, setBookingData, index]);

```

```

useEffect(() => {

  const interval = setInterval(() => {
    setCountdown( (countdown) => countdown - 1)
  }, 1000)

  if (countdown < 1) {

    endBookingSession((callback) => {
      console.log(callback)
      setBookingData({priority: null,
        reason: '',
        dates: '',
        times: '',})
      if (callback === 'login again') {
        setCookies()
        navigate('/login')
      }
      navigate('/timeout')
    })
  }

  return () => clearInterval(interval);
})

const handleForward = () => {
  if (index < apptsList.length-1) {
    setIndex( (index) => index + 1)
    setCurrentAppt(apptsList[index+1])

    if (!apptsList[index+1]['appt_id']){
      setOpacity(0)
    } else {
      setOpacity(1)
    }
  }
}

const handlePrevious = () => {
  if (index > 0) {
    setIndex( (index) => index - 1)
    setCurrentAppt(apptsList[index-1])

    if (!apptsList[index-1]['appt_id']){
      setOpacity(0)
    } else {
      setOpacity(1)
    }
  }
}

```

```

const handleNext = () => {

  // SEND currentAppt TO THE BACKEND FOR THEM TO SAVE IN THE DB
  let temp = currentAppt
  temp['appt_details'] = bookingData.reason
  chooseAppointment(currentAppt, (callback)=>{
    console.log(callback)
    setBookingData({priority: null,
      reason: '',
      dates: '',
      times: '',})
    navigate('/home')
  })

}

const handleExit = () => {
  endBookingSession((callback) => {
    console.log(callback)
    setBookingData({priority: null,
      reason: '',
      dates: '',
      times: '',})

    if (callback === 'login again') {
      setCookies()
      navigate('/login')
    }
    navigate('/home')
  })
}

if (loading) {
  return (<div>Loading...</div>);
} else if (noAppts) {
  return(
    <>
    <div>
      <div className={'centerPage'} style={{'display':'flex', 'flexDirection':'column',
'alignItems':'center', 'justifyContent':'center'}}>
        <h1>Sadly, there are no available appointments at this time. Try booking for different times and
dates.</h1>
        <div className={'buttonDiv'} onClick={handleExit}><h1>Go to the Homepage</h1></div>
      </div>
      <Timer/>
    </div>
    </>
  )
}

```

```

    return(
      <>
        <div style={{ position: "relative", width: "100%" }}>
          <div className={buttonDiv} onClick={handleExit} style={{width:'200px'}}><h3>Exit to
Home</h3></div>

          <h1 style={{textAlign:'center'}}>Summary - Booking (part 4)</h1>
          <h2 style={{textAlign:'center'}}>Confirm this appointment choice or reroll</h2>
          <div className="centerPage" style={{flexDirection:'center', 'justifyContent':'center',
'alignItems':'center', 'gap':'0px'}}>

            <h3>Date: {currentAppt['date']}</h3>
            <h3>Time: {currentAppt['timeslot']}</h3>
            <h3>Doctor: {currentAppt['staff_name']}</h3>
            <h3 style={{opacity:opacity}}>If you choose this appointment, you will be replacing somebody
elses. Their appointment will be rescheduled.</h3>

            <p style={{color:'red'}}>You have {String(Math.floor(countdown/60))
+':'+String(countdown%60).padStart(2, '0')} left to lock your choice in, or to change to for another appointment
time.</p>

            <div style={{display:'flex', 'flexDirection':'row', 'gap':'5px', 'justifyContent':'center',
'alignItems':'center'}}>
              <div className={buttonDiv} onClick={handleNext}
style={{width:'300px'}}><h3>Book!</h3></div>
              <div className={buttonDiv} onClick={handleForward} style={{pointerEvents':
(index===apptsList.length -1)? 'none' : 'auto', 'cursor': (index===apptsList.length -1)? 'none' :
'pointer', 'width':'300px'}}><h3>Forwards</h3></div>
              <div className={buttonDiv} onClick={handlePrevious} style={{pointerEvents': (index===0)?
'none' : 'auto', 'cursor': (index===0)? 'none' : 'pointer', 'width':'300px'}}><h3>Backwards</h3></div>
            </div>
          </div>
        </div>
        <Timer />
      </>
    );
  }

function Timeout() {

  const navigate = useNavigate();
  const handleClick = () => {navigate('/home')}

  return(
    <>
      <div>
        <div className={centerPage} style={{display:'flex', 'flexDirection':'column',
'alignItems':'center', 'justifyContent':'center'}}>
          <h1>You took too long to make a choice, try again with different options.</h1>
          <div className={buttonDiv} onClick={handleClick}><h1>Go to the Homepage</h1></div>
        </div>
      </div>
    )
  )
}

```

```

        </div>
        <Timer/>
    </div>
</>
);
}

```

ApiCalls.js

This file is where all my Fetch API functions are located. This includes my GET, POST, UPDATE and DELETE requests for login, registration, and fetching several things. The domain of my flask web server is specified at the top; it is localhost:5000. Also, notice how the JWT is being stored in Session Storage after login and registration, and then is being retrieved from there for all the other API requests. Early on I had some problems with error handling, and in particular not being able to detect what sort of problem was occurring, so I set up try catch blocks to be able to respond to certain errors. Also, this file is completely composed of async/await functions, which is very important when dealing with APIs.

```

export {
    uponLogin,
    uponRegister,
    fetchDates,
    fetchAppointments,
    chooseAppointment,
    endBookingSession,
    fetchPast,
    fetchFuture,
    fetchApptAmount,
    refreshJWT
};

const domain = 'http://127.0.0.1:5000/';

async function uponLogin(email, password, callback) {
    try{
        const loginResponse = await fetch(domain + 'get_token', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ email: email, password: password }),
            credentials: 'include'
        })
        if (!loginResponse.ok) {
            throw new Error('Login failed!');
        }

        const loginData = await loginResponse.json();

        console.log('message:', loginData['message']);
    }
}

```

```

        console.log('token received:', loginData['token']);
        sessionStorage.setItem('jwt', loginData['token']);
        sessionStorage.setItem('refresh_token', loginData['refresh_token']);

        try {
            const nameResponse = await fetch(domain + '/get_name', {
                method: 'GET',
                headers: {
                    'Content-Type': 'application/json',
                    'Authorization': `Bearer ${loginData['token']}`
                },
                credentials: 'include'
            })

            if (!nameResponse.ok) {
                throw new Error("Couldn't fetch name :(");
            }

            const nameData = await nameResponse.json();
            console.log(nameData['name']);
            callback(nameData);
            return

        } catch(error){
            console.error('Error:', error.message);
        }

    } catch(error){
        console.error('Error:', error);
        callback('failed');
        return
    }
}

async function uponRegister(fname, sname, email, type, password, dob, callback) {
    try {
        const response = await fetch(domain + 'register', {
            method: 'POST',
            body: JSON.stringify({ fname: fname, sname: sname, email: email, type: type, password: password, dob: dob
        })),

            headers: { 'Content-Type': 'application/json' },
            credentials: 'include'
        })

        if (!response.ok) {
            throw new Error("Couldn't register/get JWT :(");
        }

        const data = await response.json();

        if (data['error'] === "Email in use") {

```

```

        throw new Error("Email in use");
    }

    console.log('token:', data['token']);
    sessionStorage.setItem('jwt', data['token']);
    sessionStorage.setItem('refresh_token', data['refresh_token']);
    callback(1);

} catch(error){
    console.error('Error:', error.message || error);
    callback(0);
}
}

async function fetchDates(priority, callback, retried = false) {
    try {
        const token = sessionStorage.getItem('jwt');
        const response = await fetch(domain + `fetchDates?priority=${priority}`, {
            method: 'GET',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${token}`
            },
            credentials: 'include',
        });

        if (response.status === 401) {
            console.log('Token expired or unauthorised access');
            throw new Error('refresh');
        } else if (!response.ok) {
            throw new Error("Couldn't fetch dates :(");
        }

        const data = await response.json();

        if (data.error) {
            throw new Error(data['error']);
        }

        console.log(data['message']);
        callback(data['dates']);

    } catch (error) {
        console.error('Error:', error.message || error);

        if (error.message === 'refresh' && !retried) {
            try {
                await refreshJWT();
                return fetchDates(priority, callback, true);
            } catch (refreshError) {
                console.error('Failed to refresh token:', refreshError.message || refreshError);
            }
        }
    }
}

```

```

        callback('login again');
    }
} else {
    callback('error');
}
}
}

async function fetchAppointments(priority, dates, times, callback, retried=false) {
    try{
        const token = sessionStorage.getItem('jwt');
        const response = await fetch(domain + 'fetchAppointments', {
            method: 'PUT',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${token}`
            },
            body: JSON.stringify({ priority, dates, times }),
            credentials: 'include'
        })
        if (response.status === 401) {
            console.log('Token expired or unauthorised access');
            throw new Error('refresh');
        } else if (!response.ok) {
            throw new Error("Couldn't fetch appointments :(");
        }

        const data = await response.json()

        if (data['error']) {
            throw new Error(data['error']);
        }
        callback(data['appts']);
        return

    } catch(error) {
        console.error('Error:', error.message || error);
        if (error.message === 'refresh' && !retried) {
            try {
                await refreshJWT();
                return fetchAppointments(priority, dates, times, callback, true)
            } catch (error) {
                console.error('Failed to refresh token: ', error.message || error);
                callback('login again')
                return
            }
        } else {
            callback('error');
            return
        }
    }
}

```

```

    }
}

async function fetchApptAmount(type, callback, retried=false) {
  try{
    const token = sessionStorage.getItem('jwt');
    const response = await fetch(domain + `fetchApptAmount?type=${type}`, {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
      },
      credentials: 'include'
    })

    if (response.status === 401) {
      console.log('Token expired or unauthorised access');
      throw new Error('refresh');
    } else if (!response.ok) {
      throw new Error("fetch failed :(");
    }

    const data = await response.json();

    if (data['error']) {
      throw new Error(data['error']);
    }

    callback(type === 'patient' ? data['amount'] : [data['amount'], data['todayAmount']]);
    return

  } catch(error){
    console.error('Error:', error.message || error);
    if (error.message === 'refresh' && !retried) {
      try{
        await refreshJWT();
        return fetchApptAmount(type, callback, true);
      }catch(error){
        console.error('Failed to refresh token: ', error.message || error)
        callback('login again')
        return
      }
    } else {
      callback('error');
      return
    }
  }
}

async function fetchPast(type, callback, retried=false) {
  try {

```

```

const token = sessionStorage.getItem('jwt');
const response = await fetch(domain + `fetchPast?type=${type}`, {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`
  },
  credentials: 'include'
})

if (response.status === 401) {
  console.log('Token expired or unauthorised access');
  throw new Error('refresh');
} else if (!response.ok) {
  throw new Error("fetch failed :(");
}

const data = await response.json();

if (data['error']) {
  throw new Error(data['error']);
}
callback(data['appts']);

} catch(error){
  console.error('Error:', error.message || error);
  if (error.message === 'refresh' && !retried) {
    try {
      await refreshJWT();
      return fetchPast(type, callback, true)
    } catch (error) {
      console.error('Failed to refresh token: ', error.message || error);
      callback('login again')
    }
  } else {
    callback('error');
  }
}
}

async function fetchFuture(type, callback, retried=false) {
  try {
    const token = sessionStorage.getItem('jwt');
    const response = await fetch(domain + `fetchFuture?type=${type}`, {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
      },
      credentials: 'include'
    })
  }
}

```

```

        if (response.status === 401) {
            console.log('Token expired or unauthorised access');
            throw new Error('refresh');
        } else if (!response.ok) {
            throw new Error("fetch failed :(");
        }
    }
    const data = await response.json();

    if (data['error']) {
        throw new Error(data['error']);
    }
    callback(data['appts']);

} catch(error){
    console.error('Error:', error.message || error);
    if (error.message === 'refresh' && !retried) {
        try {
            await refreshJWT();
            return fetchFuture(type, callback, true)
        } catch (error) {
            console.error('Failed to refresh token: ', error.message || error);
            callback('login again')
        }
    } else {
        callback('error');
    }
}
}

}

async function chooseAppointment(appt, callback, retried=false) {
    try{
        const token = sessionStorage.getItem('jwt');
        const response = await fetch(domain + 'chooseAppointment', {
            method: 'DELETE',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${token}`
            },
            body: JSON.stringify({ appt }),
            credentials: 'include'
        })

        if (response.status === 401) {
            console.log('Token expired or unauthorised access');
            throw new Error('refresh');
        } else if (!response.ok) {
            throw new Error("Booking failed :(");
        }
    }
    const data = await response.json();

```

```

        if (data['error']) {
            throw new Error(data['error']);
        }
        callback(data['msg']);
    } catch(error){

        console.error('Error:', error.message || error);
        if (error.message === 'refresh' && !retried) {
            try {
                await refreshJWT();
                return chooseAppointment(appt, callback, true)
            } catch (error) {
                console.error('Failed to refresh token: ', error.message || error);
                callback('login again')
            }
        } else {
            callback('error')
        }
    }
}

async function endBookingSession(callback, retried=false) {
    try {
        const token = sessionStorage.getItem('jwt');
        const response = await fetch(domain + 'endBookingSession', {
            method: 'DELETE',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${token}`
            },
            credentials: 'include'
        })

        if (response.status === 401) {
            console.log('Token expired or unauthorised access');
            throw new Error('refresh');
        } else if (!response.ok) {
            throw new Error("Exit failed :(");
        }

        const data = await response.json();

        if (data['error']) {
            throw new Error(data['error']);
        }
        callback(data['msg']);
    } catch(error){

        console.error('Error:', error.message || error);
    }
}

```

```

    if (error.message === 'refresh' && !retried) {
      try {
        await refreshJWT();
        return endBookingSession(callback, true)
      } catch (error) {
        console.error('Failed to refresh token: ', error.message || error);
        callback('login again')
      }
    } else {
      callback('error');
    }
  }
}
}

```

```

async function refreshJWT() {
  const refreshToken = sessionStorage.getItem('refresh_token');
  console.log('refreshJWT function is running');

  if (refreshToken) {
    try{
      const response = await fetch(domain + '/refreshJWT', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${refreshToken}`
        },
        body: JSON.stringify({ refresh_token: refreshToken }),
        credentials: 'include'
      })

      if (!response.ok) {
        throw new Error('JWT refresh failed!');
      }
      const data = await response.json();

      console.log('message:', data['message']);
      console.log('token received:', data['token']);
      sessionStorage.setItem('jwt', data['token']);
      sessionStorage.setItem('refresh_token', data['refresh_token'])

      return response
    } catch(error){
      console.error('Error:', error.message || error);
      throw error
    }
  }

  return Promise.reject(new Error('No refresh token found'));
}

```

```
}
```

smallComponents.js

This file contains all my small components which had no real place in any other file.

The first component is the Timer component, which uses the JavaScript Date object, and its methods to get the current date, and set the timer state to it. It then saves the time and date as formatted strings for later use. After, it sets up a useEffect, which starts an Interval, which is a React hook which runs a function every interval (which in this case is 1000ms, or 1 second). After the interval passes, the setTime() function is called, which updates the state of time. Since this is contained in a useEffect, the cleanup code is run first, which clears the interval (otherwise we would have exponential re-renders and the website would crash) and then it renders the component from the beginning, with a new date, and it repeats the process. The other two components/functions are my back button and my HubOrSub function, which has been explained in the design [here](#).

```
import React from "react";
import {useState, useEffect, memo} from "react";
import { useCookies, useBooking } from "../userContexts";
// eslint-disable-next-line
import { BrowserRouter as Router, Routes, Route, Navigate, Link, useNavigate } from "react-router-dom";
import Stack from "../stack";

export{
  Back,
  hubOrSub,
}

export const Timer = memo(function Timer() {

  const [time, setTime] = useState(new Date());
  const formattedTime = time.toLocaleTimeString()
  const formattedDate = formatDate(time);

  useEffect(() => {
    const timer = setInterval(() => setTime(new Date()), 1000);

    return () => clearInterval(timer);
  }, [])

  function formatDate(date) {
    const day = String(date.getDate()).padStart(2, "0");
    const month = String(date.getMonth() + 1).padStart(2, "0");
    const year = date.getFullYear();
    return `${day}/${month}/${year}`;
  }
}
```

```

    return(
      <>
        <h3 className={'Timer'}>{formattedDate} {formattedTime}</h3>
      </>
    );
  });

function Back() {

  const navigate = useNavigate();
  const { cookies, setCookies } = useCookies();
  const { setBookingData } = useBooking();
  const prevPageStack = new Stack();

  const handleClick = (event) => {

    if (cookies.currentPage === 'BookAppointment1') {
      setBookingData({priority: null,
        reason: '',
        dates: '',
        times: '',})
    }

    prevPageStack.items = cookies.prevPageStack || [];

    let prevPage = prevPageStack.pop()

    let newCookies = {
      type: cookies.type,
      name: cookies.name,
      prevPageStack: prevPageStack.items,
      currentPage: prevPage,
      backUsed: true,
      authBool: cookies.authBool,
    }

    setCookies(newCookies);
    navigate('/'+prevPage);
  }

  return(
    <>
      <div onClick={handleClick}>
        <div className={'Back'} >
          <h3>Back</h3>
        </div>
      </div>
    </>
  );
}

```

```

    );
}

function hubOrSub(pageName, cookies) { //UPON CALLING THIS FUNCTION, PASS IN THE PAGE NAME AND THE COOKIES FROM
useCookies()

    //WE HAVE 2 TYPES OF PAGE, A HUB AND A SUB.
    // A HUB IS A PAGE WITH NO BACK BUTTONS, SUCH AS THE HOMEPAGE
    // A SUB IS A PAGE WITH A BACK BUTTON
    // UPON ENTERING A HUB, THE STACK MUST BE WIPED

    const hubOrSubDict = {
        'home': 'hub',
        'staffhome': 'hub',
        'FutureAppointments': 'sub',
        'PastAppointments': 'sub',
        'BookAppointment1': 'sub',
        'BookAppointment2': 'sub',
        'BookAppointment3': 'sub',
    }

    let prevPageStack = new Stack();
    prevPageStack.items = cookies.prevPageStack || []; //stack hydration
    let prevPage = cookies.currentPage

    const newPageType = hubOrSubDict[pageName];

    if (newPageType === 'hub') {
        prevPageStack.purge()
    } else if (newPageType === 'sub' && !cookies.backUsed && !(prevPage === pageName)) {
        prevPageStack.push(prevPage)
    }

    return {
        type: cookies.type,
        name: cookies.name,
        prevPageStack: prevPageStack.items, //technically stack dehydration
        currentPage: pageName,
        backUsed: false,
        authBool: cookies.authBool,
    }
}

```

stack.js

This is a file which contains my implementation of a stack, in JavaScript, using a class. This was my first time using OOP in JavaScript, and it is very similar to Python. It uses ‘this’ instead of ‘self’, and everything else follows normal JS logic (such as curly brackets

for wrapping functions). I require this implementation for my prevPageStack, used to make the back button work.

```
//notes
// quite similar to python OOP, but self -> this
// this.items is the stacks items, stored as an array
// arrays have a length attribute which i can call on this.items.length

export default class Stack {

  constructor(){
    this.items = [];
  }

  isEmpty(){
    return (this.items.length === 0)
  }

  push(val){
    this.items[this.items.length] = val;
  }

  pop(){
    if (this.isEmpty()) {
      throw new Error('Stack is empty!')
    } else {
      const rear = this.items[this.items.length-1]
      this.items.length = this.items.length -1
      return rear
    }
  }

  peek(){
    if (this.isEmpty()) {
      throw new Error('Stack is empty!')
    } else {
      return this.items[this.items.length-1]
    }
  }

  purge(){
    while (true) {
      if (this.isEmpty()) {
        return
      } else {
        this.items.pop()
      }
    }
  }
}
```

userContexts.js

This file is to store both initialisations of my contexts. There are two of them: Cookies Context and Booking Context. The Cookies Context stores general things, such as the user type, their name, their previously visited pages, and more. The Booking Context stores values to do with the booking process, such as their chosen dates, their priority level, and more.

```
import React, { createContext, useContext, useState } from "react";

const CookiesContext = createContext();
const BookingContext = createContext();

export const CookiesProvider = ({ children }) => {
  const [cookies, setCookies] = useState({
    type: null,
    name: null,
    prevPageStack: [],
    currentPage: null,
    backUsed : false,
    authBool: false,
  });

  return (
    <CookiesContext.Provider value={{ cookies, setCookies }}>
      {children}
    </CookiesContext.Provider>
  );
};

export const BookingProvider = ({ children }) => {
  const [bookingData, setBookingData] = useState({
    priority: null,
    reason: '',
    dates: '',
    times: '',
  });

  return (
    <BookingContext.Provider value={{ bookingData, setBookingData }}>
      {children}
    </BookingContext.Provider>
  );
};

export const useCookies = () => useContext(CookiesContext);
export const useBooking = () => useContext(BookingContext);
```

testData.js

This is a file which I created during the coding process to help perform iterative testing while working on different frontend components. Since it is all dummy data, there is no point in showing the code, as it is simply two variables which are being exported.

App.css

This is simply a CSS file containing all the styles for every single component and JSX class in the app. There is no sense in copy pasting it as there is no actual CSS logic being used.

Favicon.png

This file is the favicon for the website, but in PNG format instead of .ico.

test.js

A small file I created to check if certain things work in JavaScript, there is a python equivalent of this in the backend.

Auto-Generated Files

Generated files meant for use, but gone unused:

- App.test.js
- Index.css
- ReportWebVitals.js
- SetupTests.js
- .gitignore - Auto generated, and unchanged by myself

Generated files, not meant/required to be edited:

- Package-lock.json
- Package.json
- Index.html
- Manifest.json
- Robots.txt
- Node_modules – A folder which contains every single node module required for this app, which is a lot.

Backend

pycache

This is a folder of cache files (.pyc), and it is automatically generated after the python programs in the files below have been run a few times, to decrease runtime in the future.

.env

A .env file is a text file containing environment files for an application. In my case, I'm using it to store my JWT secret key, and values required by psycopg2 to connect to my Postgres DB (such as host, port, etc.).

Backend.py

This file is a cross between middleware and backend. It starts up the Flask App/Web Server. It also contains all the endpoints for the API calls made by the frontend, and for some of the endpoints, it does the processing, such as for registration and login. It also serves as a location for the CORS object to be instantiated, to make sure the API route is kept safe (even though I am sending it from port to port).

```
from flask import Flask, request, jsonify
from flask_jwt_extended import JWTManager, create_access_token, jwt_required, get_jwt_identity, create_refresh_token
from flask_cors import CORS
import psycopg2
import os
from dotenv import load_dotenv
import datetime
from datetime import timedelta

from MyHash import custom_hash
from fetch_dates import processing_dates
from booking_algorithm import fetch_appointments, choose_appointment, end_booking_session

load_dotenv()

app = Flask(__name__)
app.config['JWT_SECRET_KEY'] = os.getenv("JWT_SECRET_KEY")
app.config['JWT_TOKEN_LOCATION'] = ['headers']
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(seconds=10)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(days=30)
jwt = JWTManager(app)
CORS(app, supports_credentials=True, origins=['http://localhost:3000'], expose_headers=["Content-Type",
"Authorization"])

conn_config = [os.getenv("HOST"), os.getenv("DBNAME"), os.getenv("USER"), os.getenv("PASSWORD"), os.getenv("PORT")]

@app.route('/get_token', methods=['POST'])
```

```

def get_token():
    data = request.get_json()
    email = data.get('email')
    password = data.get('password')

    #find email and corresponding pwd hash in db, hash pwd before checking, then return id

    try:
        conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2],
password=conn_config[3], port=conn_config[4])
        cur = conn.cursor()
    except Exception as error:
        return jsonify({'error':error})

    cur.execute('''SELECT salt
                    FROM "Users"
                    WHERE email = %s''',
                (email, ))

    salt = cur.fetchone()[0]
    password_hash, salt = custom_hash(password=password, salt=salt)

    cur.execute('''SELECT user_id
                    FROM "Users"
                    WHERE email=%s AND password_hash=%s''',
                (email, str(password_hash)))
    fetched = cur.fetchone()

    if fetched == None:
        cur.close()
        conn.close()
        return jsonify({'message': 'Invalid credentials'}), 401

    id = fetched[0]
    token = create_access_token(identity=id)
    refresh_token = create_refresh_token(identity=id)
    refresh_token_hash, refresh_token_salt = custom_hash(password=refresh_token)

    print(id, str(refresh_token_hash), refresh_token_salt)
    cur.execute('''UPDATE "Users"
                    SET refresh_token_hash = %s, refresh_token_salt = %s
                    WHERE user_id = %s''', (str(refresh_token_hash), refresh_token_salt, id))
    conn.commit()

    cur.close()
    conn.close()

    return jsonify({'message': 'Login successful', 'token':token, 'refresh_token':refresh_token, 'error':None})

```

```

@app.route('/get_name', methods=['GET'])
@jwt_required()
def get_name():
    id = get_jwt_identity()

    try:
        conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2],
password=conn_config[3], port=conn_config[4])
        cur = conn.cursor()
    except Exception as error:
        return jsonify({'error':error})

    cur.execute(''SELECT f_name, l_name, user_type FROM "Users" WHERE user_id=%s'', (id,))
    names = cur.fetchone()
    name = names[0]+ ' ' +names[1]
    type = names[2]

    cur.close()
    conn.close()

    return jsonify({'name':name, 'type':type})

@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()
    password_hash, salt = custom_hash(password=data.get('password'))

    try:
        conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2],
password=conn_config[3], port=conn_config[4])
        cur = conn.cursor()
    except Exception as error:
        return jsonify({'error':error})

    cur.execute(''SELECT email FROM "Users" WHERE email=%s'', (data.get('email'),))
    email_check = cur.fetchone()
    if email_check != None:
        return jsonify({"error": "Email in use"})

    cur.execute(''INSERT INTO "Users" (f_name, l_name, email, user_type, password_hash, salt)
VALUES (%s, %s, %s, %s, %s, %s)'', (data.get('fname'), data.get('sname'), data.get('email'),
data.get('type'), str(password_hash), salt))
    print('inserted users', data.get('fname'))
    conn.commit()
    cur.execute(''SELECT user_id FROM "Users"
WHERE email=%s AND password_hash=%s'', (data.get('email'), str(password_hash)))
    id = cur.fetchone()[0]
    print('got id')

```

```

    if data.get('type') == 'patient':
        prev_date = data.get('dob')
        new_date = prev_date[6:10] + '-' + prev_date[3:5] + '-' + prev_date[0:2]
        cur.execute('INSERT INTO Patient_Details (patient_id, dob)
                    VALUES (%s, %s)', (id, new_date))
    else:
        cur.execute('INSERT INTO Staff_Details (staff_id, verified)
                    VALUES (%s, 'Y')', (id, ))
    conn.commit()

    token = create_access_token(identity=id)
    refresh_token = create_refresh_token(identity=id)
    refresh_token_hash, refresh_token_salt = custom_hash(refresh_token)

    cur.execute('UPDATE "Users"
                SET refresh_token_hash = %s, refresh_token_salt = %s
                WHERE user_id = %s', (str(refresh_token_hash), refresh_token_salt, id))
    conn.commit()

    cur.close()
    conn.close()
    return jsonify({'message': 'registration successful', 'token':token, 'refresh_token':refresh_token,
                    'error':None})

@app.route('/fetchDates', methods=['GET'])
@jwt_required()
def fetch_dates():
    priority = request.args.get('priority')
    if not priority or priority == 'null':
        end_booking_session(id)
        return jsonify({'error': 'No appt passed'}), 400
    dates = processing_dates(priority) #outsourced to another file to reduce clutter
    if dates == 'no staff':
        return jsonify({'message':'unsuccessful', 'error':'no staff', 'dates':None})
    else:
        return jsonify({'message':'fetched successfully', 'dates':dates, 'error':None})

@app.route('/fetchAppointments', methods=['PUT'])
@jwt_required()
def fetchAppointments():
    data = request.get_json()
    priority = data.get('priority')
    dates = data.get('dates')
    times = data.get('times')
    id = get_jwt_identity()

    if not priority or priority == 'null':
        end_booking_session(id)

```

```

        return jsonify({'error': 'No appt passed'}), 400

    print(times)
    print(dates)
    print(priority)

    appointments = fetch_appointments(priority, dates, times, id) #outsourced to another file to reduce clutter

    return jsonify({'message': 'fetched successfully', 'appts': appointments, 'error': None}), 201

@app.route('/chooseAppointment', methods=['DELETE'])
@jwt_required()
def chooseAppointment():
    data = request.get_json()
    appt = data.get('appt')
    id = get_jwt_identity()
    if not appt or appt == 'null':
        end_booking_session(id)
        return jsonify({'error': 'No appt passed'}), 400

    confirmation = choose_appointment(appt, id)

    return jsonify({'msg': confirmation, 'error': None}), 204

@app.route('/endBookingSession', methods=['DELETE'])
@jwt_required()
def endBookingSession():
    id = get_jwt_identity()

    confirmation = end_booking_session(id)

    return jsonify({'msg': confirmation, 'error': None}), 204

@app.route('/fetchPast', methods=['GET'])
@jwt_required()
def fetchPast():
    id = get_jwt_identity()
    type = request.args.get('type')
    if not type:
        return jsonify({'error': 'No type passed'}), 400

    conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
    cur = conn.cursor()

    if type == 'patient':

```

```

        cur.execute('''SELECT f_name, l_name, TO_CHAR(appt_time, 'HH24:MI'), TO_CHAR(appt_date, 'DD/MM/YYYY'),
appt_details
                        FROM appointments a INNER JOIN staff_view v ON a.staff_id = v.user_id
                        WHERE patient_id = %s AND status='completed' ''', (id,))

        result = cur.fetchall()
        print(result)
        if result == []:
            return jsonify({'error': 'no appts'})

        appts = [{'staff_name': i[0] + ' ' + i[1], 'appt_time': i[2], 'appt_date': i[3], 'appt_details': i[4]} for i in
result]

        cur.close()
        conn.close()

        return jsonify({'appts': appts}), 200

    elif type == 'staff':

        cur.execute('''SELECT f_name, l_name, TO_CHAR(appt_time, 'HH24:MI'), TO_CHAR(appt_date, 'DD/MM/YYYY'),
appt_details
                        FROM appointments a INNER JOIN patient_view v ON a.patient_id = v.user_id
                        WHERE staff_id = %s AND status='completed' ''', (id,))

        result = cur.fetchall()
        if result == []:
            return jsonify({'error': 'no appts'})

        appts = [{'patient_name': i[0] + ' ' + i[1], 'appt_time': i[2], 'appt_date': i[3], 'appt_details': i[4]} for i in
result]

        cur.close()
        conn.close()

        return jsonify({'appts': appts}), 200

@app.route('/fetchFuture', methods=['GET'])
@jwt_required()
def fetchFuture():
    id = get_jwt_identity()
    type = request.args.get('type')
    if type == 'null':
        return jsonify({'error': 'No type passed'}), 400

    conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
    cur = conn.cursor()

    if type == 'patient':

```

```

        cur.execute('''SELECT f_name, l_name, TO_CHAR(appt_time, 'HH24:MI'), TO_CHAR(appt_date, 'DD/MM/YYYY'),
appt_details
                        FROM appointments a INNER JOIN staff_view v ON a.staff_id = v.user_id
                        WHERE patient_id = %s AND status='scheduled' ''', (id,))

        result = cur.fetchall()
        print(result)
        if result == []:
            return jsonify({'error': 'no appts'})

        appts = [{'staff_name': i[0]+' '+i[1], 'appt_time': i[2], 'appt_date': i[3], 'appt_details': i[4]} for i in
result]

        cur.close()
        conn.close()

        return jsonify({'appts': appts}), 200

    elif type == 'staff':

        cur.execute('''SELECT f_name, l_name, TO_CHAR(appt_time, 'HH24:MI'), TO_CHAR(appt_date, 'DD/MM/YYYY'),
appt_details
                        FROM appointments a INNER JOIN patient_view v ON a.patient_id = v.user_id
                        WHERE staff_id = %s AND status='scheduled' ''', (id,))

        result = cur.fetchall()
        if result == []:
            return jsonify({'error': 'no appts'})

        appts = [{'patient_name': i[0]+' '+i[1], 'appt_time': i[2], 'appt_date': i[3], 'appt_details': i[4]} for i in
result]

        cur.close()
        conn.close()

        return jsonify({'appts': appts}), 200

@app.route('/fetchApptAmount', methods=['GET'])
@jwt_required()
def fetchApptAmount():
    id = get_jwt_identity()
    type = request.args.get('type')
    print(type)

    conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
    cur = conn.cursor()

    if type == 'patient':
        cur.execute('''SELECT appt_id
                        FROM appointments
                        WHERE status = 'scheduled' AND patient_id = %s ''', (id,))

        amount = len(cur.fetchall())

```

```

        print(amount)
        return jsonify({'amount':amount}), 200

    elif type == 'staff':
        cur.execute('''SELECT appt_id
                        FROM appointments
                        WHERE status = 'scheduled' AND staff_id = %s ''', (id,))
        amount = len(cur.fetchall())

        now = datetime.datetime.now()
        today = now.strftime('%d/%m/%Y')
        cur.execute('''SELECT appt_id
                        FROM appointments
                        WHERE status = 'scheduled' AND staff_id = %s AND appt_date = %s''', (id, today))
        today_amount = len(cur.fetchall())
        print(amount)
        print(today_amount)

        cur.close()
        conn.close()

        return jsonify({'amount':amount, 'todayAmount':today_amount}), 200
    else:

        cur.close()
        conn.close()

        return jsonify({'error': 'No type passed'}), 400

@app.route('/refreshJWT', methods=['POST'])
@jwt_required(refresh=True)
def refreshJWT():
    id = get_jwt_identity()

    header = request.headers.get('Authorization')
    old_refresh_token = header[7:]

    conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
    cur = conn.cursor()

    cur.execute('''SELECT refresh_token_salt
                    FROM "Users"
                    WHERE user_id = %s''', (id, ))
    refresh_token_salt = cur.fetchone()[0]

    token_hash, refresh_token_salt = custom_hash(password=old_refresh_token, salt=refresh_token_salt)
    cur.execute('''SELECT user_id
                    FROM "Users"
                    WHERE user_id=%s AND refresh_token_hash=%s''',

```

```

        (id, str(token_hash)))
    fetched = cur.fetchone()

    if fetched == None:
        return jsonify({'error': 'invalid token'}), 401

    new_access_token = create_access_token(identity=id)
    new_refresh_token = create_refresh_token(identity=id)
    refresh_token_hash, refresh_token_salt = custom_hash(password=new_refresh_token, salt=refresh_token_salt)

    cur.execute('''UPDATE "Users"
                  SET refresh_token_hash = %s, refresh_token_salt = %s
                  WHERE user_id = %s''', (str(refresh_token_hash), refresh_token_salt, id))
    conn.commit()

    cur.close()
    conn.close()

    return jsonify({'message': 'token sent', 'token': new_access_token, 'refresh_token': new_refresh_token,
                    'error': None})

# starts the backend
if __name__ == '__main__':
    app.run(debug=True)

```

fetch_dates.py

This file contains the function for retrieval of all the available dates for the next 3 months, in dictionary form. The process is explained in the UML Sequence Diagram. This is the first time that I practiced the usage of the datetime and calendar libraries, which is the reason why I know how to use some of their methods much better than I used to.

```

import calendar
import datetime
import os
from dotenv import load_dotenv
import psycopg2

load_dotenv()
conn_config = [os.getenv("HOST"), os.getenv("DBNAME"), os.getenv("USER"), os.getenv("PASSWORD"), os.getenv("PORT")]

def processing_dates(priority):
    conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
    cur = conn.cursor()

```

```

# 24 available appointments a day PER verified doctor (staff)
cur.execute('''SELECT staff_id FROM Staff_Details WHERE verified='Y' ''')
amount = len(cur.fetchall())
daily_amount = 24*amount
if daily_amount == 0:
    return 'no staff'

now = datetime.datetime.now()
current_month = int(now.strftime('%m'))
current_year = int(now.strftime('%Y'))

days_list = []
for i in range(current_month, current_month+3):
    if i > 12:
        j = i - 12
        y = current_year+1
    else:
        j = i
        y = current_year
    days_list.append(
        [str(d).zfill(2)+'/'+str(j).zfill(2)+'/'+str(y) for d in range(1,calendar.monthrange(y, j)[-1]+1)]
    )

dates = {'dates':[dict.fromkeys(days_list[i], '') for i in range(3)]}

for i in range(3):
    cur.execute('''SELECT TO_CHAR(appt_date, 'DD/MM/YYYY')
                  FROM Appointments
                  WHERE status = 'scheduled' AND appt_date BETWEEN %s AND %s''',
              (days_list[i][0], days_list[i][-1]))
    appts = cur.fetchall()

    for key in dates['dates'][i]:
        count = appts.count((key, ))
        if count < daily_amount:
            dates['dates'][i][key] = True
        else:
            dates['dates'][i][key] = False

for i in range(3):
    cur.execute('''SELECT TO_CHAR(appt_date, 'DD/MM/YYYY')
                  FROM Appointments
                  WHERE status = 'scheduled' AND (appt_date BETWEEN %s AND %s) AND priority>%s''',
              (days_list[i][0], days_list[i][-1], priority))
    appts = cur.fetchall()

    if len(appts) == 0:
        break

    for key in dates['dates'][i]:

```

```

        if (key, ) in appts and not dates['dates'][i][key]:
            dates['dates'][i][key] = True

    for key in dates['dates'][0]:
        if key == now.strftime('%d/%m/%Y'):
            dates['dates'][0][key] = False
            break
        else:
            dates['dates'][0][key] = False

    cur.close()
    conn.close()

    return dates

```

booking_algorithm.py

This file is a collection of 3 functions. The first function performs the processing for fetching all the available appointments based on user specifications, the second function performs cleanup for the first function after locking in the appointment choice for the user, and the third function performs cleanup code without locking in an appointment choice. All the processes are explained in the UML sequence diagram.

```

import os
from dotenv import load_dotenv
import psycpg2
import datetime
import yagmail

load_dotenv()
conn_config = [os.getenv("HOST"), os.getenv("DBNAME"), os.getenv("USER"), os.getenv("PASSWORD"), os.getenv("PORT")]

def fetch_appointments(priority, dates, times, id):

    # converting times + dates to proper formats:

    times_list = times.split(', ')
    times_list.remove('')
    # this is a hardcoded fact
    times_dict = {'09:00-13:00':['09:00', '09:30', '10:00', '10:30', '11:00', '11:30', '12:00', '12:30'],
                  '13:00-17:00':['13:00', '13:30', '14:00', '14:30', '15:00', '15:30', '16:00', '16:30'],
                  '17:00-21:00':['17:00', '17:30', '18:00', '18:30', '19:00', '19:30', '20:00', '20:30']}

    times = []
    for time in times_list:
        times += times_dict[time]

    dates_list = dates.split(', ')
    dates_list.remove('')

```

```

    dates = dates_list

    # first we get valid doctor ids for the appointments

    conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
    cur = conn.cursor()

    cur.execute('''SELECT user_id, f_name, l_name
                    FROM staff_view
                    WHERE verified='Y' ''')
    staff_list = cur.fetchall()

    staff_dict = {}
    for member in staff_list:
        staff_dict[member[0]] = member[1] + ' ' + member[2]

    priorities = [1,2,3]
    for i in range(1, 4):
        if priority > i:
            priorities.remove(i)
        if priority <= i:
            break

    # list of appts (where an appt is a dict) generator, based on user preferences
    generated_appts = []
    required_appts = []
    for day in dates:
        for timeslot in times:
            required_appts.append({'patient_id':id, 'timeslot':timeslot, 'timeslot':timeslot.zfill(5), 'date':day,
'priority':priority})
            for staff_id in staff_dict:
                generated_appts.append({'patient_id':id, 'staff_id':staff_id, 'staff_name':staff_dict[staff_id],
'timeslot':timeslot.zfill(5), 'date':day, 'priority':priority})

    all_appts_final = []
    repeated_appts_list = []

    cur.execute('''SELECT appt_id, staff_id, TO_CHAR(appt_time, 'HH24:MI'), appt_date, priority, status, patient_id
                    FROM appointments
                    WHERE appt_time IN %s AND appt_date IN %s AND (status != 'completed')
                    ORDER BY appt_date, appt_time''',
                    (tuple([time.zfill(5) for time in times]), tuple(dates)))
    result = cur.fetchall()
    existing_appts = []
    for entry in result:
        if entry[6] != id:

```

```

        existing_appts.append({'appt_id':entry[0], 'staff_id':entry[1],
'staff_name':staff_dict[entry[1]], 'timeslot':entry[2], 'date':entry[3].strftime('%d/%m/%Y'), 'priority':entry[4],
'status':entry[5]})
    else:
        repeated_appts_list.append([entry[3].strftime('%d/%m/%Y'), entry[2]])

# MAKE COMPARISON BETWEEN generated_appts AND existing_appts, CHECKING TIME, STAFF, AND DATE.
# IF THERE IS A PERFECT MATCH, DO NOT ADD IT TO THE FINAL, ELSE ADD IT GIVEN THAT THE SAME TIME HASN'T ALREADY
BEEN ADDED

for appt in generated_appts:
    conflict = False
    for entry in existing_appts:
        if (entry['timeslot'] == appt['timeslot'] and entry['date'] == appt['date'] and entry['staff_id'] ==
appt['staff_id']):
            conflict = True
            break
    if not conflict and not ([appt['date'], appt['timeslot']] in repeated_appts_list):
        all_appts_final.append(appt)
        repeated_appts_list.append([appt['date'], appt['timeslot']])

for appt in existing_appts:
    if [appt['date'], appt['timeslot']] not in repeated_appts_list:
        if appt['priority'] > priority and appt['status'] == 'scheduled':
            all_appts_final.append(appt)
            repeated_appts_list.append([appt['date'], appt['timeslot']])
    next

insert = []
update = []
for appt in all_appts_final:
    try:
        appt_id = appt['appt_id']
        status = f'scheduled + reserved by {id}'
        update.append([status, appt_id])
    except KeyError:
        temp = list(appt.values())
        temp.append('reserved')
        temp.pop(2)
        insert.append(temp)

cur.executemany('INSERT INTO appointments (patient_id, staff_id, appt_time, appt_date, priority, status)
VALUES (%s, %s, %s, %s, %s, %s)', insert)

conn.commit()

cur.executemany('UPDATE appointments
SET status = %s
WHERE appt_id = %s', update)

conn.commit()

```

```

cur.close()
conn.close()

return all_appts_final

def choose_appointment(appt, id):

    conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
    cur = conn.cursor()

    date = appt['date']
    time = appt['timeslot']
    priority = appt['priority']
    staff_id = appt['staff_id']
    appt_details = appt['appt_details']

    try:
        appt_id = appt['appt_id']
    except KeyError:
        appt_id = None

    if not appt_id:

        # not replacing an appt

        cur.execute('''UPDATE appointments
                        SET status = 'scheduled', appt_details = %s
                        WHERE appt_date = %s AND appt_time = %s AND priority = %s AND staff_id = %s AND patient_id = %s
                        ''',
                    (appt_details ,date, time, priority, staff_id, id))
        conn.commit()

        cur.execute('''DELETE FROM appointments
                        WHERE patient_id = %s AND status = 'reserved' ''', (id, ))
        conn.commit()

        cur.execute('''UPDATE appointments
                        SET status = 'scheduled'
                        WHERE status = %s''', (f"scheduled + reserved by {id}",))
        conn.commit()

    else:

        # replacing and rescheduling an appt

        cur.execute('''DELETE FROM appointments
                        WHERE patient_id = %s AND status = 'reserved' ''', (id, ))

```

```

conn.commit()

cur.execute('''SELECT patient_id, priority, appt_date, appt_details, email
                FROM appointments a INNER JOIN patient_view v ON a.patient_id = v.user_id
                WHERE appt_id = %s''', (appt_id, ))
other_id, other_priority, appt_date, other_appt_details, other_email = cur.fetchone()
other_priority -= 1

cur.execute('''UPDATE appointments
                SET patient_id = %s, priority = %s, status = 'scheduled', appt_details = %s
                WHERE appt_id = %s''', (id, priority, appt_details, appt_id))
conn.commit()

cur.execute('''SELECT user_id
                FROM staff_view
                WHERE verified='Y' ''')
staff = cur.fetchall()
staff_list = [i[0] for i in staff]

times = ['09:00', '09:30', '10:00', '10:30', '11:00', '11:30', '12:00', '12:30',
          '13:00', '13:30', '14:00', '14:30', '15:00', '15:30', '16:00', '16:30',
          '17:00', '17:30', '18:00', '18:30', '19:00', '19:30', '20:00', '20:30']

timechange = datetime.timedelta(days=1)

flag = False
while not flag:

    day_str = appt_date.strftime('%d/%m/%Y')

    cur.execute('''SELECT staff_id, TO_CHAR(appt_time, 'HH24:MI')
                    FROM appointments
                    WHERE appt_date = %s''', (day_str,))
    fetched_appts = cur.fetchall()
    print(fetched_appts)

    for timeslot in times:
        print(timeslot)
        for staff_id in staff_list:
            print(staff_id)
            if not ((staff_id, timeslot) in fetched_appts):
                chosen_appt = (other_id, staff_id, timeslot, day_str, other_priority, 'scheduled',
other_appt_details)

                flag = True
                break
            else:
                next

        if flag:
            break

```

```

        appt_date += timechange

        cur.execute('''INSERT INTO appointments (patient_id, staff_id, appt_time, appt_date, priority, status,
appt_details)

                        VALUES (%s, %s, %s, %s, %s, %s, %s)''', chosen_appt)
        conn.commit()

        cur.execute('''UPDATE appointments

                        SET status = 'scheduled'

                        WHERE status = %s''', (f"scheduled + reserved by {id}",))
        conn.commit()

        yag = yagmail.SMTP("gpqueue.nea@gmail.com", os.getenv('APP_PWD'))
        yag.send(
            to=other_email,
            subject="Important Appointment Information",
            contents=f'''Your appointment has been rescheduled.

            Check the website for more details.'''
        )

        cur.close()
        conn.close()

        return 'success'

def end_booking_session(id):

    conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
    cur = conn.cursor()

    cur.execute('''DELETE FROM appointments

                    WHERE patient_id = %s AND status = 'reserved' ''', (id, ))
    conn.commit()

    cur.execute('''UPDATE appointments

                    SET status = 'scheduled'

                    WHERE status = %s''', (f"scheduled + reserved by {id}",))
    conn.commit()

    return 'success'

```

MyHash.py

This is a file where I coded my own password hashing algorithm, utilising the hashlib and os libraries. Originally it was going to be larger, but I was able to abstract a lot. It is kept in a separate file/function for better modularity.

```
import os
```

```

import hashlib

def custom_hash(password, salt=None):

    if not salt:
        used_salt = os.urandom(16) # for uniqueness
    else:
        used_salt = salt

    hash_value = hashlib.pbkdf2_hmac('sha256', password.encode(), used_salt, 100000) # very strong recursive hashing algorithm
    hash_value_int = int.from_bytes(hash_value, byteorder='big')

    return hash_value_int, used_salt

```

Cronjob.py

This is the file in which I wrote my cronjob script. This script is run every 1st and 31st minute of every hour.

```

# This will be the script that my cronjob runs
#first we change appt status, then we do reminders

import psycpg2
from dotenv import load_dotenv
import os
import datetime
import yagmail

load_dotenv()
conn_config = [os.getenv("HOST"), os.getenv("DBNAME"), os.getenv("USER"), os.getenv("PASSWORD"), os.getenv("PORT")]

now = datetime.datetime.now()
date = now.strftime('%Y-%m-%d')
time = now.strftime('%H:%M')
timechange_week = datetime.timedelta(days=7)
timechange_day = datetime.timedelta(days=1)
timechange_appt = datetime.timedelta(minutes=29)
week = (now + timechange_week).strftime('%d/%m/%Y')
day = (now + timechange_day).strftime('%d/%m/%Y')
next_appt = (now + timechange_appt).strftime('%H:%M')

conn = psycpg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3],
port=conn_config[4])
cur = conn.cursor()

print(date, time)

cur.execute('''UPDATE appointments

```

```

        SET status = 'completed'
        WHERE appt_date <= %s AND appt_time < %s '', (date, time,))
conn.commit()
yag = yagmail.SMTP("gpqueue.nea@gmail.com", os.getenv('APP_PWD'))

#week before reminder for patients
if time == '09:01':
    cur.execute('''SELECT patient_id
                  FROM appointments
                  WHERE appt_date = %s'', (week,))
    result = cur.fetchall()
    result = tuple([i[0] for i in result])

    if len(result) > 0:
        cur.execute('''SELECT email
                      FROM "Users"
                      WHERE user_id IN %s'', (result, ))
        emails = [i[0] for i in cur.fetchall()]
        for email in emails:
            yag.send(
                to=email,
                subject="Appointment Reminder for Next Week",
                contents=f'''This is your reminder email! You have an appointment in a week on {week}.
                Check the website for more details.'''
            )

#day before reminder for patients
cur.execute('''SELECT patient_id
              FROM appointments
              WHERE appt_date = %s'', (day,))
result = cur.fetchall()
result = tuple([i[0] for i in result])

if len(result) > 0:
    cur.execute('''SELECT email
                  FROM "Users"
                  WHERE user_id IN %s'', (result, ))
    emails = [i[0] for i in cur.fetchall()]
    for email in emails:
        yag.send(
            to=email,
            subject="Appointment Reminder for Tomorrow",
            contents=f'''This is your reminder email! You have an appointment in a day on {day}.
            Check the website for more details.'''
        )

# 30min before reminder for staff
cur.execute('''SELECT staff_id
              FROM appointments

```

```

        WHERE appt_date = %s AND appt_time = %s'', (date, next_appt))
result = cur.fetchall()
result = tuple([i[0] for i in result])

if len(result) > 0:
    cur.execute('''SELECT email
                    FROM "Users"
                    WHERE user_id IN %s'', (result, ))
    emails = [i[0] for i in cur.fetchall()]
    for email in emails:
        yag.send(
            to=email,
            subject="30 Mintue Appointment Reminder",
            contents=f'''This is your reminder email! You have an appointment with a patient in 30 minutes at
{next_appt}.
                    Check the website for more details.'''
        )

cur.close()
conn.close()

```

PostgreSQL Scripts

While creating my database, and performing tests with it, I had many PGSQL (PostgreSQL) scripts. One of these is the one which creates my database. The rest are just for testing purposes, and were constantly modified, so some of the values will not make sense.

DB.pqsql

This is the script containing my database build. It creates the user, patient_details, staff_details, and appointments tables. It also creates the patient_view and staff_view views and sets the DateStyle to by DMY so that getting dates from the DB using a SELECT statement is much easier.

```
ALTER DATABASE "GPQueue" SET DateStyle = 'DMY';

CREATE TABLE IF NOT EXISTS "Users" (
    user_id SERIAL PRIMARY KEY,
    f_name VARCHAR(100) NOT NULL,
    l_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    user_type VARCHAR(255) NOT NULL CHECK ( user_type IN ('patient', 'staff')),
    password_hash TEXT NOT NULL,
    salt BYTEA NOT NULL,
    refresh_token_hash TEXT,
    refresh_token_salt BYTEA,
    register_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS Patient_Details (
    patient_id INTEGER PRIMARY KEY,
    dob DATE NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES "Users"(user_id)
);

CREATE TABLE IF NOT EXISTS Staff_Details (
    staff_id INTEGER PRIMARY KEY,
    verified VARCHAR(5) NOT NULL CHECK (verified IN ('Y', 'N')),
    FOREIGN KEY (staff_id) REFERENCES "Users"(user_id)
);

CREATE TABLE IF NOT EXISTS Appointments (
    appt_id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL,
    staff_id INTEGER NOT NULL,
    priority INTEGER NOT NULL,
    appt_date DATE NOT NULL,
    appt_time TIME NOT NULL,
    appt_details TEXT,
    creation_date TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```

    status TEXT NOT NULL,
    -- RESERVED IS FOR APPOINTMENTS THAT MAY BE CHOSEN BY THE USER WHILE BOOKING,
    -- SCHEDULED + RESERVED IS FOR APPOINTMENTS THAT MAY BE CHOSEN BY THE USER WHILE BOOKING,
    -- WHILE THE APPT IS ALREADY SCHEDULED WITH ANOTHER USER
    -- status IN ('scheduled', 'completed', 'reserved', 'scheduled + reserved by <OTHER ID>'))
    FOREIGN KEY (patient_id) REFERENCES Patient_Details(patient_id),
    FOREIGN KEY (staff_id) REFERENCES Staff_Details(staff_id)
);

CREATE VIEW Patient_View AS
SELECT
    u.user_id,
    u.f_name,
    u.l_name,
    u.email,
    u.password_hash,
    u.register_date,
    u.salt,
    pd.dob
FROM "Users" u
JOIN Patient_Details pd ON u.user_id = pd.patient_id;

CREATE VIEW Staff_View AS
SELECT
    u.user_id,
    u.f_name,
    u.l_name,
    u.email,
    u.password_hash,
    u.register_date,
    u.salt,
    sd.verified
FROM "Users" u
JOIN Staff_Details sd ON u.user_id = sd.staff_id;

```

InsertStuff.pgsql

This file was created for random insert statements, usually a onetime thing. There is no point in showing the contents of this file, as they change all the time.

InsertTestUsers.pgsql

This file contains a script, of which parts of were generated through the use of AI; specifically, all the user details. This is only because it was an easy, tedious task, otherwise I would've done it myself, as I have for all the other files.

```

BEGIN;

-- Insert Users (2 patients and 3 staff)
INSERT INTO "Users" (f_name, l_name, email, user_type, password_hash, salt)

```

```

VALUES
    ('John', 'Doe', 'johndoe@example.com', 'patient', 'hash12345', decode('aabbcc', 'hex')), -- Patient 1
    ('Jane', 'Smith', 'janesmith@example.com', 'patient', 'hash67890', decode('ddeeff', 'hex')), -- Patient 2
    ('Dr. Alice', 'Johnson', 'alicejohnson@example.com', 'staff', 'hashabcde', decode('112233', 'hex')), -- Staff 1
    ('Dr. Bob', 'Brown', 'bobbrown@example.com', 'staff', 'hashfghij', decode('445566', 'hex')), -- Staff 2
    ('Dr. Clara', 'Evans', 'claraevans@example.com', 'staff', 'hashklmno', decode('778899', 'hex')); -- Staff 3

INSERT INTO Patient_Details (patient_id, dob)
SELECT user_id, '1990-01-15' FROM "Users" WHERE email = 'johndoe@example.com';

INSERT INTO Patient_Details (patient_id, dob)
SELECT user_id, '1985-06-22' FROM "Users" WHERE email = 'janesmith@example.com';

INSERT INTO Staff_Details (staff_id, verified)
SELECT user_id, 'Y' FROM "Users" WHERE email = 'alicejohnson@example.com';

INSERT INTO Staff_Details (staff_id, verified)
SELECT user_id, 'Y' FROM "Users" WHERE email = 'bobbrown@example.com';

INSERT INTO Staff_Details (staff_id, verified)
SELECT user_id, 'Y' FROM "Users" WHERE email = 'claraevans@example.com';

COMMIT;

```

populate.pgsql

This file was created in order to populate the appointments table with a large volume of appointments. I would often change the values that would be input, so those are not important.

```

BEGIN;

DO $$
DECLARE
    start_time TIME := '09:00';
    end_time TIME := '12:00';
    "incrementing_time" TIME := start_time;
    staff_ids INTEGER[] := ARRAY[5,3,4];
    patient_ids INTEGER[] := ARRAY[1,2];
    appt_date DATE := '2025-01-14';
    staff_index INTEGER := 1;
    patient_index INTEGER := 1;
    appointment_count INTEGER := 3;
BEGIN
    WHILE "incrementing_time" <= end_time LOOP
        INSERT INTO Appointments (patient_id, staff_id, priority, appt_date, appt_time, status)
        VALUES (

```

```

        patient_ids[patient_index],
        staff_ids[staff_index],
        appointment_count,
        appt_date,
        LEFT("incrementing_time"::VARCHAR, 5),
        'scheduled'
    );

    staff_index := 3 - staff_index;
    patient_index := 3 - patient_index;

    "incrementing_time" := "incrementing_time" + INTERVAL '30 minutes';
END LOOP;
END $$;

COMMIT;

```

WIPE.pgsql

This file, as you can guess from the file name, wipes all the tables in the database clean. This is so that I can have a fresh start if I have too many entries in specific tables. This is also useful if I only need to delete the entities in one or two tables, as I can comment out everything else. The statements executed at the beginning and end disable and re-enable the foreign key constraints, so that the database doesn't stop the file from executing when I attempt to wipe tables. I also used this to delete specific entities, as shown in the current iteration of this file below (at the time of writing):

```

DO $$
BEGIN
    EXECUTE 'SET session_replication_role = replica';

    -- EXECUTE 'DELETE FROM "Users" WHERE user_id=5';
    -- EXECUTE 'DELETE FROM staff_details WHERE staff_id=5';

    -- EXECUTE 'DELETE FROM Appointments';
    --EXECUTE 'DELETE FROM Patient_Details';
    --EXECUTE 'DELETE FROM Staff_Details';

    -- EXECUTE 'DELETE FROM "Users"';

    EXECUTE 'SET session_replication_role = DEFAULT';
END $$;

```


Testing

This is the testing section. Since for most of my project, the user inputs information through buttons, it is practically impossible for the user to input incorrect information to crash the program, except in the Login and Registration pages. Here I will be testing my different sections, with a range of possible user inputs, and checking if the correct event occurs, or the correct data is displayed. My evidence will be supplied with screenshots of the input, the result, and data in database tables if required. At the end, I will perform Acceptance testing on each of my objectives to determine if I have met my requirements.

Modular Testing

Note: I have turned some features off during testing. For instance, the passwords will usually show up as dots, however I have turned this off during testing.

Testing

Test No.	Function/ Module/ Page involved	Test type:	Data input	Expected outcome	Actual outcome	Pass/Fail
1a	Login Page/Registration page	Normal	Clicking the link on the login page	Registration page is displayed	Registration page is displayed	Pass
1b	Login Page/Registration page	Normal	Clicking the link on the registration page	Login page is displayed	Login page is displayed	Pass
2a	Registration page	Normal	Valid data is entered, and Register! button is clicked.	User is taken to patient homepage, DB is updated	User is taken to patient homepage, DB is updated	Pass
2b	Registration page	Erroneous	Existing email is entered	Email in use error message is shown; DB is not updated	Email in use error message is shown; DB is not updated	Pass
2c	Registration page	Erroneous	Password not meeting requirements is entered	Fields error message is shown; DB is not updated	Fields error message is shown; DB is not updated	Pass
2d	Registration page	Erroneous	An invalid DOB is entered	Fields error message is shown; DB is not updated	Fields error message is shown; DB is not updated	Pass
2e	Registration page	Normal	Valid data is entered, and register as staff button is clicked.	User is taken to a page with a message, DB is updated	User is taken to a page with a message, DB is updated	Pass
3a	Login Page	Normal	Existing email, correct password	User is taken to homepage	User is taken to homepage	Pass
3bi	Login Page	Normal	Existing email, incorrect password	Error message is displayed	Error message was not displayed	Fail
3bii	Login Page	Normal	Incorrect email or password	Error message is displayed	Error message is displayed	Pass
4a	Patient Homepage	Normal	Pressing button to go to Future Appointments	Future Appointments page is displayed	Future Appointments page is displayed	Pass
4b	Patient Homepage	Normal	Pressing button to go to Past Appointments	Past Appointments page is displayed	Past Appointments page is displayed	Pass

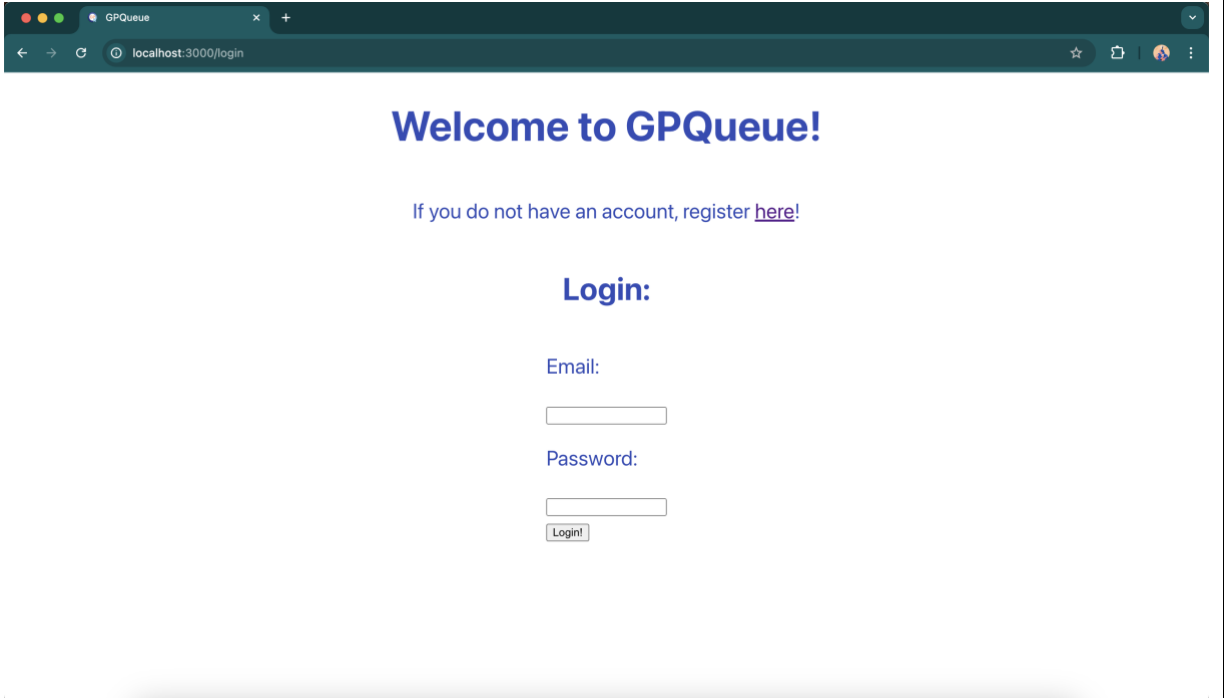
4c	Patient Homepage	Normal	Pressing button to go to BookAppointment1	BookAppointment1 page is displayed	BookAppointment1 page is displayed	Pass
4d	Patient Homepage	Normal	Entering the page while patient has X future appointments (where $X \geq 0$)	Appointment count message reads the number of future appointments.	Appointment count message reads the number of future appointments.	Pass
5a	BookAppointment1	Normal	Selecting a reason, then pressing Continue.	Reason is displayed at the top, then user is routed to BookAppointment2	Reason is displayed at the top, then user is routed to BookAppointment2	Pass
5b	BookAppointment1	Normal	Selecting a reason, selecting another, then pressing Continue.	Reason is displayed at the top, then it is replaced with the next reason, then user is routed to BookAppointment2	Reason is displayed at the top, then it is replaced with the next reason, then user is routed to BookAppointment2	Pass
5c	BookAppointment1	Erroneous	Pressing Continue without selecting a reason	Custom error message is displayed.	Custom error message is displayed	Pass
5di	BookAppointment1	Erroneous	Selecting a reason and pressing continue when there are no staff registered	User is sent back to the homepage	User was sent to the loading screen for BookAppointments2	Fail
5dii	BookAppointment1	Erroneous	Selecting a reason and pressing continue when there are no staff registered	User is sent back to the homepage	User was sent back to the homepage	Pass
6a	BookAppointment2	Normal	Pressing a couple green dates, then pressing continue	All the chosen dates are shown, then the user is taken to the BookAppointment3 page	All the chosen dates are shown, then the user is taken to the BookAppointment3 page	Pass
6b	BookAppointment2	Normal	Pressing the same green date twice	The date is shown, then removed	The date was shown, then removed	Pass
6c	BookAppointment2	Normal	Pressing next month, choosing some green dates, then pressing previous month	Month is changed, chosen dates are displayed, then month is changed back	Month is changed, chosen dates are displayed, then month is changed back	Pass
6d	BookAppointment2	Erroneous	Pressing next month 4 times	Month is changed twice, then button stops working after the second press	Month changed twice, then stopped working after the second press	Pass
6e	BookAppointment2	Erroneous	Pressing next month twice, then pressing previous month 4 times	Month is changed twice, then changed back to the original, then the button stops working	Month is changed twice, then changed back to the original, then the button stops working	Pass
6f	BookAppointment2	Erroneous	Pressing continue without choosing dates	Custom error message is displayed.	Custom error message was displayed	Pass
6g	BookAppointment2	Normal	Entering the page, while every appointment on a day (26/01/2025) is booked	The day is greyed out.	The day is greyed out.	Pass
7a	BookAppointment3	Erroneous	Clicking continue without choosing a timeframe	A custom error message is displayed	A custom error message is displayed	Pass

7b	BookAppointment3	Normal	Clicking on a timeframe, then clicking on it again	The timeframe is displayed, then removed	The timeframe is displayed, then removed	Pass
7c	BookAppointment3	Normal	Choosing a timeframe then clicking continue	The final booking page is displayed.	The final booking page is displayed.	Pass
8a	BookAppointment4	Normal	Entering the page, with previous choices including a low priority level and a clear date and any timeframe.	The page is displayed with fetched data shown correctly. Database is filled with reserved appointments.	The page is displayed with fetched data shown correctly. Database is filled with reserved appointments.	Pass
8bi	BookAppointment4	Normal	Entering the page, with previous choices including a high priority level and a booked date and any timeframe.	The page is displayed with fetched data shown correctly, with an added message for each appointment saying that choosing it will mean replacing it. Database is updated with reserved appointments. Pressing the forward/previous buttons will show other appointments.	The page is displayed with fetched data shown correctly, without the replacement message. The replacement message appears once clicking the forward button, but the displayed appt details do not actually change.	Fail
8bii	BookAppointment4	Normal	Entering the page, with previous choices including a high priority level and a booked date and any timeframe.	The page is displayed with fetched data shown correctly, with an added message for each appointment saying that choosing it will mean replacing it. Database is updated with reserved appointments.	The page is displayed with fetched data shown correctly, with an added message for each appointment saying that choosing it will mean replacing it. Database is updated with reserved appointments.	Pass
8ci	BookAppointment4	Normal	Pressing Book on an appointment where it already is assigned to someone	The appointment will be reassigned, and the user will be taken home. The other reserved appointments are updated back to their original state.	Syntax error occurred in the backend, handled by error handling, but reserved appointments not updated back and appointment not reassigned.	Fail
8cii	BookAppointment4	Normal	Pressing Book on an appointment where it already is assigned to someone	The appointment will be reassigned, and the user will be taken home. The other reserved appointments are updated back to their original state.	The appointment will be reassigned, and the user will be taken home. The other reserved appointments are updated back to their original state.	Pass
8d	BookAppointment4	Normal	Pressing forward and previous	When forward is pressed, the next appointment is	When forward is pressed, the next appointment is	Pass

				shown. When previous is pressed, the previous appointment is shown.	shown. When previous is pressed, the previous appointment is shown.	
8e	BookAppointment4	Normal	Pressing Book on an unassigned appointment.	Appointment is changed in the database and the user is taken home, and other reserved appointments are removed.	Appointment is changed in the database and the user is taken home, and other reserved appointments are removed.	Pass
8f	BookAppointment4	Normal	Pressing Exit.	Reserved appointments are removed/changed back to their original state, and the user is taken home.	Reserved appointments are removed/changed back to their original state, and the user is taken home.	Pass
8g	BookAppointment4	Normal	Letting the timer run out.	Reserved appointments are removed/changed back to their original state, and the user is taken to the timeout page.	Reserved appointments are removed/changed back to their original state, and the user is taken to the timeout page.	Pass
9	Timeout page	Normal	Pressing the button	User is taken home.	User is taken home.	Pass
10a	Future Appointments page (for patients)	Normal	Patient enters page while they have no future appointments	No appointments message is displayed.	No appointments message is displayed.	Pass
10b	Future Appointments page (for patients)	Normal	Patient enters page while they have future appointments	The appointments are displayed correctly	The appointments are displayed correctly	Pass
11a	Future Appointments page (for staff)	Normal	Staff enters page while they have no future appointments	No appointments message is displayed	No appointments message is displayed	Pass
11b	Future Appointments page (for staff)	Normal	Staff enters page while they have future appointments	Appointments are displayed correctly, with forward/previous buttons	Appointments are displayed correctly, with forward/previous buttons	Pass
11c	Future Appointments page (for staff)	Normal	Staff presses forward and previous buttons	Next triplet of appointments is displayed, then previous triplet of appointments is displayed	Next triplet of appointments is displayed, then previous triplet of appointments is displayed	Pass
12a	Past Appointments page	Normal	User enters page while they have no past appointments	No appointments message is displayed	No appointments message is displayed	Pass
12b	Past Appointments page	Normal	User enters page while they have past appointments	Appointments are displayed correctly, with forward/previous buttons	Appointments are displayed correctly, with forward/previous buttons	Pass
12c	Past Appointments page	Normal	User presses forward and previous buttons	Next triplet of appointments is displayed, then previous triplet of appointments is displayed	Next triplet of appointments is displayed, then previous triplet of appointments is displayed	Pass

13a	Staff homepage	Normal	Pressing button to go to Future Appointments	Future Appointments page is displayed	Future Appointments page is displayed	Pass
13b	Staff homepage	Normal	Pressing button to go to Past Appointments	Past Appointments page is displayed	Past Appointments page is displayed	Pass
13c	Staff homepage	Normal	Entering the page while the staff members has an X amount of appointments today, and Y amount of appointments in the future.	Future Appointment count message reads Y, and Today Appointment count message reads X.	Future Appointment count message reads Y, and Today Appointment count message reads X.	Pass
14	Any Page	Erroneous	Editing the URL to access any page	User is routed back to Login	User is routed back to Login	Pass
15	Any Page with a back button	Normal	Pressing the back button	User is routed back to the previous page.	User is routed back to the previous page.	Pass
16a	Any Page with the top bar.	Normal	Pressing the Logo	User is routed back to their correct home page, which displays correctly	Staff member was routed back to the patient homepage with a glitched appointment count	Fail
16b	Any Page with the top bar.	Normal	Pressing the Logo	User is routed back to their correct home page, which displays correctly		

Evidence

Test No.	Screenshots
1a	Before: 

After:

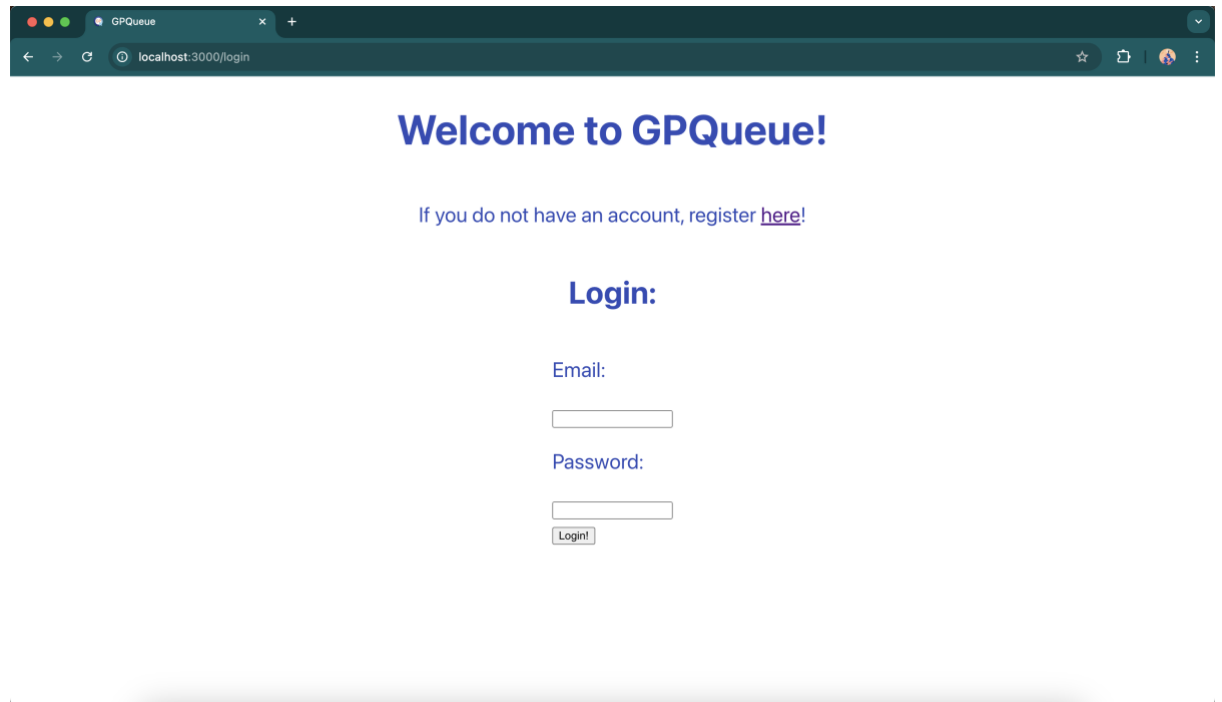
The screenshot shows a web browser window with the address bar displaying 'localhost:3000/register'. The page content includes a large blue heading 'Welcome to GPQueue!', a link 'here!' for login, and a 'Register:' section. The registration form fields are: Email, Password, Confirm Password, First Name, Surname, and Date of Birth (DD/MM/YYYY). A note states: '(Your password must be larger than 10 characters and include capitals, numbers, and special characters)'. A 'Register!' button is present. A 'Register as Staff' link is in the bottom right corner.

1b

Before:

This screenshot is identical to the one above, showing the same registration page layout with the heading 'Welcome to GPQueue!', login link, registration form fields, password requirements, and buttons/links.

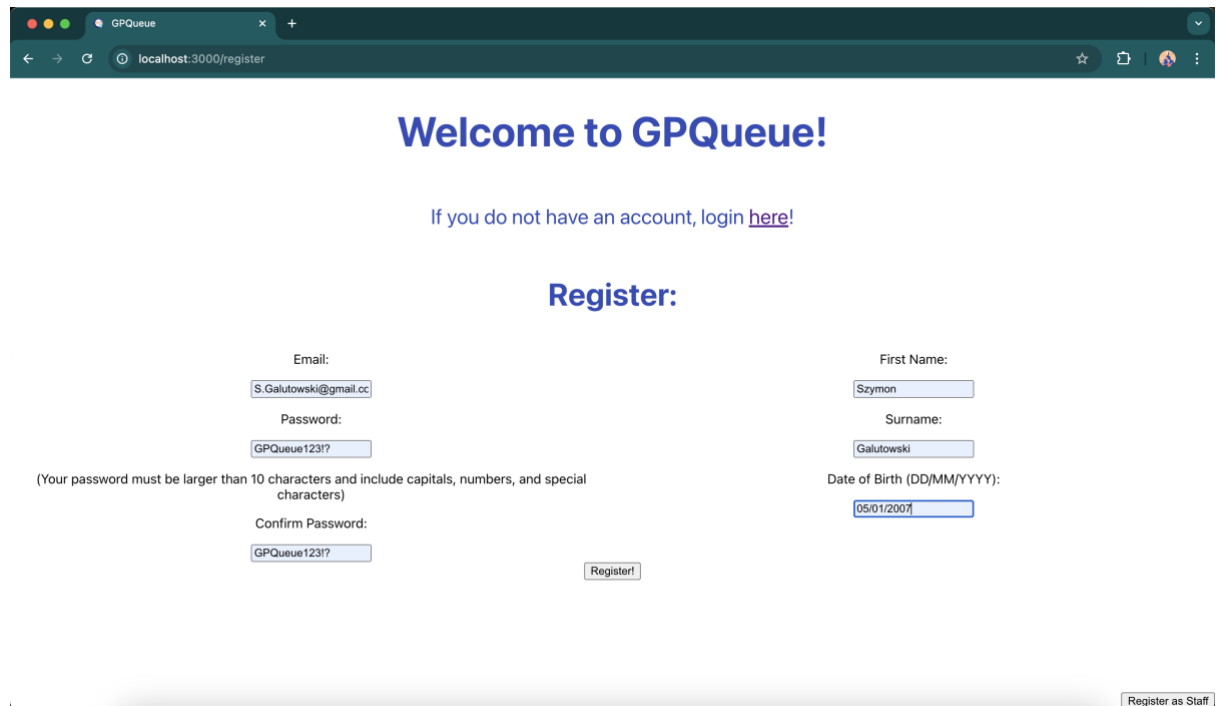
After:



A screenshot of a web browser showing the GPQueue login page. The browser's address bar displays 'localhost:3000/login'. The page has a dark header with the GPQueue logo. The main content area is white and features the heading 'Welcome to GPQueue!' in blue. Below this, a link 'here!' is provided for users without an account. The 'Login:' section contains two input fields for 'Email:' and 'Password:', followed by a 'Login!' button.

2a

Before:



A screenshot of a web browser showing the GPQueue register page. The browser's address bar displays 'localhost:3000/register'. The page has a dark header with the GPQueue logo. The main content area is white and features the heading 'Welcome to GPQueue!' in blue. Below this, a link 'here!' is provided for users without an account. The 'Register:' section contains several input fields: 'Email:' (with the value 'S.Galutowski@gmail.cc'), 'First Name:' (with the value 'Szymon'), 'Surname:' (with the value 'Galutowski'), 'Password:' (with the value 'GPQueue123!?' and a note '(Your password must be larger than 10 characters and include capitals, numbers, and special characters)'), 'Confirm Password:' (with the value 'GPQueue123!?'), and 'Date of Birth (DD/MM/YYYY):' (with the value '05/01/2007'). A 'Register!' button is located at the bottom right of the form. A 'Register as Staff' button is visible in the bottom right corner of the page.

After:

GPQueue

localhost:3000/home



Welcome, Szymon Galutowski!

22/01/2025 18:55:37

Home

Book an Appointment

You have 0 appointment(s) coming up in the future.

View Future Appointments

View Past Appointments

Database:

1 row returned							
user_id	f_name	l_name	email	user_type	password_hash		
integer	character varying	character varying	character varying	character varying	text		
1	Szymon	Galutowski	S.Galutowski@gmail.com	patient	52821115376340010321119493579477666998171967254563725900494731716944170483397	xx51D1BDDBC	

2b

Before:

GPQueue

localhost:3000/register

Welcome to GPQueue!

If you do not have an account, login [here!](#)

Register:

Email:

S.Galutowski@gmail.cc

Password:

ThisIsATest123!

(Your password must be larger than 10 characters and include capitals, numbers, and special characters)

Confirm Password:

ThisIsATest123!

First Name:

Stanley

Surname:

Galutowski

Date of Birth (DD/MM/YYYY):

27/07/2007

Register!

Register as Staff

After:

GPQueue

localhost:3000/register

Welcome to GPQueue!

If you do not have an account, login [here](#)!

Register:

Email:

S.Galutowski@gmail.cc

Password:

ThisIsATest123!

(Your password must be larger than 10 characters and include capitals, numbers, and special characters)

Confirm Password:

ThisIsATest123!

Register!

First Name:

Stanley

Surname:

Galutowski

Date of Birth (DD/MM/YYYY):

27/07/2007

Email already in use.

Register as Staff

Database:

1 row returned

user_id	f_name	l_name	email	user_type	password_hash		
integer	character varying	character varying	character varying	character varying	text		
1	1	Szymon	Galutowski	S.Galutowski@gmail.com	patient	52821115376340010321119493579477666998171967254563725900494731716944170483397	\x51D18DD8C

2c

Before:

GPQueue

localhost:3000/register

Welcome to GPQueue!

If you do not have an account, login [here](#)!

Register:

Email:

23020097@woodhouse

Password:

password

(Your password must be larger than 10 characters and include capitals, numbers, and special characters)

Confirm Password:

password

Register!

First Name:

Simon

Surname:

Gallagher

Date of Birth (DD/MM/YYYY):

06/01/2007

After:

GPQueue

localhost:3000/register

Welcome to GPQueue!

If you do not have an account, login [here!](#)

Register:

Email:

Password:

(Your password must be larger than 10 characters and include capitals, numbers, and special characters)
Confirm Password:

First Name:

Surname:

Date of Birth (DD/MM/YYYY):

Register!

Please make sure all fields are filled in correctly.

Register as Staff

Database:

1 row returned

user_id integer	f_name character varying	l_name character varying	email character varying	user_type character varying	password_hash text
1	Szymon	Galutowski	S.Galutowski@gmail.com	patient	52821115376340010321119493579477666998171967254563725900494731716944170483397

2d

Before:

GPQueue

localhost:3000/register

Welcome to GPQueue!

If you do not have an account, login [here!](#)

Register:

Email:

Password:

(Your password must be larger than 10 characters and include capitals, numbers, and special characters)
Confirm Password:

First Name:

Surname:

Date of Birth (DD/MM/YYYY):

Register!

Register as Staff

After:

The screenshot shows the GPQueue registration page at localhost:3000/register. The page has a dark header with the GPQueue logo. The main content area is white with a blue heading "Welcome to GPQueue!". Below this is a link "If you do not have an account, login [here!](#)". The "Register:" section contains several input fields: "Email:" (23020097@woodhouse), "Password:" (Password123!), "Confirm Password:" (Password123!), "First Name:" (Simon), "Surname:" (Gallagher), and "Date of Birth (DD/MM/YYYY):" (64/1/1945). A red error message "Please make sure all fields are filled in correctly." is displayed below the password fields. A "Register!" button is at the bottom right. A "Register as Staff" link is in the bottom right corner.

Database:

1 row returned

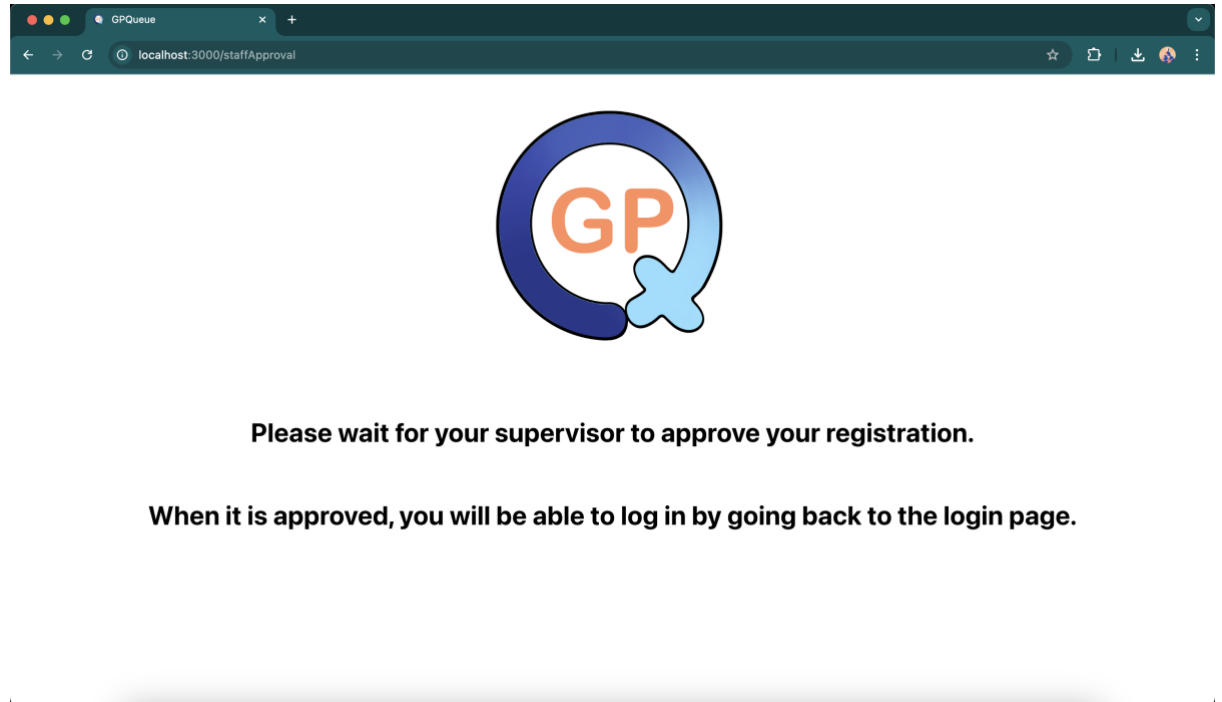
user_id	f_name	l_name	email	user_type	password_hash	
integer	character varying	character varying	character varying	character varying	text	
1	1	Szymon	Galutowski	S.Galutowski@gmail.com	patient	52821115376340010321119493579477666998171967254563725900494731716944170483397 \x51D1BDDBC

2e

Before:

The screenshot shows the GPQueue registration page at localhost:3000/register. The page has a dark header with the GPQueue logo. The main content area is white with a blue heading "Welcome to GPQueue!". Below this is a link "If you do not have an account, login [here!](#)". The "Register:" section contains several input fields: "Email:" (23020097@woodhouse), "Password:" (StrongPass123), "Confirm Password:" (StrongPass123), "First Name:" (Simon), "Surname:" (Gallagher), and "Date of Birth (DD/MM/YYYY):" (06/06/2007). A "Register!" button is at the bottom right. A "Register as Staff" link is in the bottom right corner.

After:

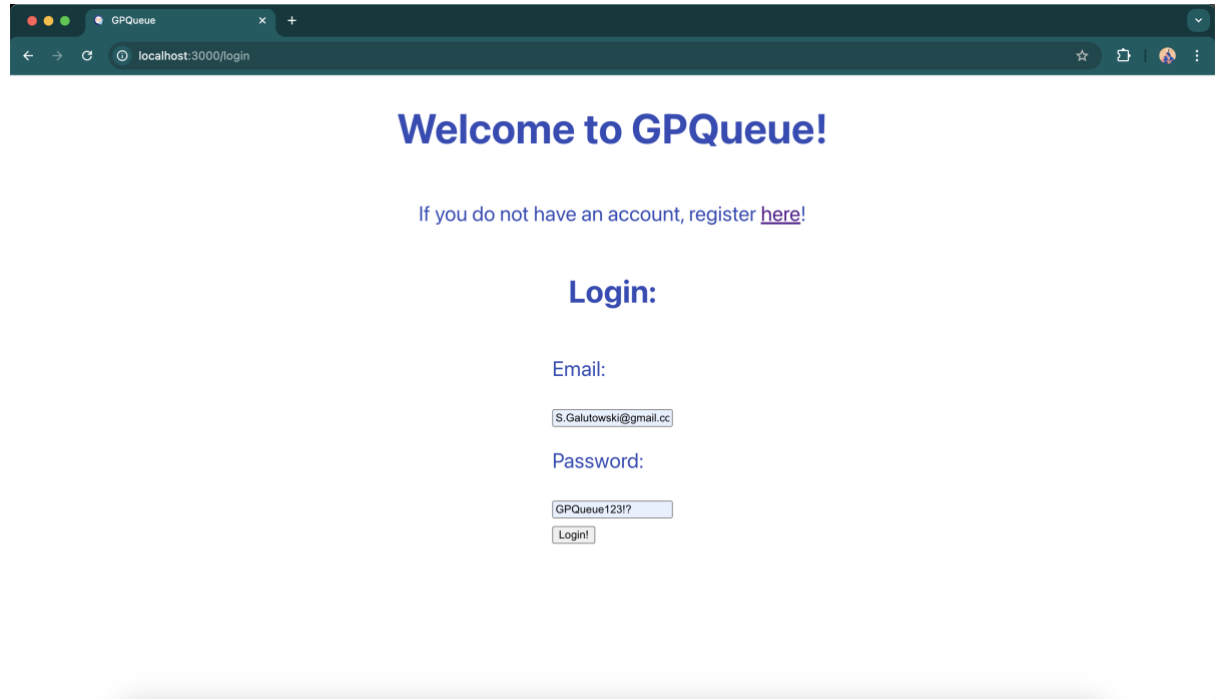


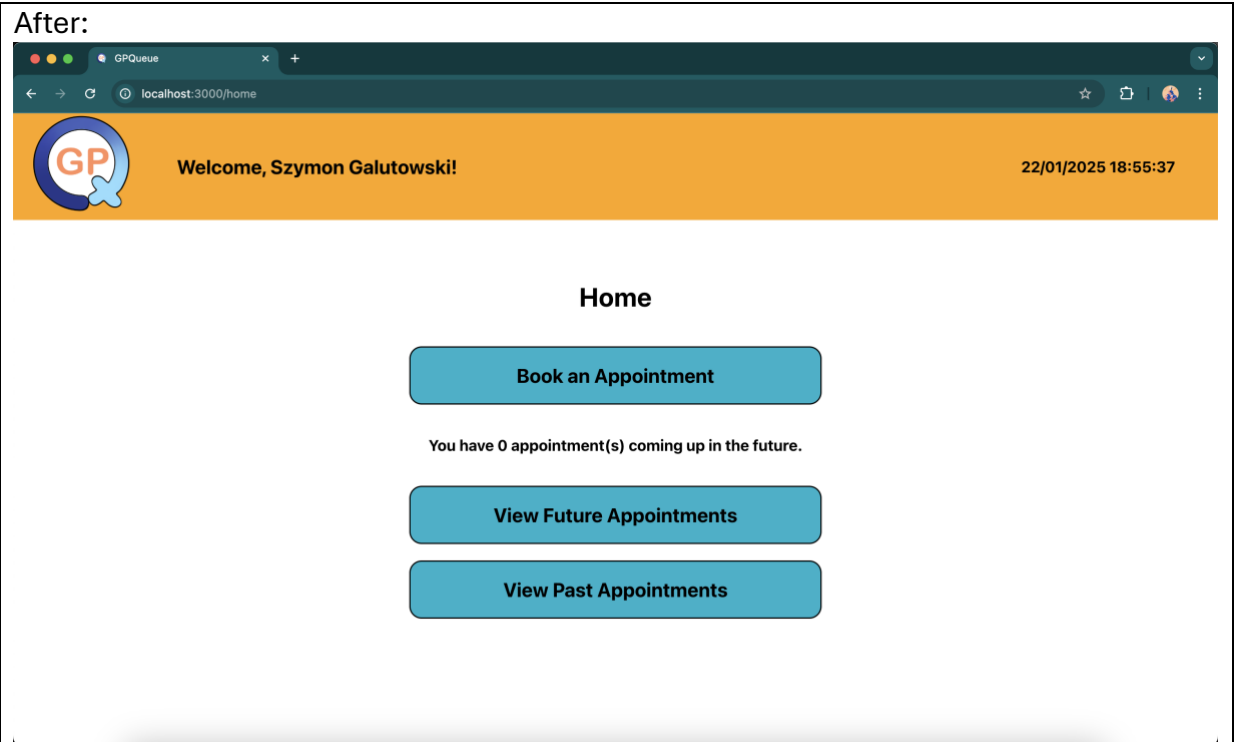
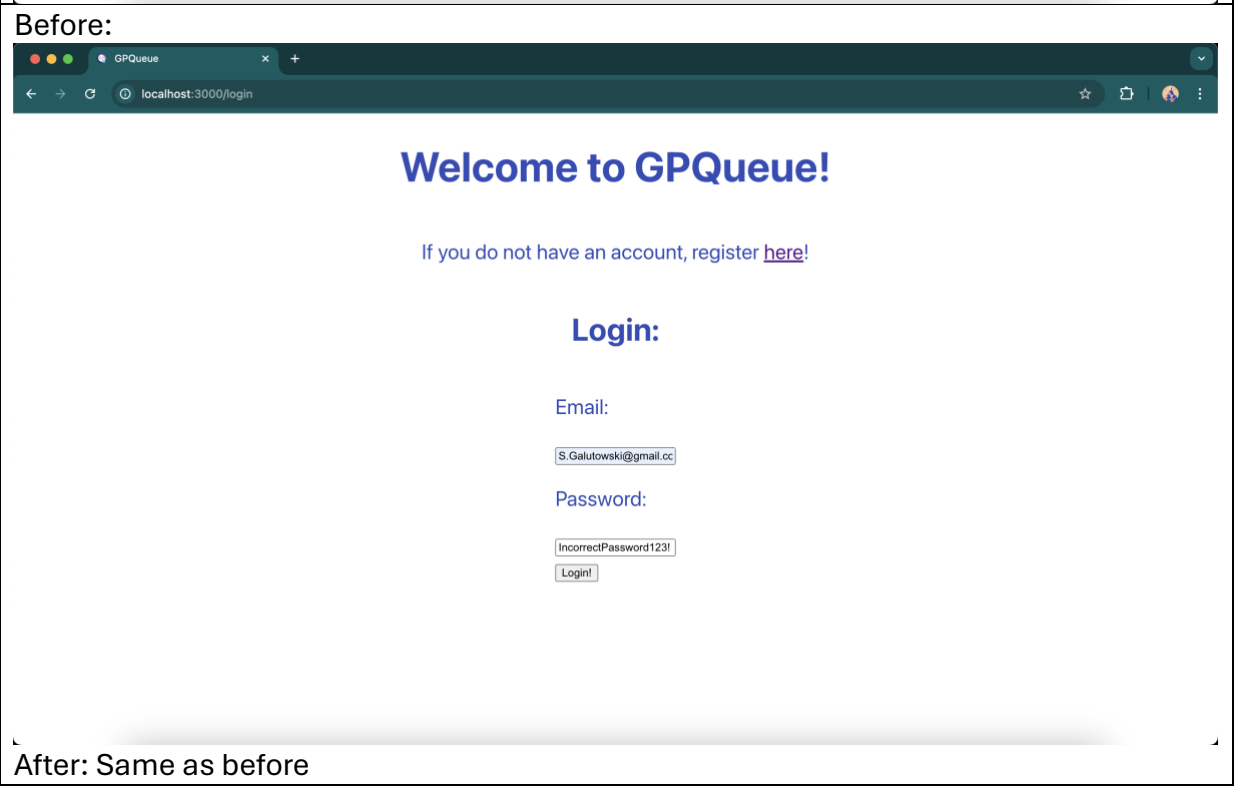
Database:

2 rows returned							
	user_id integer	f_name character varying	l_name character varying	email character varying	user_type character varying	password_hash text	
1	1	Szymon	Galutowski	S.Galutowski@gmail.com	patient	52821115376340010321119493579477666998171967254563725900494731716944170483397	\x51D
2	5	Simon	Gallagher	23020097@woodhouse.ac.uk	staff	69817376654452984917321470325205762282327725938838371917689339511870521824316	\x5AD

3a

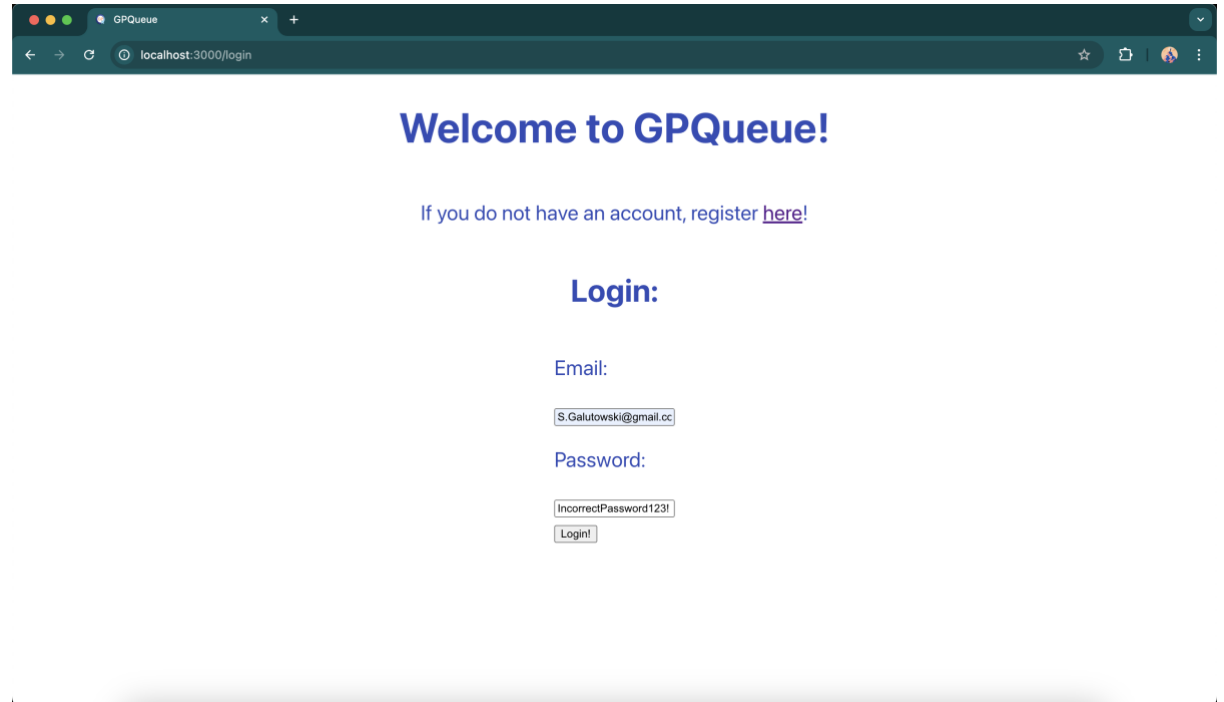
Before:



	After: 
3bi	Before:  After: Same as before

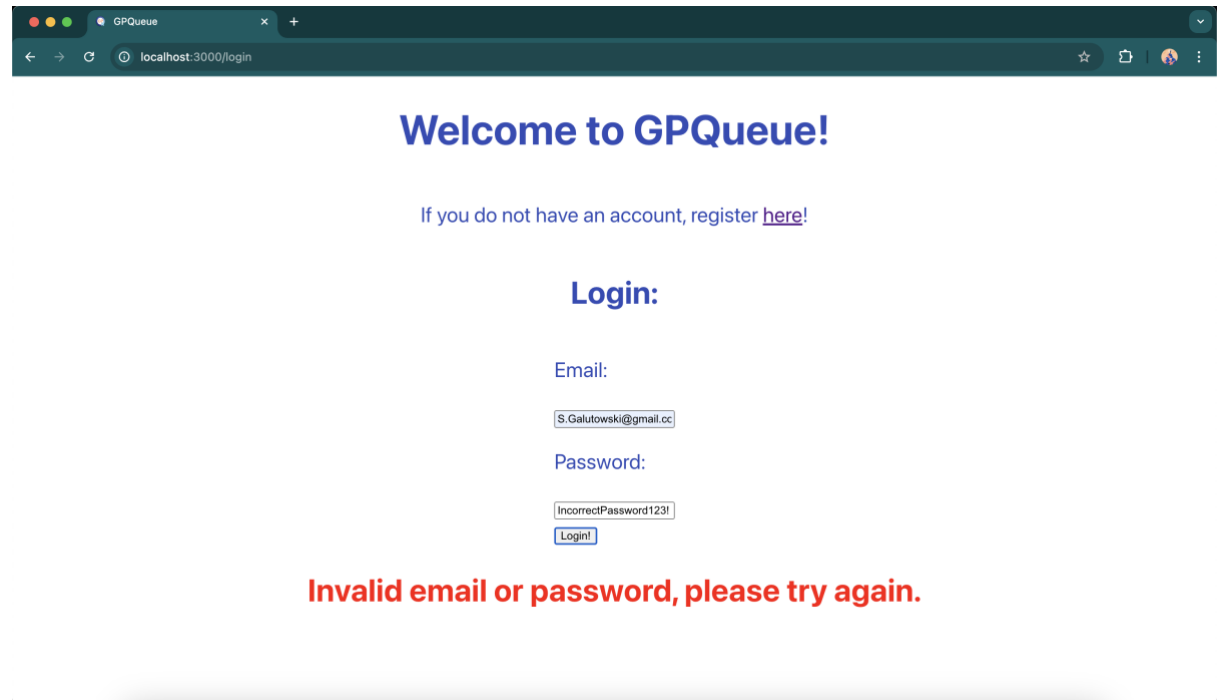
3bii

Before:



A screenshot of a web browser window showing the GPQueue login page. The browser's address bar displays 'localhost:3000/login'. The page content includes a heading 'Welcome to GPQueue!', a link to register, and a login form. The login form has fields for 'Email' (containing 'S.Galutowski@gmail.cc') and 'Password' (containing 'IncorrectPassword123!'), followed by a 'Login!' button.

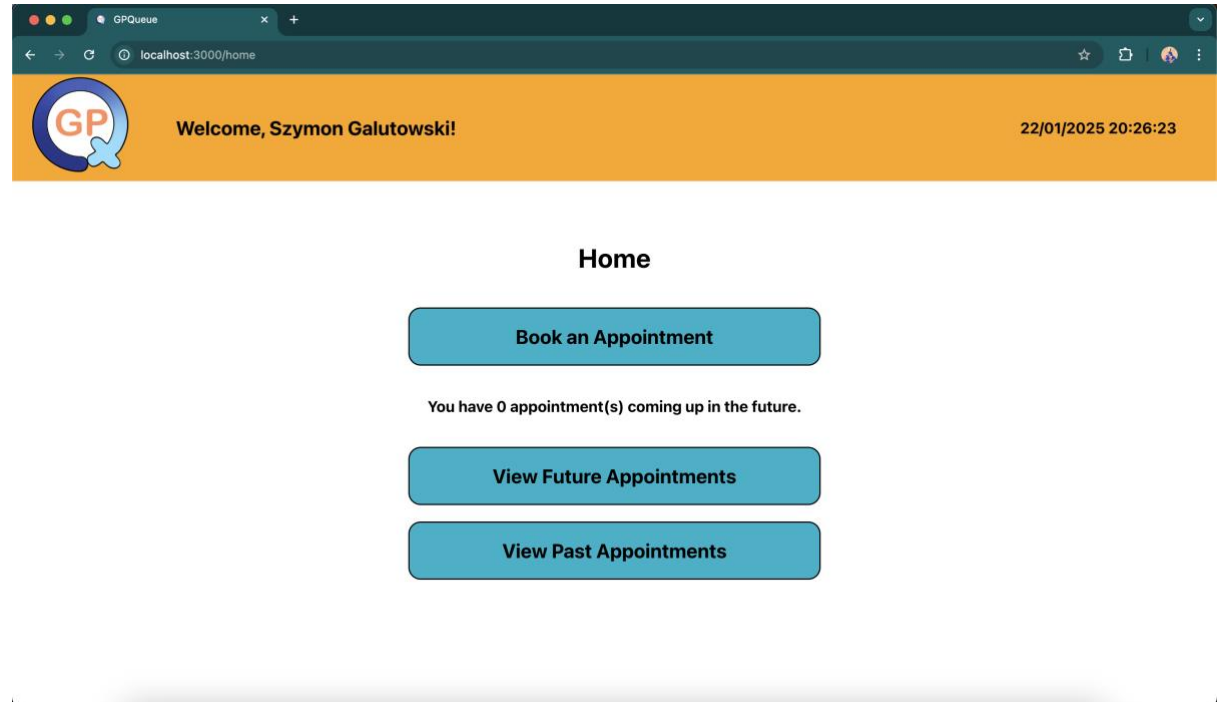
After:



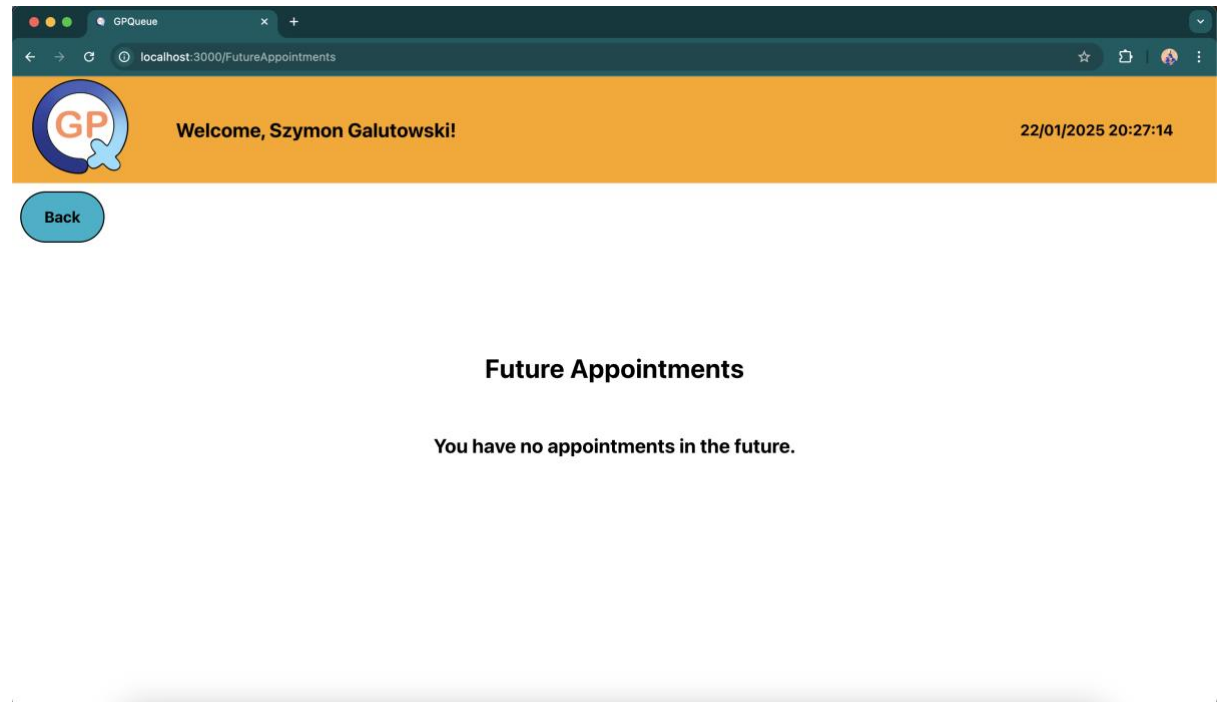
A screenshot of the same GPQueue login page, but with an error message displayed at the bottom: 'Invalid email or password, please try again.' in red text. The login form fields and the 'Login!' button remain visible above the error message.

4a

Before:

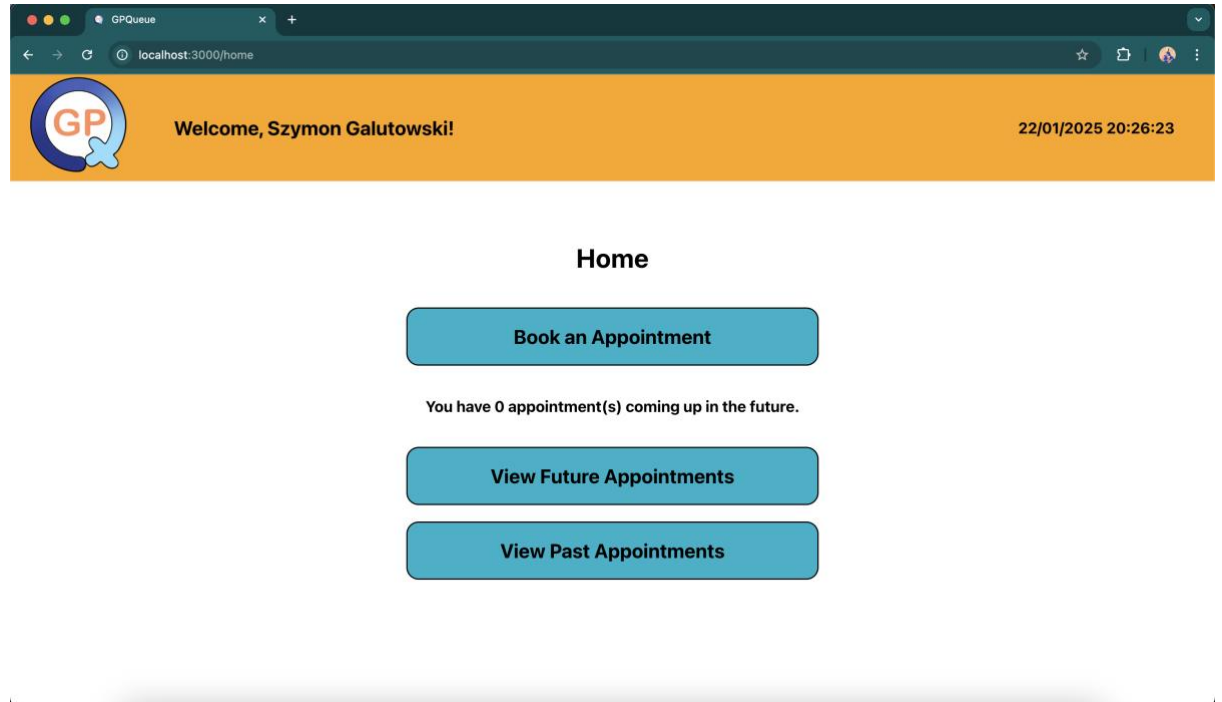


After:



4b

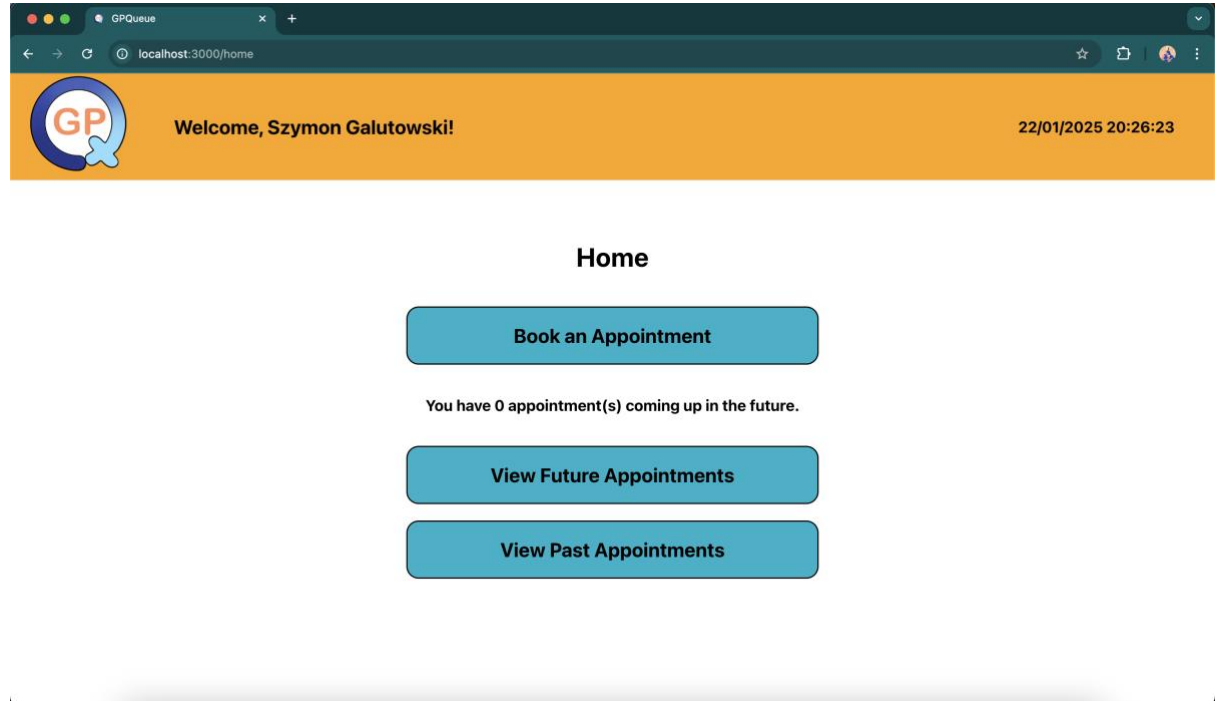
Before:



After:

4c

Before:



After:

Booking (Part 1)

Choose your main reason for making an appointment.

Reason:

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

Continue

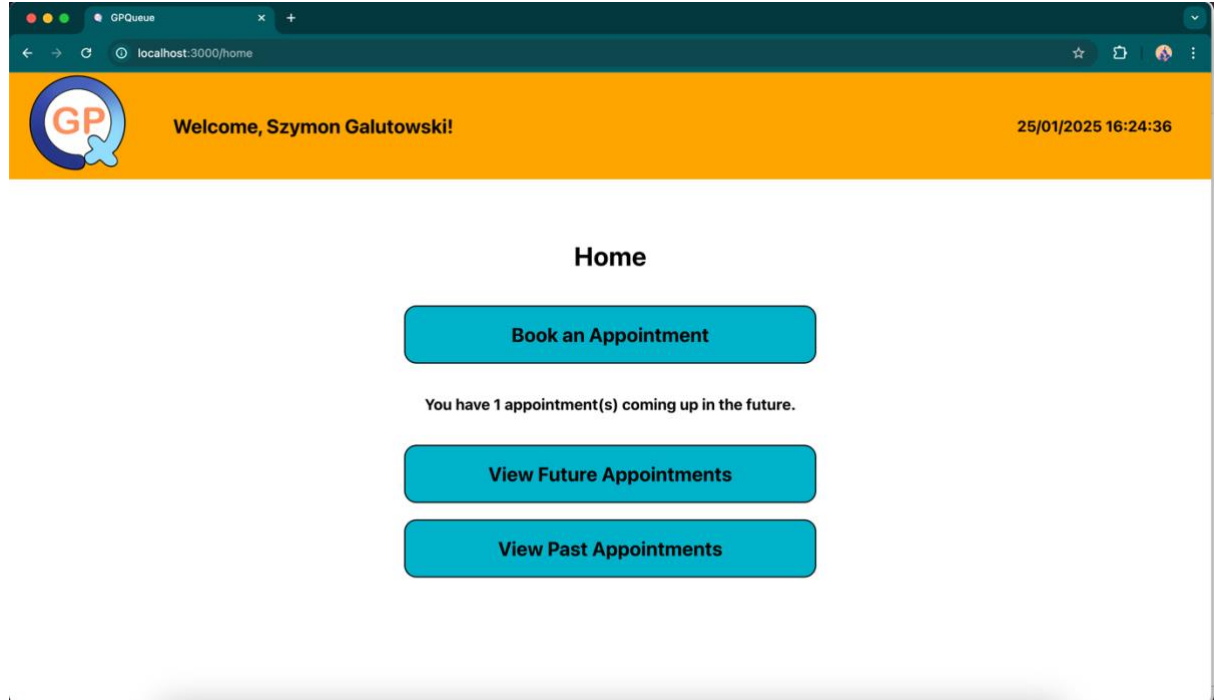
22/01/2025 20:29:35

4d

Database shows that Szymon Galutowski (id = 1) has 1 future appointment:

26 rows returned									
	appt_id integer	patient_id integer	staff_id integer	priority integer	appt_date date	appt_time character varying	appt_details text	creation_date timestamp with time zone	status text
1	185	7	6	3	2025-01-26	09:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
2	186	7	6	3	2025-01-26	09:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
3	187	7	6	3	2025-01-26	10:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
4	188	7	6	3	2025-01-26	10:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
5	189	7	6	3	2025-01-26	11:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
6	190	7	6	3	2025-01-26	11:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
7	191	7	6	3	2025-01-26	12:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
8	192	7	6	3	2025-01-26	12:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
9	193	7	6	3	2025-01-26	13:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
10	194	7	6	3	2025-01-26	13:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
11	195	7	6	3	2025-01-26	14:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
12	196	7	6	3	2025-01-26	14:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
13	197	7	6	3	2025-01-26	15:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
14	198	7	6	3	2025-01-26	15:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
15	199	7	6	3	2025-01-26	16:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
16	200	7	6	3	2025-01-26	16:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
17	201	7	6	3	2025-01-26	17:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
18	202	7	6	3	2025-01-26	17:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
19	203	7	6	3	2025-01-26	18:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
20	204	7	6	3	2025-01-26	18:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
21	205	7	6	3	2025-01-26	19:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
22	206	7	6	3	2025-01-26	19:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
23	207	7	6	3	2025-01-26	20:00:00	null	2025-01-24 13:45:28.614312+00	scheduled
24	208	7	6	3	2025-01-26	20:30:00	null	2025-01-24 13:45:28.614312+00	scheduled
25	163	1	6	3	2025-03-16	10:00	Routine vaccinations	2025-01-24 12:11:00.285326+00	scheduled
26	170	9	6	1	2025-02-11	09:30	Anxiety/Depression (worsening)	2025-01-24 12:59:40.977785+00	scheduled

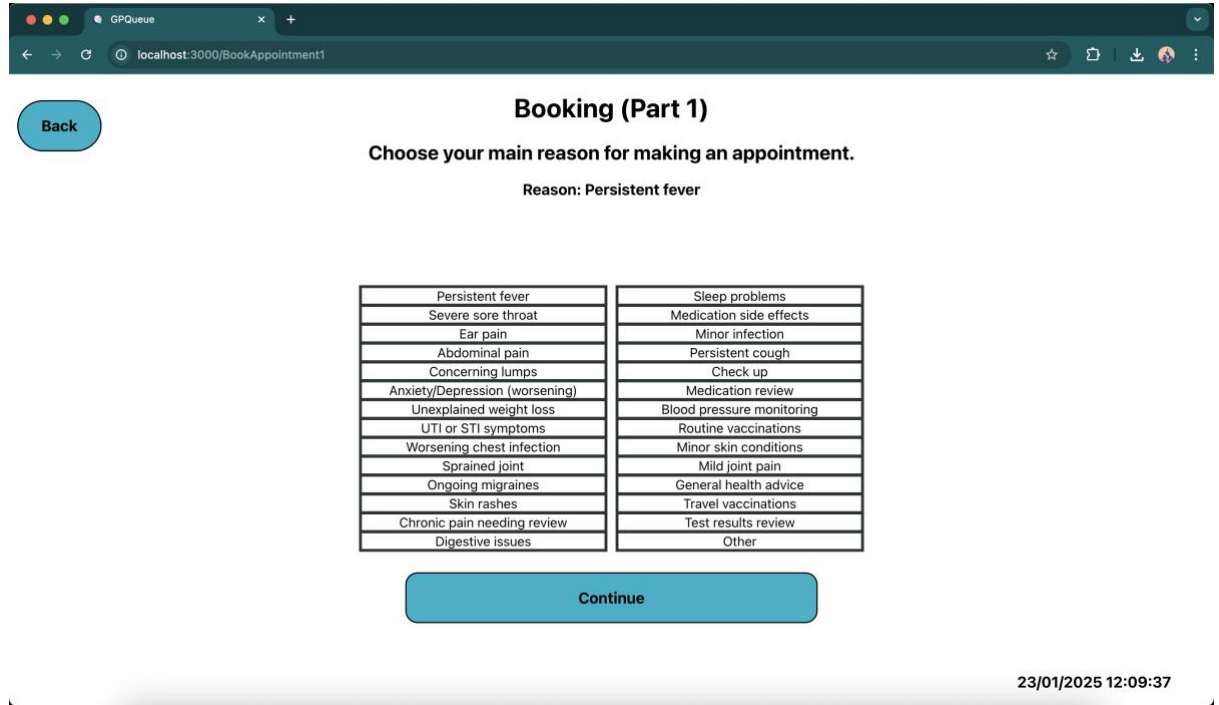
Homepage:



A screenshot of a web browser showing the GPQueue homepage. The browser's address bar displays 'localhost:3000/home'. The page has a yellow header with a 'GP' logo on the left, a welcome message 'Welcome, Szymon Galutowski!' in the center, and a timestamp '25/01/2025 16:24:36' on the right. The main content area is white and titled 'Home'. It features three blue buttons: 'Book an Appointment', 'View Future Appointments', and 'View Past Appointments'. Between the first and second buttons, a message states 'You have 1 appointment(s) coming up in the future.'

5a

Selecting a reason:



A screenshot of the 'Booking (Part 1)' page in the GPQueue application. The browser's address bar shows 'localhost:3000/BookAppointment1'. The page has a white background with a blue 'Back' button in the top left. The title 'Booking (Part 1)' is centered, followed by the instruction 'Choose your main reason for making an appointment.' Below this, the text 'Reason: Persistent fever' is displayed. A table with two columns lists various reasons for appointments. The first column includes: Persistent fever, Severe sore throat, Ear pain, Abdominal pain, Concerning lumps, Anxiety/Depression (worsening), Unexplained weight loss, UTI or STI symptoms, Worsening chest infection, Sprained joint, Ongoing migraines, Skin rashes, Chronic pain needing review, and Digestive issues. The second column includes: Sleep problems, Medication side effects, Minor infection, Persistent cough, Check up, Medication review, Blood pressure monitoring, Routine vaccinations, Minor skin conditions, Mild joint pain, General health advice, Travel vaccinations, Test results review, and Other. A blue 'Continue' button is positioned below the table. The timestamp '23/01/2025 12:09:37' is located in the bottom right corner.

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

After:

Back

Booking (part 2)

Select your preferred days

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected:

Previous Month

Next Month

Continue

23/01/2025 12:09:55

5b

First reason:

Back

Booking (Part 1)

Choose your main reason for making an appointment.

Reason: Ear pain

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

Continue

23/01/2025 12:55:33

Second reason:

GPQueue

localhost:3000/BookAppointment1

Back

Booking (Part 1)

Choose your main reason for making an appointment.

Reason: Abdominal pain

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

Continue

23/01/2025 12:55:50

After:

GPQueue

localhost:3000/BookAppointment2

Back

Booking (part 2)

Select your preferred days

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected:

Previous Month

Next Month

Continue

23/01/2025 12:57:38

5c

Before:

GPQueue

localhost:3000/BookAppointment1

Booking (Part 1)

Choose your main reason for making an appointment.

Reason:

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

Continue

23/01/2025 13:05:15

After:

GPQueue

localhost:3000/BookAppointment1

Booking (Part 1)

Choose your main reason for making an appointment.

Reason:

Please choose a reason before proceeding!

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

Continue

23/01/2025 13:05:27

5di

Before:

Booking (Part 1)

Choose your main reason for making an appointment.

Reason: Other

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

Continue

23/01/2025 13:07:35

After:

Loading...

5dii

Before:

GPQueue

localhost:3000/BookAppointment1

Booking (Part 1)

Choose your main reason for making an appointment.

Reason: Other

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

Continue

23/01/2025 13:15:36

After:

GPQueue

localhost:3000/home

GP

Welcome, Szymon Galutowski!

23/01/2025 13:15:54

Home

Book an Appointment

You have 0 appointment(s) coming up in the future.

View Future Appointments

View Past Appointments

6a

Dates chosen:

GPQueue

localhost:3000/BookAppointment2

Back

Booking (part 2)

Select your preferred days

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected: 24/01/2025, 27/01/2025, 11/02/2025, 15/02/2025, 27/03/2025, 23/03/2025,

Previous Month

Next Month

Continue

23/01/2025 13:20:29

After:

GPQueue

localhost:3000/BookAppointment3

Back

Booking (part 3)

Select your preferred timeframes

9:00 - 13:00

13:00 - 17:00

17:00 - 21:00

Selection:

Continue

23/01/2025 13:20:48

6b

Before:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

Back

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected: 24/01/2025,

Previous Month

Next Month

Continue

23/01/2025 13:21:41

After:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

Back

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected:

Previous Month

Next Month

Continue

23/01/2025 13:21:58

6c

Before:

GPQueue

localhost:3000/BookAppointment2

Back

Booking (part 2)

Select your preferred days

01/02/2025	02/02/2025	03/02/2025	04/02/2025
05/02/2025	06/02/2025	07/02/2025	08/02/2025
09/02/2025	10/02/2025	11/02/2025	12/02/2025
13/02/2025	14/02/2025	15/02/2025	16/02/2025
17/02/2025	18/02/2025	19/02/2025	20/02/2025
21/02/2025	22/02/2025	23/02/2025	24/02/2025
25/02/2025	26/02/2025	27/02/2025	28/02/2025

Current month: February

Days selected: 15/02/2025, 16/02/2025,

Previous Month

Next Month

Continue

23/01/2025 13:24:22

After:

GPQueue

localhost:3000/BookAppointment2

Back

Booking (part 2)

Select your preferred days

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected: 15/02/2025, 16/02/2025,

Previous Month

Next Month

Continue

23/01/2025 13:24:34

6d

Before:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

Back

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected:

Previous Month

Next Month

Continue

23/01/2025 13:25:01

After:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

Back

01/03/2025	02/03/2025	03/03/2025	04/03/2025
05/03/2025	06/03/2025	07/03/2025	08/03/2025
09/03/2025	10/03/2025	11/03/2025	12/03/2025
13/03/2025	14/03/2025	15/03/2025	16/03/2025
17/03/2025	18/03/2025	19/03/2025	20/03/2025
21/03/2025	22/03/2025	23/03/2025	24/03/2025
25/03/2025	26/03/2025	27/03/2025	28/03/2025
29/03/2025	30/03/2025	31/03/2025	

Current month: March

Days selected:

Previous Month

Next Month

Continue

23/01/2025 13:25:16

6e

Before:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

Back

01/03/2025	02/03/2025	03/03/2025	04/03/2025
05/03/2025	06/03/2025	07/03/2025	08/03/2025
09/03/2025	10/03/2025	11/03/2025	12/03/2025
13/03/2025	14/03/2025	15/03/2025	16/03/2025
17/03/2025	18/03/2025	19/03/2025	20/03/2025
21/03/2025	22/03/2025	23/03/2025	24/03/2025
25/03/2025	26/03/2025	27/03/2025	28/03/2025
29/03/2025	30/03/2025	31/03/2025	

Current month: March

Days selected:

Previous Month

Next Month

Continue

23/01/2025 13:26:43

After:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

Back

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected:

Previous Month

Next Month

Continue

23/01/2025 13:26:51

6f

Before:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

Back

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected:

Previous Month

Next Month

Continue

23/01/2025 13:28:20

After:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

Back

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected:

Please select at least one day before continuing!

Previous Month

Next Month

Continue

23/01/2025 13:28:30

6g

Before:

GPQueue

localhost:3000/BookAppointment1

Booking (Part 1)

Choose your main reason for making an appointment.

Reason: Test results review

Persistent fever	Sleep problems
Severe sore throat	Medication side effects
Ear pain	Minor infection
Abdominal pain	Persistent cough
Concerning lumps	Check up
Anxiety/Depression (worsening)	Medication review
Unexplained weight loss	Blood pressure monitoring
UTI or STI symptoms	Routine vaccinations
Worsening chest infection	Minor skin conditions
Sprained joint	Mild joint pain
Ongoing migraines	General health advice
Skin rashes	Travel vaccinations
Chronic pain needing review	Test results review
Digestive issues	Other

Continue

24/01/2025 13:45:46

After:

GPQueue

localhost:3000/BookAppointment2

Booking (part 2)

Select your preferred days

01/01/2025	02/01/2025	03/01/2025	04/01/2025
05/01/2025	06/01/2025	07/01/2025	08/01/2025
09/01/2025	10/01/2025	11/01/2025	12/01/2025
13/01/2025	14/01/2025	15/01/2025	16/01/2025
17/01/2025	18/01/2025	19/01/2025	20/01/2025
21/01/2025	22/01/2025	23/01/2025	24/01/2025
25/01/2025	26/01/2025	27/01/2025	28/01/2025
29/01/2025	30/01/2025	31/01/2025	

Current month: January

Days selected:

Previous Month

Next Month

Continue

24/01/2025 13:46:03

7a

Before:

GPQueue

localhost:3000/BookAppointment3

Booking (part 3)

Select your preferred timeframes

Back

9:00 - 13:00

13:00 - 17:00

17:00 - 21:00

Selection:

Continue

25/01/2025 16:30:32

After:

GPQueue

localhost:3000/BookAppointment3

Booking (part 3)

Select your preferred timeframes

Back

9:00 - 13:00

13:00 - 17:00

17:00 - 21:00

Selection:

Please select at least one timeframe!

Continue

25/01/2025 16:30:43

7b

Clicking on a timeframe:

GPQueue

localhost:3000/BookAppointment3

Booking (part 3)

Select your preferred timeframes

Back

9:00 - 13:00

13:00 - 17:00

17:00 - 21:00

Selection: 13:00-17:00,

Continue

25/01/2025 16:31:36

Clicking on the same timeframe again:

GPQueue

localhost:3000/BookAppointment3

Booking (part 3)

Select your preferred timeframes

Back

9:00 - 13:00

13:00 - 17:00

17:00 - 21:00

Selection:

Continue

25/01/2025 16:31:51

7c

Before:

GPQueue

localhost:3000/BookAppointment3

Back

Booking (part 3)

Select your preferred timeframes

9:00 - 13:00

13:00 - 17:00

17:00 - 21:00

Selection: 17:00-21:00,

Continue

25/01/2025 16:32:37

After:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 21/02/2025

Time: 17:00

Doctor: Simon Gallagher

You have 4:59 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

25/01/2025 16:32:57

8a

Page:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 23/02/2025

Time: 09:00

Doctor: Simon Gallagher

You have 3:28 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

25/01/2025 16:36:04

(I scrolled through all of the appointments, and none of them had the replacement message.)

Database:

JS	ApiCalls.js	JS	5 - appointments.js	M	populate.pgsq	M	Backend.py	Results: unnamed-query-1	Results: unnamed-query-2	X
	appt_id	patient_id	staff_id	priority	appt_date	appt_time	appt_details	creation_date	status	
	integer	integer	integer	integer	date	character varying	text	timestamp with time zone	text	
46	236	1	6	3	2025-02-23	18:30	null	2025-01-25 16:34:14.616986+00	reserved	
47	237	1	6	3	2025-02-23	19:00	null	2025-01-25 16:34:14.616986+00	reserved	
48	238	1	6	3	2025-02-23	19:30	null	2025-01-25 16:34:14.616986+00	reserved	
49	239	1	6	3	2025-02-23	20:00	null	2025-01-25 16:34:14.616986+00	reserved	
50	240	1	6	3	2025-02-23	20:30	null	2025-01-25 16:34:14.616986+00	reserved	
51	241	1	6	3	2025-02-24	09:00	null	2025-01-25 16:34:14.616986+00	reserved	
52	242	1	6	3	2025-02-24	09:30	null	2025-01-25 16:34:14.616986+00	reserved	
53	243	1	6	3	2025-02-24	10:00	null	2025-01-25 16:34:14.616986+00	reserved	
54	244	1	6	3	2025-02-24	10:30	null	2025-01-25 16:34:14.616986+00	reserved	
55	245	1	6	3	2025-02-24	11:00	null	2025-01-25 16:34:14.616986+00	reserved	
56	246	1	6	3	2025-02-24	11:30	null	2025-01-25 16:34:14.616986+00	reserved	
57	247	1	6	3	2025-02-24	12:00	null	2025-01-25 16:34:14.616986+00	reserved	
58	248	1	6	3	2025-02-24	12:30	null	2025-01-25 16:34:14.616986+00	reserved	
59	249	1	6	3	2025-02-24	13:00	null	2025-01-25 16:34:14.616986+00	reserved	
60	250	1	6	3	2025-02-24	13:30	null	2025-01-25 16:34:14.616986+00	reserved	
61	251	1	6	3	2025-02-24	14:00	null	2025-01-25 16:34:14.616986+00	reserved	
62	252	1	6	3	2025-02-24	14:30	null	2025-01-25 16:34:14.616986+00	reserved	
63	253	1	6	3	2025-02-24	15:00	null	2025-01-25 16:34:14.616986+00	reserved	
64	254	1	6	3	2025-02-24	15:30	null	2025-01-25 16:34:14.616986+00	reserved	
65	255	1	6	3	2025-02-24	16:00	null	2025-01-25 16:34:14.616986+00	reserved	
66	256	1	6	3	2025-02-24	16:30	null	2025-01-25 16:34:14.616986+00	reserved	
67	257	1	6	3	2025-02-24	17:00	null	2025-01-25 16:34:14.616986+00	reserved	
68	258	1	6	3	2025-02-24	17:30	null	2025-01-25 16:34:14.616986+00	reserved	
69	259	1	6	3	2025-02-24	18:00	null	2025-01-25 16:34:14.616986+00	reserved	
70	260	1	6	3	2025-02-24	18:30	null	2025-01-25 16:34:14.616986+00	reserved	
71	261	1	6	3	2025-02-24	19:00	null	2025-01-25 16:34:14.616986+00	reserved	
72	262	1	6	3	2025-02-24	19:30	null	2025-01-25 16:34:14.616986+00	reserved	
73	263	1	6	3	2025-02-24	20:00	null	2025-01-25 16:34:14.616986+00	reserved	
74	264	1	6	3	2025-02-24	20:30	null	2025-01-25 16:34:14.616986+00	reserved	

8bi

Upon entering the page:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 26/01/2025

Time: 12:30

Doctor: Simon Gallagher

You have 4:50 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

25/01/2025 20:05:44

After pressing forwards/backwards:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 26/01/2025

Time: 12:30

Doctor: Simon Gallagher

If you choose this appointment, you will be replacing somebody elses. Their appointment will be rescheduled.

You have 4:25 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

25/01/2025 20:06:10

8bii

Upon entering the page:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 26/01/2025

Time: 09:00

Doctor: Simon Gallagher

If you choose this appointment, you will be replacing somebody elses. Their appointment will be rescheduled.

You have 4:56 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

25/01/2025 20:07:00

After pressing forwards/backwards:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 26/01/2025

Time: 12:00

Doctor: Simon Gallagher

If you choose this appointment, you will be replacing somebody elses. Their appointment will be rescheduled.

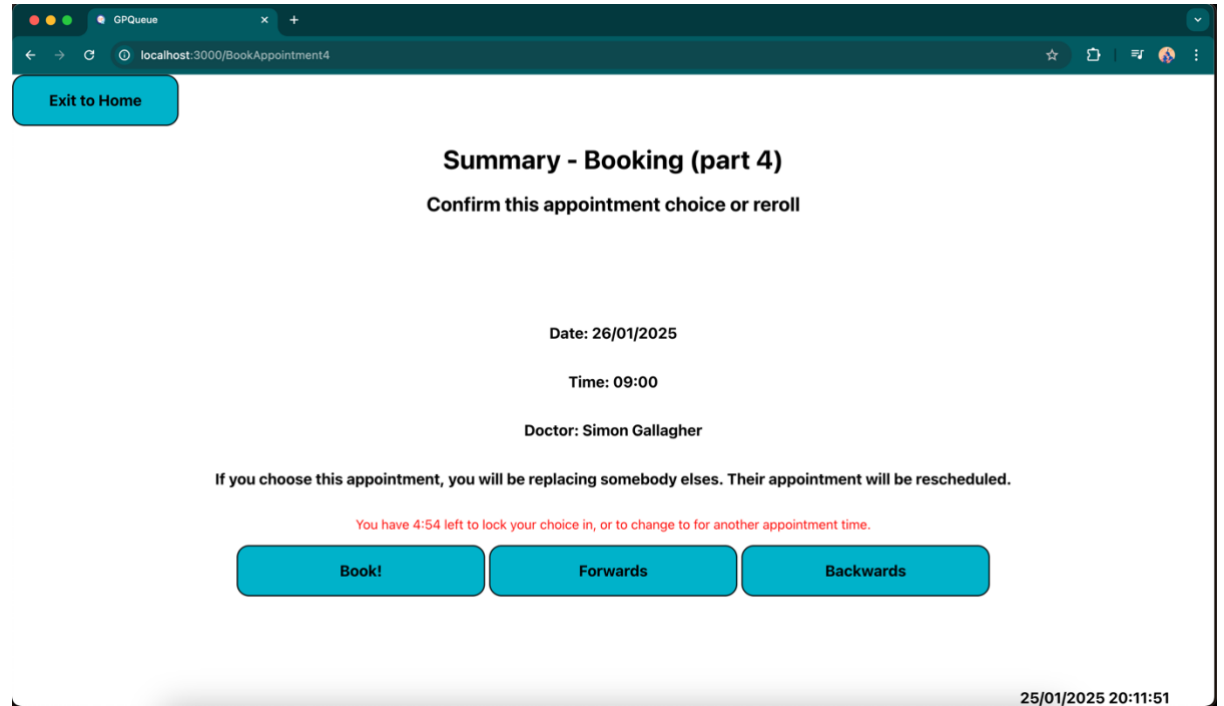
You have 4:25 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

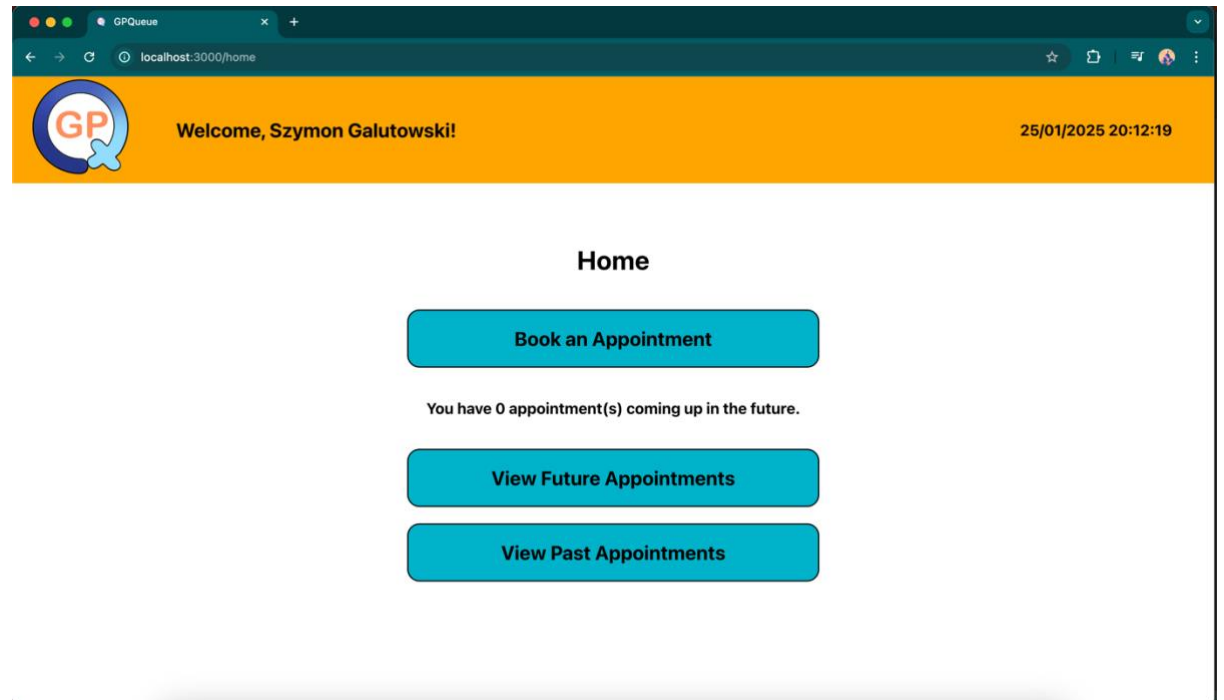
25/01/2025 20:07:34

8ci

Before:



After:



Database (failed):

24 rows returned

	appt_id integer	patient_id integer	staff_id integer	priority integer	appt_date date	appt_time character varying	appt_details text	creation_date timestamp with time zone	status text
1	273	7	6	3	2025-01-26	09:00	null	2025-01-25 16:47:22.437353+00	scheduled + reserved by 1
2	289	7	6	3	2025-01-26	17:00	null	2025-01-25 16:47:22.437353+00	scheduled
3	290	7	6	3	2025-01-26	17:30	null	2025-01-25 16:47:22.437353+00	scheduled
4	291	7	6	3	2025-01-26	18:00	null	2025-01-25 16:47:22.437353+00	scheduled
5	274	7	6	3	2025-01-26	09:30	null	2025-01-25 16:47:22.437353+00	scheduled + reserved by 1
6	292	7	6	3	2025-01-26	18:30	null	2025-01-25 16:47:22.437353+00	scheduled
7	275	7	6	3	2025-01-26	10:00	null	2025-01-25 16:47:22.437353+00	scheduled + reserved by 1
8	293	7	6	3	2025-01-26	19:00	null	2025-01-25 16:47:22.437353+00	scheduled
9	294	7	6	3	2025-01-26	19:30	null	2025-01-25 16:47:22.437353+00	scheduled
10	295	7	6	3	2025-01-26	20:00	null	2025-01-25 16:47:22.437353+00	scheduled
11	296	7	6	3	2025-01-26	20:30	null	2025-01-25 16:47:22.437353+00	scheduled
12	276	7	6	3	2025-01-26	10:30	null	2025-01-25 16:47:22.437353+00	scheduled + reserved by 1
13	277	7	6	3	2025-01-26	11:00	null	2025-01-25 16:47:22.437353+00	scheduled + reserved by 1
14	278	7	6	3	2025-01-26	11:30	null	2025-01-25 16:47:22.437353+00	scheduled + reserved by 1
15	279	7	6	3	2025-01-26	12:00	null	2025-01-25 16:47:22.437353+00	scheduled + reserved by 1
16	280	7	6	3	2025-01-26	12:30	null	2025-01-25 16:47:22.437353+00	scheduled + reserved by 1
17	281	7	6	3	2025-01-26	13:00	null	2025-01-25 16:47:22.437353+00	scheduled
18	282	7	6	3	2025-01-26	13:30	null	2025-01-25 16:47:22.437353+00	scheduled
19	283	7	6	3	2025-01-26	14:00	null	2025-01-25 16:47:22.437353+00	scheduled
20	284	7	6	3	2025-01-26	14:30	null	2025-01-25 16:47:22.437353+00	scheduled
21	285	7	6	3	2025-01-26	15:00	null	2025-01-25 16:47:22.437353+00	scheduled
22	286	7	6	3	2025-01-26	15:30	null	2025-01-25 16:47:22.437353+00	scheduled
23	287	7	6	3	2025-01-26	16:00	null	2025-01-25 16:47:22.437353+00	scheduled
24	288	7	6	3	2025-01-26	16:30	null	2025-01-25 16:47:22.437353+00	scheduled

8cii

Before:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 26/01/2025

Time: 15:00

Doctor: Simon Gallagher

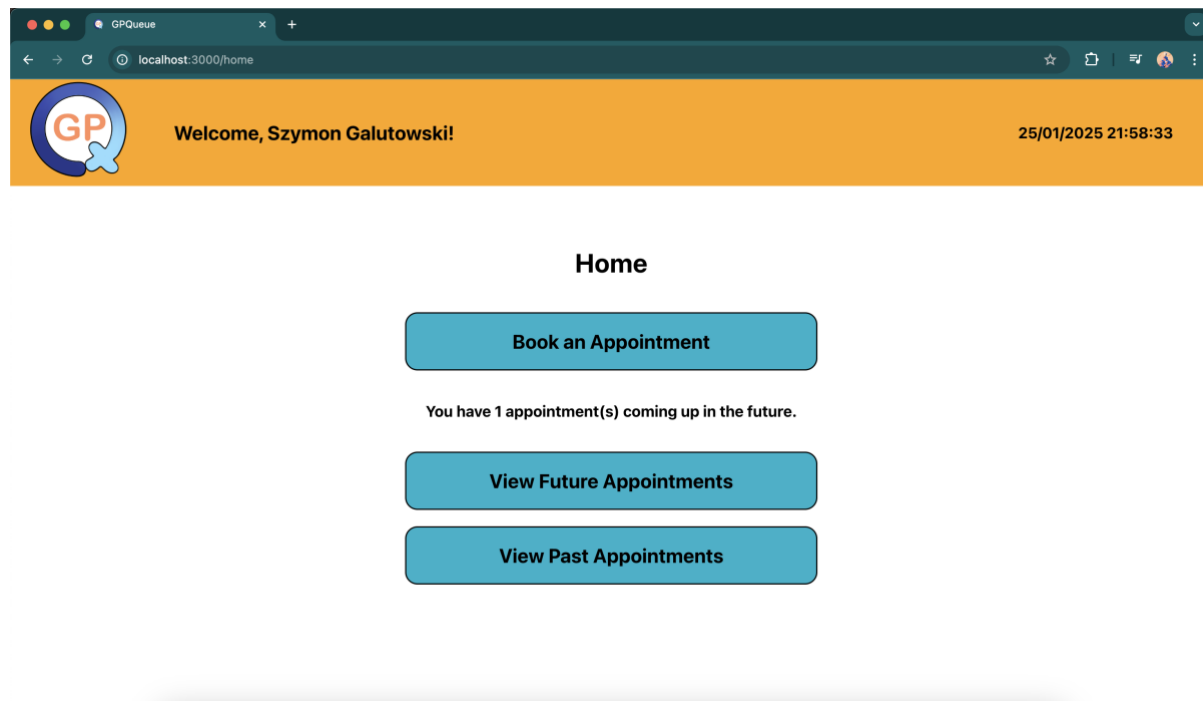
If you choose this appointment, you will be replacing somebody elses. Their appointment will be rescheduled.

You have 4:39 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

25/01/2025 21:58:12

After:



Database (correct):

25 rows returned									
	appt_id integer	patient_id integer	staff_id integer	priority integer	appt_date date	appt_time character varying	appt_details text	creation_date timestamp with time zone	status text
1	401	1	6	3	2025-01-26	15:00	Persistent fever	2025-01-25 21:56:29.957168+00	scheduled
2	413	7	6	2	2025-01-27	09:00	null	2025-01-25 21:58:30.134422+00	scheduled
3	397	7	6	3	2025-01-26	13:00	null	2025-01-25 21:56:29.957168+00	scheduled
4	398	7	6	3	2025-01-26	13:30	null	2025-01-25 21:56:29.957168+00	scheduled
5	399	7	6	3	2025-01-26	14:00	null	2025-01-25 21:56:29.957168+00	scheduled
6	400	7	6	3	2025-01-26	14:30	null	2025-01-25 21:56:29.957168+00	scheduled
7	402	7	6	3	2025-01-26	15:30	null	2025-01-25 21:56:29.957168+00	scheduled
8	403	7	6	3	2025-01-26	16:00	null	2025-01-25 21:56:29.957168+00	scheduled
9	404	7	6	3	2025-01-26	16:30	null	2025-01-25 21:56:29.957168+00	scheduled
10	389	7	6	3	2025-01-26	09:00	null	2025-01-25 21:56:29.957168+00	scheduled
11	390	7	6	3	2025-01-26	09:30	null	2025-01-25 21:56:29.957168+00	scheduled
12	391	7	6	3	2025-01-26	10:00	null	2025-01-25 21:56:29.957168+00	scheduled
13	392	7	6	3	2025-01-26	10:30	null	2025-01-25 21:56:29.957168+00	scheduled
14	393	7	6	3	2025-01-26	11:00	null	2025-01-25 21:56:29.957168+00	scheduled
15	394	7	6	3	2025-01-26	11:30	null	2025-01-25 21:56:29.957168+00	scheduled
16	395	7	6	3	2025-01-26	12:00	null	2025-01-25 21:56:29.957168+00	scheduled
17	396	7	6	3	2025-01-26	12:30	null	2025-01-25 21:56:29.957168+00	scheduled
18	405	7	6	3	2025-01-26	17:00	null	2025-01-25 21:56:29.957168+00	scheduled
19	406	7	6	3	2025-01-26	17:30	null	2025-01-25 21:56:29.957168+00	scheduled
20	407	7	6	3	2025-01-26	18:00	null	2025-01-25 21:56:29.957168+00	scheduled
21	408	7	6	3	2025-01-26	18:30	null	2025-01-25 21:56:29.957168+00	scheduled
22	409	7	6	3	2025-01-26	19:00	null	2025-01-25 21:56:29.957168+00	scheduled
23	410	7	6	3	2025-01-26	19:30	null	2025-01-25 21:56:29.957168+00	scheduled
24	411	7	6	3	2025-01-26	20:00	null	2025-01-25 21:56:29.957168+00	scheduled
25	412	7	6	3	2025-01-26	20:30	null	2025-01-25 21:56:29.957168+00	scheduled

8d

Pressing forward:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 24/03/2025

Time: 14:00

Doctor: Simon Gallagher

You have 4:49 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

25/01/2025 22:13:16

Pressing backwards:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 24/03/2025

Time: 13:30

Doctor: Simon Gallagher

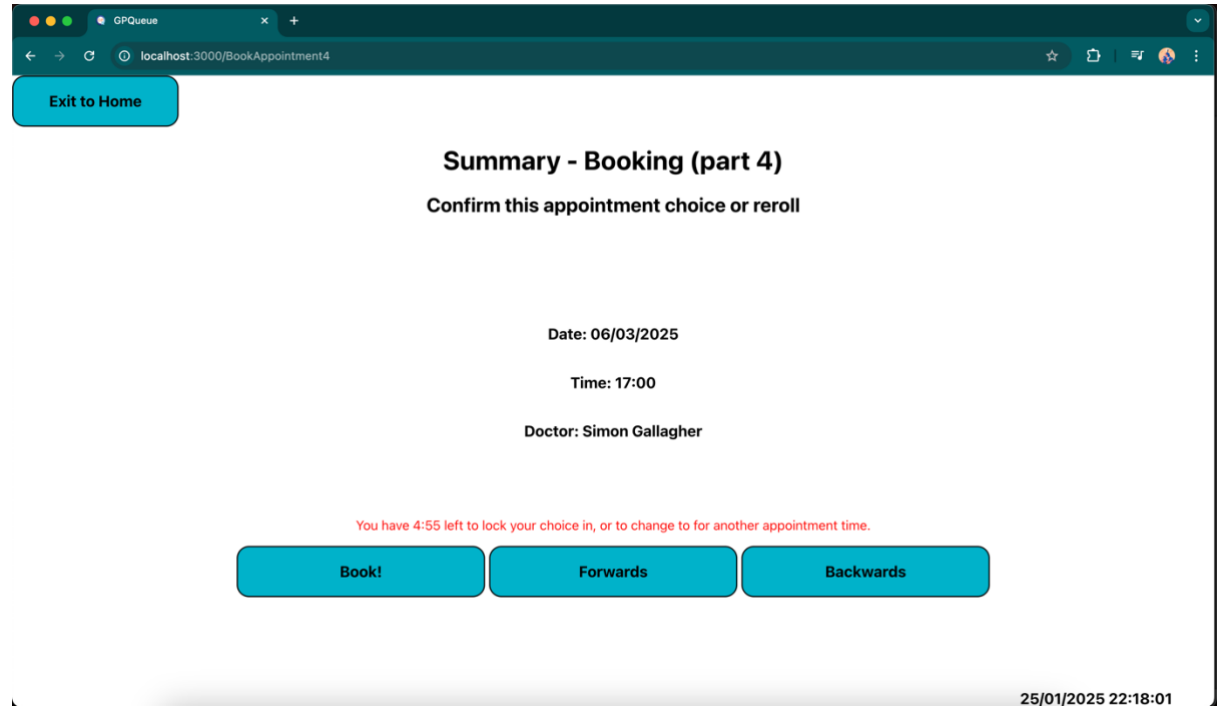
You have 4:30 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

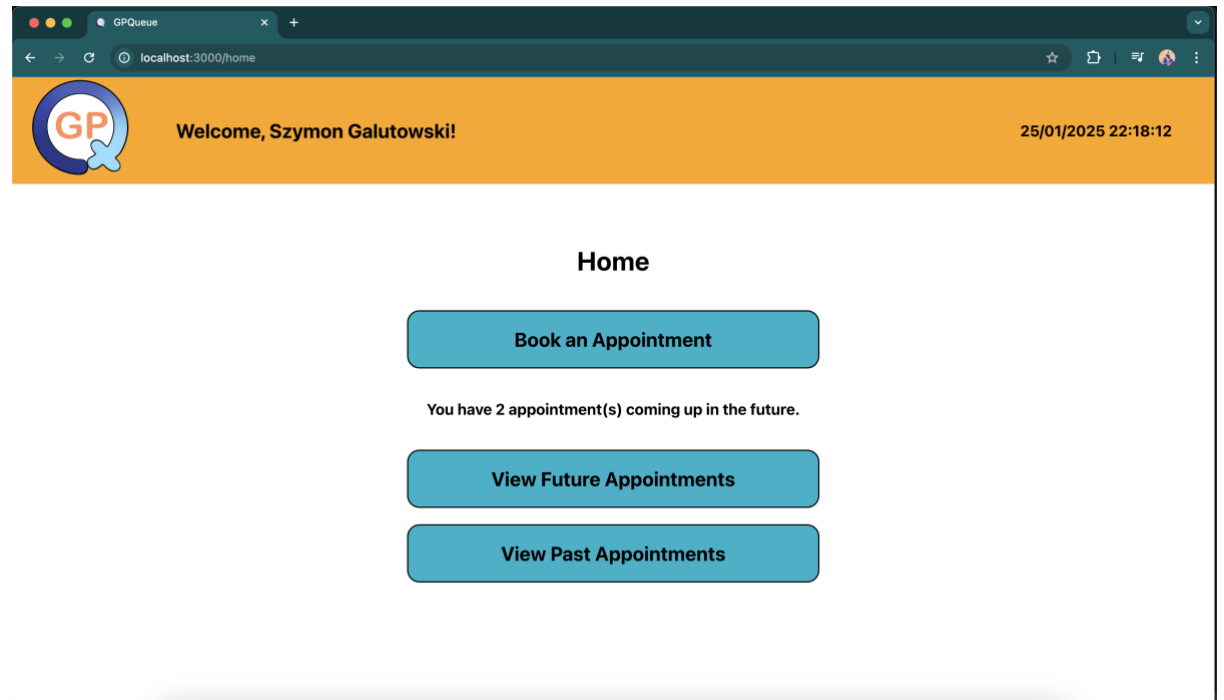
25/01/2025 22:13:35

8e

Before:



After:



Database:

26 rows returned

	appt_id integer	patient_id integer	staff_id integer	priority integer	appt_date date	appt_time character varying	appt_details text	creation_date timestamp with time zone	status text
1	401	1	6	3	2025-01-26	15:00	Persistent fever	2025-01-25 21:56:29.957168+00	scheduled
2	413	7	6	2	2025-01-27	09:00	null	2025-01-25 21:58:30.134422+00	scheduled
3	397	7	6	3	2025-01-26	13:00	null	2025-01-25 21:56:29.957168+00	scheduled
4	398	7	6	3	2025-01-26	13:30	null	2025-01-25 21:56:29.957168+00	scheduled
5	399	7	6	3	2025-01-26	14:00	null	2025-01-25 21:56:29.957168+00	scheduled
6	400	7	6	3	2025-01-26	14:30	null	2025-01-25 21:56:29.957168+00	scheduled
7	402	7	6	3	2025-01-26	15:30	null	2025-01-25 21:56:29.957168+00	scheduled
8	403	7	6	3	2025-01-26	16:00	null	2025-01-25 21:56:29.957168+00	scheduled
9	404	7	6	3	2025-01-26	16:30	null	2025-01-25 21:56:29.957168+00	scheduled
10	389	7	6	3	2025-01-26	09:00	null	2025-01-25 21:56:29.957168+00	scheduled
11	390	7	6	3	2025-01-26	09:30	null	2025-01-25 21:56:29.957168+00	scheduled
12	391	7	6	3	2025-01-26	10:00	null	2025-01-25 21:56:29.957168+00	scheduled
13	392	7	6	3	2025-01-26	10:30	null	2025-01-25 21:56:29.957168+00	scheduled
14	393	7	6	3	2025-01-26	11:00	null	2025-01-25 21:56:29.957168+00	scheduled
15	394	7	6	3	2025-01-26	11:30	null	2025-01-25 21:56:29.957168+00	scheduled
16	395	7	6	3	2025-01-26	12:00	null	2025-01-25 21:56:29.957168+00	scheduled
17	396	7	6	3	2025-01-26	12:30	null	2025-01-25 21:56:29.957168+00	scheduled
18	405	7	6	3	2025-01-26	17:00	null	2025-01-25 21:56:29.957168+00	scheduled
19	406	7	6	3	2025-01-26	17:30	null	2025-01-25 21:56:29.957168+00	scheduled
20	407	7	6	3	2025-01-26	18:00	null	2025-01-25 21:56:29.957168+00	scheduled
21	430	1	6	3	2025-03-06	17:00	Minor skin conditions	2025-01-25 22:17:54.184364+00	scheduled
22	408	7	6	3	2025-01-26	18:30	null	2025-01-25 21:56:29.957168+00	scheduled
23	409	7	6	3	2025-01-26	19:00	null	2025-01-25 21:56:29.957168+00	scheduled
24	410	7	6	3	2025-01-26	19:30	null	2025-01-25 21:56:29.957168+00	scheduled
25	411	7	6	3	2025-01-26	20:00	null	2025-01-25 21:56:29.957168+00	scheduled
26	412	7	6	3	2025-01-26	20:30	null	2025-01-25 21:56:29.957168+00	scheduled

8f

Before:

GPQueue

localhost:3000/BookAppointment4

Exit to Home

Summary - Booking (part 4)

Confirm this appointment choice or reroll

Date: 27/01/2025

Time: 11:30

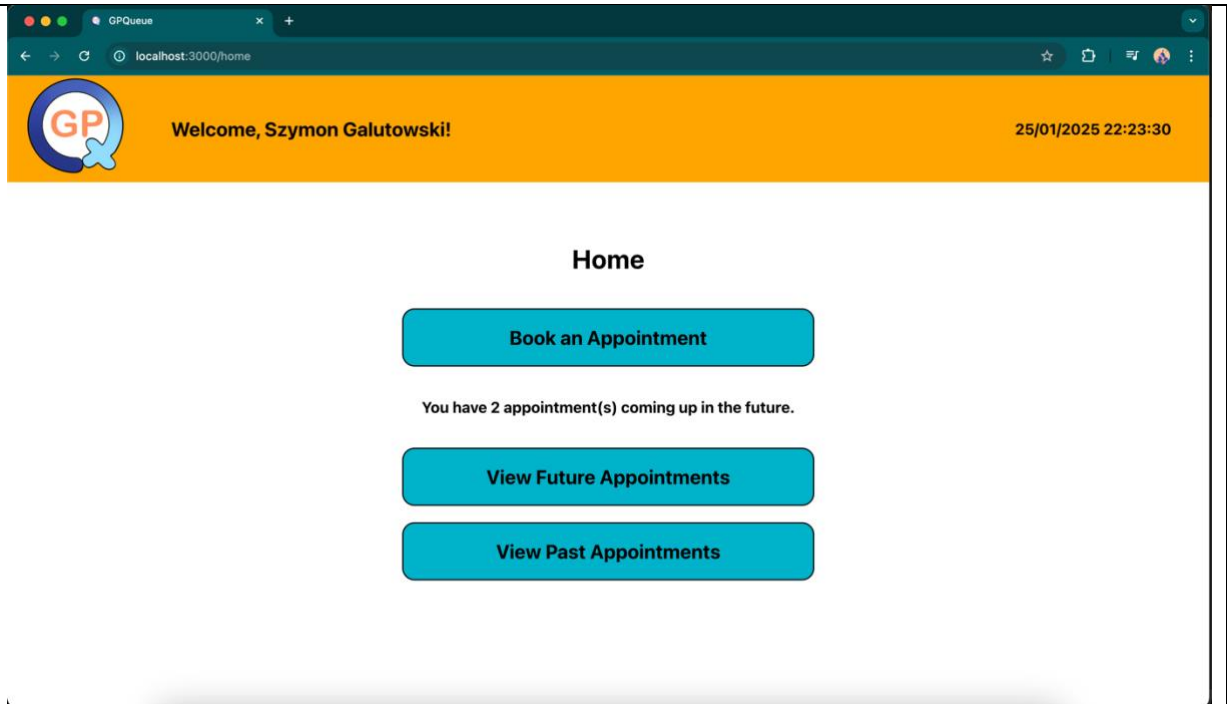
Doctor: Simon Gallagher

You have 4:47 left to lock your choice in, or to change to for another appointment time.

Book! Forwards Backwards

25/01/2025 22:23:10

After:



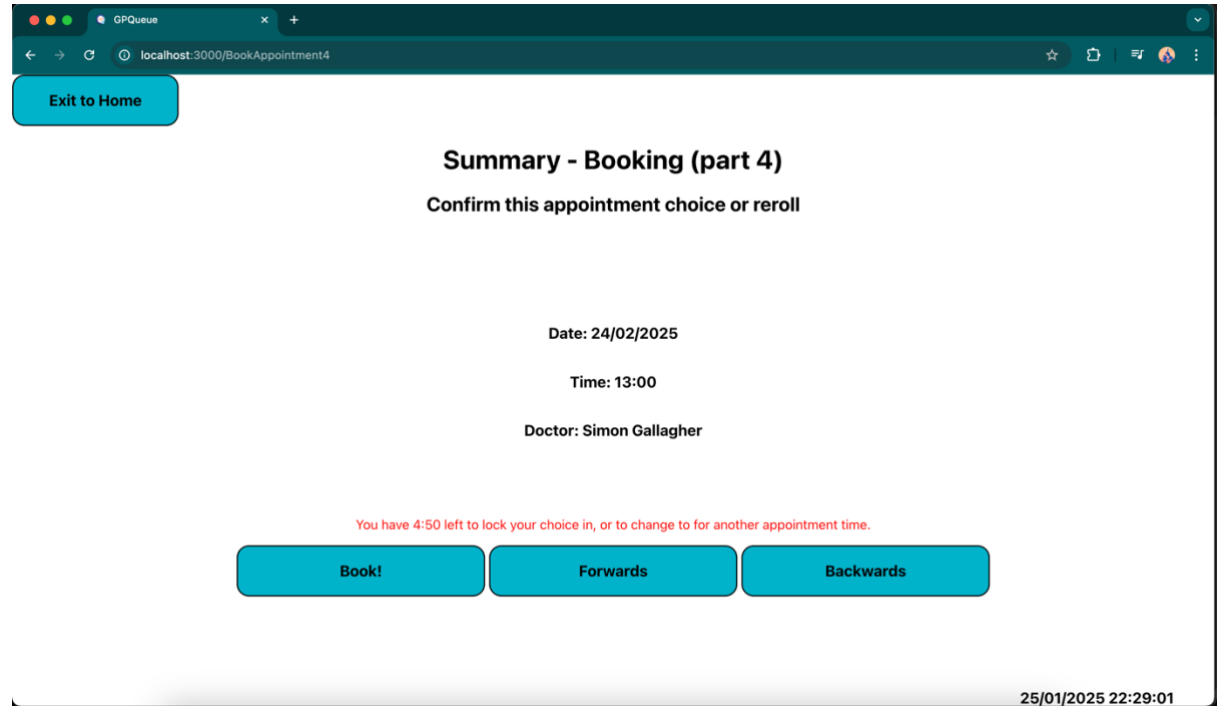
Database (unchanged, no difference from test 8e):

26 rows returned

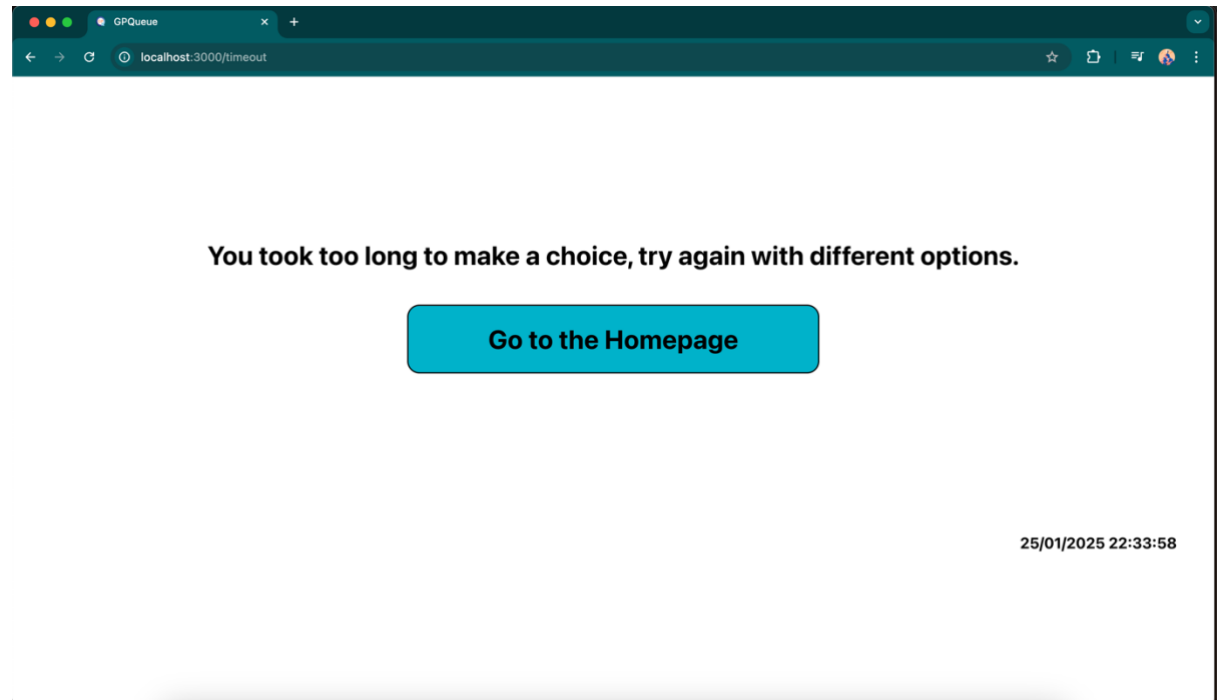
	appt_id integer	patient_id integer	staff_id integer	priority integer	appt_date date	appt_time character varying	appt_details text	creation_date timestamp with time zone	status text
1	401	1	6	3	2025-01-26	15:00	Persistent fever	2025-01-25 21:56:29.957168+00	scheduled
2	413	7	6	2	2025-01-27	09:00	null	2025-01-25 21:58:30.134422+00	scheduled
3	397	7	6	3	2025-01-26	13:00	null	2025-01-25 21:56:29.957168+00	scheduled
4	398	7	6	3	2025-01-26	13:30	null	2025-01-25 21:56:29.957168+00	scheduled
5	399	7	6	3	2025-01-26	14:00	null	2025-01-25 21:56:29.957168+00	scheduled
6	400	7	6	3	2025-01-26	14:30	null	2025-01-25 21:56:29.957168+00	scheduled
7	402	7	6	3	2025-01-26	15:30	null	2025-01-25 21:56:29.957168+00	scheduled
8	403	7	6	3	2025-01-26	16:00	null	2025-01-25 21:56:29.957168+00	scheduled
9	404	7	6	3	2025-01-26	16:30	null	2025-01-25 21:56:29.957168+00	scheduled
10	389	7	6	3	2025-01-26	09:00	null	2025-01-25 21:56:29.957168+00	scheduled
11	390	7	6	3	2025-01-26	09:30	null	2025-01-25 21:56:29.957168+00	scheduled
12	391	7	6	3	2025-01-26	10:00	null	2025-01-25 21:56:29.957168+00	scheduled
13	392	7	6	3	2025-01-26	10:30	null	2025-01-25 21:56:29.957168+00	scheduled
14	393	7	6	3	2025-01-26	11:00	null	2025-01-25 21:56:29.957168+00	scheduled
15	394	7	6	3	2025-01-26	11:30	null	2025-01-25 21:56:29.957168+00	scheduled
16	395	7	6	3	2025-01-26	12:00	null	2025-01-25 21:56:29.957168+00	scheduled
17	396	7	6	3	2025-01-26	12:30	null	2025-01-25 21:56:29.957168+00	scheduled
18	405	7	6	3	2025-01-26	17:00	null	2025-01-25 21:56:29.957168+00	scheduled
19	406	7	6	3	2025-01-26	17:30	null	2025-01-25 21:56:29.957168+00	scheduled
20	407	7	6	3	2025-01-26	18:00	null	2025-01-25 21:56:29.957168+00	scheduled
21	430	1	6	3	2025-03-06	17:00	Minor skin conditions	2025-01-25 22:17:54.184364+00	scheduled
22	408	7	6	3	2025-01-26	18:30	null	2025-01-25 21:56:29.957168+00	scheduled
23	409	7	6	3	2025-01-26	19:00	null	2025-01-25 21:56:29.957168+00	scheduled
24	410	7	6	3	2025-01-26	19:30	null	2025-01-25 21:56:29.957168+00	scheduled
25	411	7	6	3	2025-01-26	20:00	null	2025-01-25 21:56:29.957168+00	scheduled
26	412	7	6	3	2025-01-26	20:30	null	2025-01-25 21:56:29.957168+00	scheduled

8g

Before:



After:



Database (unchanged, no difference from test 8f):

26 rows returned

	appt_id integer	patient_id integer	staff_id integer	priority integer	appt_date date	appt_time character varying	appt_details text	creation_date timestamp with time zone	status text
1	401	1	6	3	2025-01-26	15:00	Persistent fever	2025-01-25 21:56:29.957168+00	scheduled
2	413	7	6	2	2025-01-27	09:00	null	2025-01-25 21:58:30.134422+00	scheduled
3	397	7	6	3	2025-01-26	13:00	null	2025-01-25 21:56:29.957168+00	scheduled
4	398	7	6	3	2025-01-26	13:30	null	2025-01-25 21:56:29.957168+00	scheduled
5	399	7	6	3	2025-01-26	14:00	null	2025-01-25 21:56:29.957168+00	scheduled
6	400	7	6	3	2025-01-26	14:30	null	2025-01-25 21:56:29.957168+00	scheduled
7	402	7	6	3	2025-01-26	15:30	null	2025-01-25 21:56:29.957168+00	scheduled
8	403	7	6	3	2025-01-26	16:00	null	2025-01-25 21:56:29.957168+00	scheduled
9	404	7	6	3	2025-01-26	16:30	null	2025-01-25 21:56:29.957168+00	scheduled
10	389	7	6	3	2025-01-26	09:00	null	2025-01-25 21:56:29.957168+00	scheduled
11	390	7	6	3	2025-01-26	09:30	null	2025-01-25 21:56:29.957168+00	scheduled
12	391	7	6	3	2025-01-26	10:00	null	2025-01-25 21:56:29.957168+00	scheduled
13	392	7	6	3	2025-01-26	10:30	null	2025-01-25 21:56:29.957168+00	scheduled
14	393	7	6	3	2025-01-26	11:00	null	2025-01-25 21:56:29.957168+00	scheduled
15	394	7	6	3	2025-01-26	11:30	null	2025-01-25 21:56:29.957168+00	scheduled
16	395	7	6	3	2025-01-26	12:00	null	2025-01-25 21:56:29.957168+00	scheduled
17	396	7	6	3	2025-01-26	12:30	null	2025-01-25 21:56:29.957168+00	scheduled
18	405	7	6	3	2025-01-26	17:00	null	2025-01-25 21:56:29.957168+00	scheduled
19	406	7	6	3	2025-01-26	17:30	null	2025-01-25 21:56:29.957168+00	scheduled
20	407	7	6	3	2025-01-26	18:00	null	2025-01-25 21:56:29.957168+00	scheduled
21	430	1	6	3	2025-03-06	17:00	Minor skin conditions	2025-01-25 22:17:54.184364+00	scheduled
22	408	7	6	3	2025-01-26	18:30	null	2025-01-25 21:56:29.957168+00	scheduled
23	409	7	6	3	2025-01-26	19:00	null	2025-01-25 21:56:29.957168+00	scheduled
24	410	7	6	3	2025-01-26	19:30	null	2025-01-25 21:56:29.957168+00	scheduled
25	411	7	6	3	2025-01-26	20:00	null	2025-01-25 21:56:29.957168+00	scheduled
26	412	7	6	3	2025-01-26	20:30	null	2025-01-25 21:56:29.957168+00	scheduled

9

Before:

GPQueue

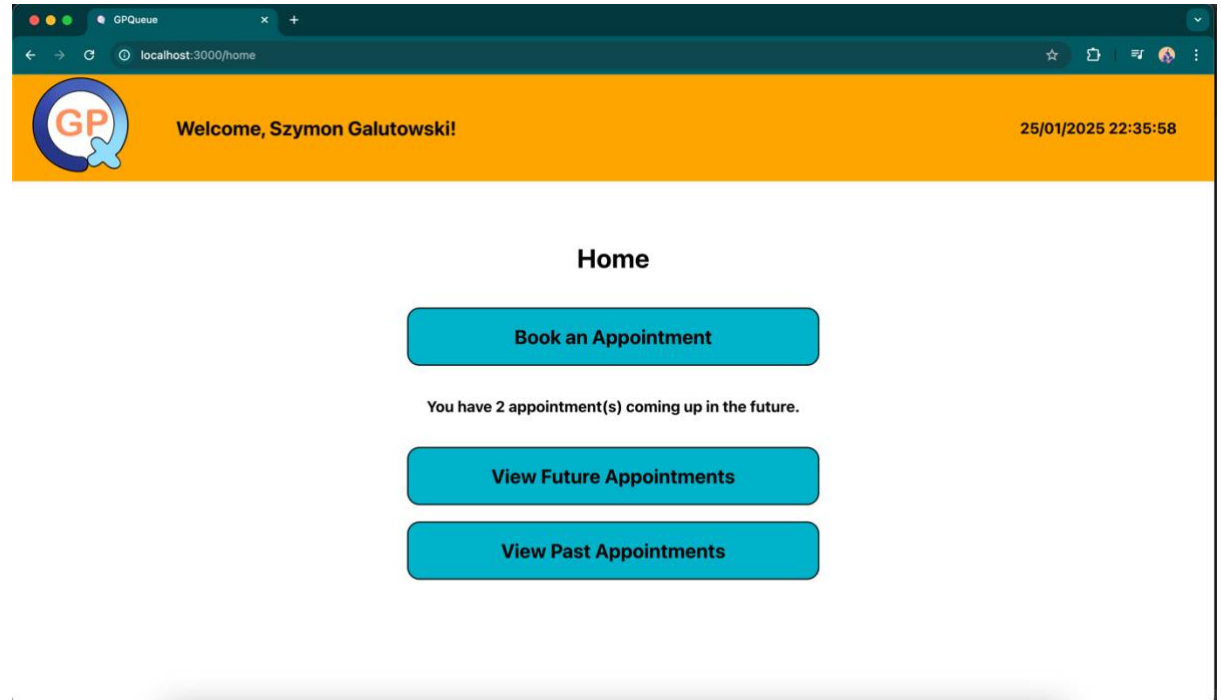
localhost:3000/timeout

You took too long to make a choice, try again with different options.

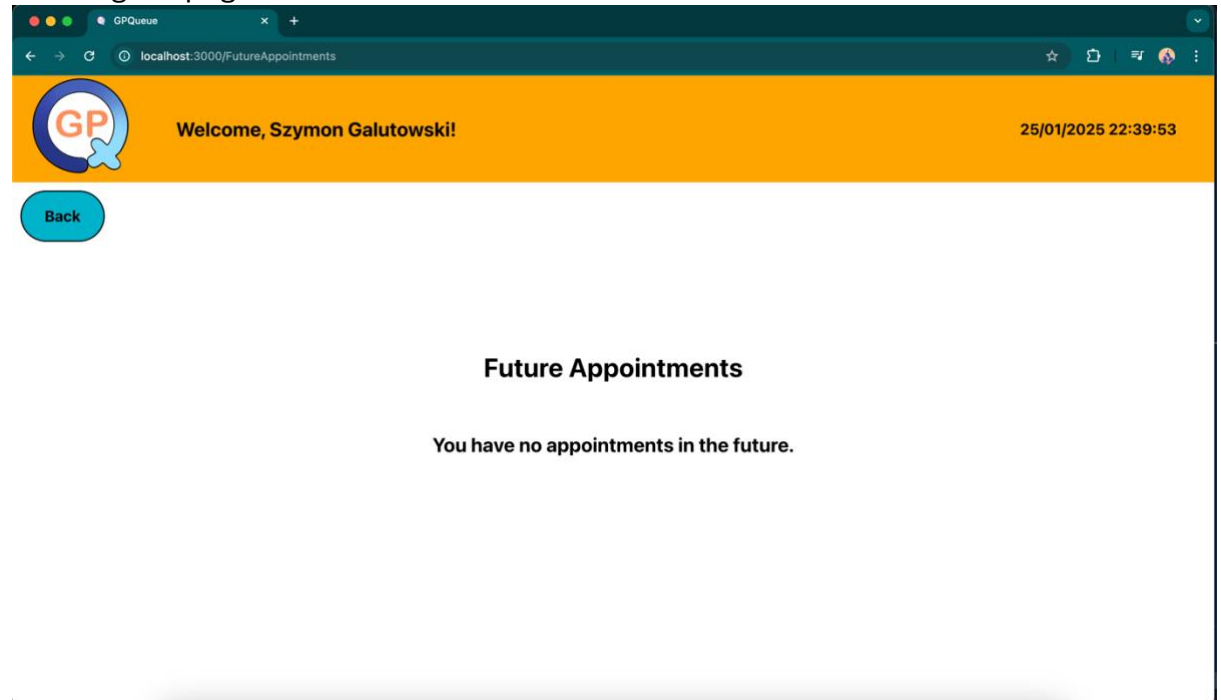
Go to the Homepage

25/01/2025 22:35:49

After:



10a Entering the page:



10b

Entering the page:

The screenshot shows a web browser window with the address bar displaying 'localhost:3000/FutureAppointments'. The page has a dark blue header with a 'GP' logo on the left, a welcome message 'Welcome, Szymon Galutowski!' in the center, and a timestamp '25/01/2025 22:44:55' on the right. Below the header is a blue 'Back' button. The main content area is titled 'Future Appointments' and contains three appointment cards. Each card displays the appointment time, date, staff name (Dr Simon Gallagher), and details (scary symptom 1, 2, or 3).

Appointment Time	Appointment Date	Staff	Details
10:30	28/01/2025	Dr Simon Gallagher	scary symptom 1
17:00	31/01/2025	Dr Simon Gallagher	scary symptom 3
12:30	29/01/2025	Dr Simon Gallagher	scary symptom 2

11a

Entering the page:

The screenshot shows a web browser window with the address bar displaying 'localhost:3000/FutureAppointments'. The page has a dark blue header with a 'GP' logo on the left, a welcome message 'Welcome, Simon Gallagher!' in the center, and a timestamp '25/01/2025 22:46:46' on the right. Below the header is a blue 'Back' button. The main content area is titled 'Future Appointments' and displays the message 'You have no appointments in the future.'

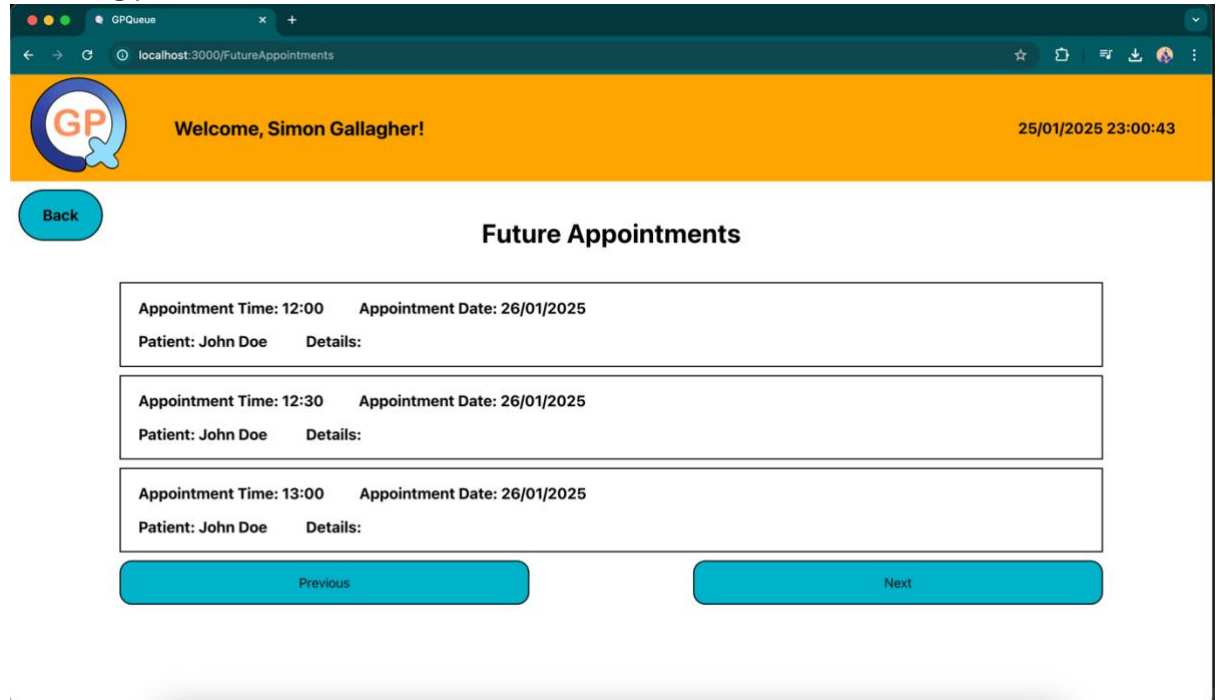
11b Entering the page: (where details = null, so this is correct)

The screenshot shows a web browser window with the URL `localhost:3000/FutureAppointments`. The page has an orange header bar with a 'GP' logo on the left, the text 'Welcome, Simon Gallagher!' in the center, and the timestamp '25/01/2025 22:48:02' on the right. Below the header is a 'Back' button. The main heading is 'Future Appointments'. There are three appointment cards, each with a white background and a thin border. Each card contains the following text: 'Appointment Time: 09:00', 'Appointment Date: 26/01/2025', 'Patient: John Doe', and 'Details:'. At the bottom of the page are two blue buttons labeled 'Previous' and 'Next'.

11c Pressing next:

The screenshot shows the same web browser window as in 11b, but the timestamp is now '25/01/2025 23:00:36'. The 'Future Appointments' section now displays three different appointment cards. Each card contains the following text: 'Appointment Time: 15:00', 'Appointment Date: 26/01/2025', 'Patient: John Doe', and 'Details:'. The 'Back' button and the 'Previous' and 'Next' buttons remain in the same positions.

Pressing previous:

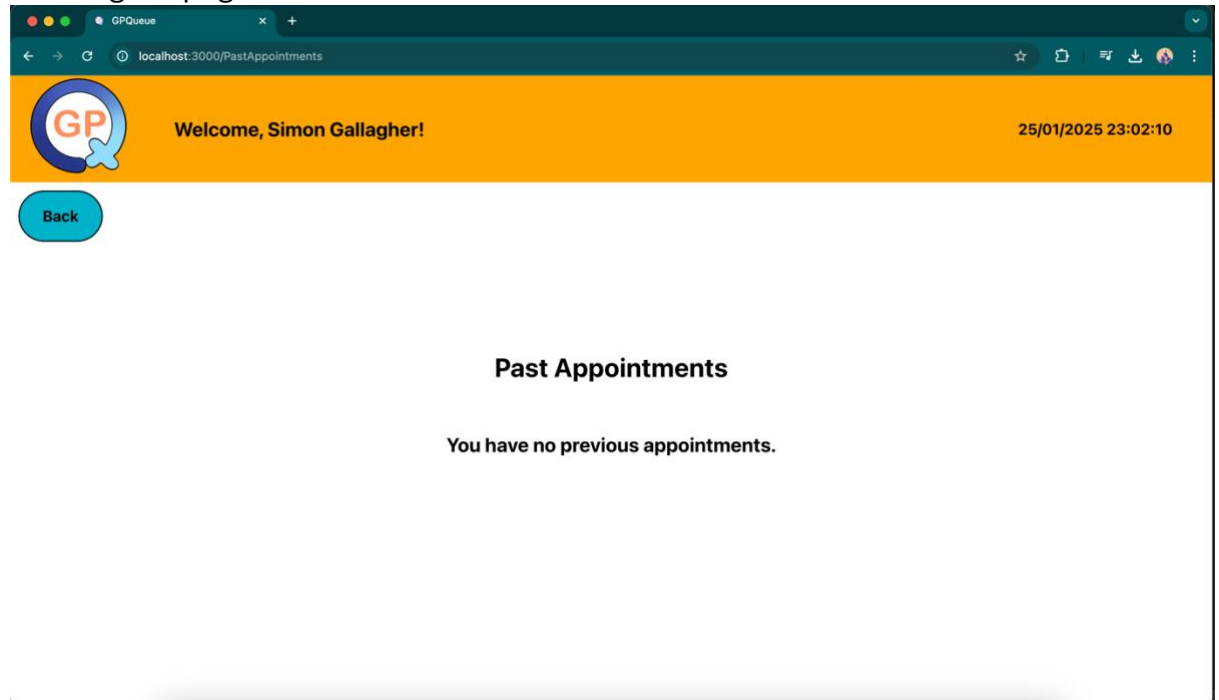


The screenshot shows a web browser window with the URL `localhost:3000/FutureAppointments`. The page has a yellow header bar with a 'GP' logo on the left, the text 'Welcome, Simon Gallagher!' in the center, and the timestamp '25/01/2025 23:00:43' on the right. Below the header is a blue 'Back' button. The main content area is titled 'Future Appointments' and displays three appointment cards. Each card shows 'Appointment Time' and 'Appointment Date' (all for 26/01/2025), followed by 'Patient: John Doe' and a 'Details:' link. At the bottom of the cards are two blue buttons: 'Previous' and 'Next'.

Appointment Time	Appointment Date	Patient	Details
12:00	26/01/2025	John Doe	
12:30	26/01/2025	John Doe	
13:00	26/01/2025	John Doe	

12a

Entering the page:



The screenshot shows a web browser window with the URL `localhost:3000/PastAppointments`. The page has a yellow header bar with a 'GP' logo on the left, the text 'Welcome, Simon Gallagher!' in the center, and the timestamp '25/01/2025 23:02:10' on the right. Below the header is a blue 'Back' button. The main content area is titled 'Past Appointments' and displays the message 'You have no previous appointments.'

12b

Entering the page:

GPQueue

localhost:3000/PastAppointments

Welcome, Simon Gallagher!

25/01/2025 23:03:14

Back

Past Appointments

Appointment Time: 09:00	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Appointment Time: 09:30	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Appointment Time: 10:00	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Previous Next

12c

Pressing Next:

GPQueue

localhost:3000/PastAppointments

Welcome, Simon Gallagher!

25/01/2025 23:04:07

Back

Past Appointments

Appointment Time: 15:00	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Appointment Time: 15:30	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Appointment Time: 16:00	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Previous Next

Pressing Previous:

GPQueue

localhost:3000/PastAppointments

Welcome, Simon Gallagher!

25/01/2025 23:04:18

Back

Past Appointments

Appointment Time: 12:00	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Appointment Time: 12:30	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Appointment Time: 13:00	Appointment Date: 22/01/2025
Patient: John Doe	Details:

Previous

Next

13a Before:

GPQueue

localhost:3000/StaffHome

Welcome, Simon Gallagher!

25/01/2025 23:17:49

Home

You have 24 appointment(s) to tend to in the future.

0 of those appointments are today.

View Future Appointments

View Past Appointments

After:

GPQueue

localhost:3000/FutureAppointments

Welcome, Simon Gallagher!

25/01/2025 23:18:04

Back

Future Appointments

Appointment Time: 09:00	Appointment Date: 26/01/2025
Patient: John Doe	Details:

Appointment Time: 09:30	Appointment Date: 26/01/2025
Patient: John Doe	Details:

Appointment Time: 10:00	Appointment Date: 26/01/2025
Patient: John Doe	Details:

Previous

Next

13b

Before:

GPQueue

localhost:3000/StaffHome

Welcome, Simon Gallagher!

25/01/2025 23:18:22

Home

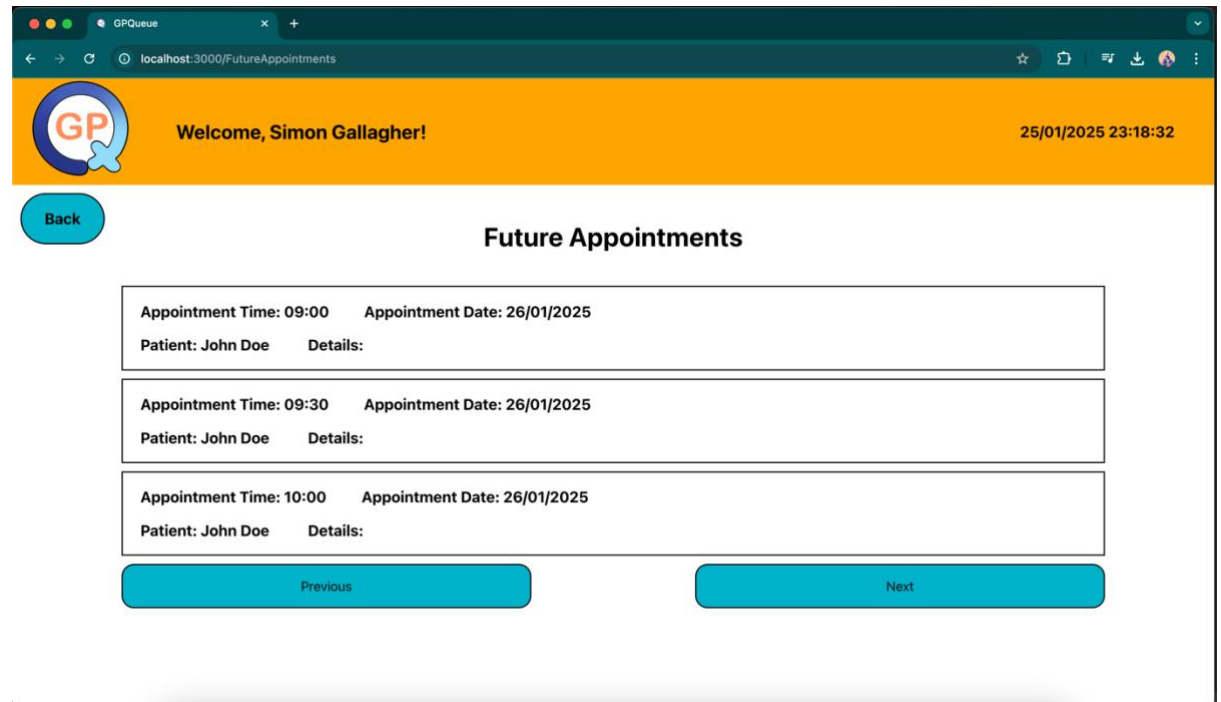
You have 24 appointment(s) to tend to in the future.

0 of those appointments are today.

View Future Appointments

View Past Appointments

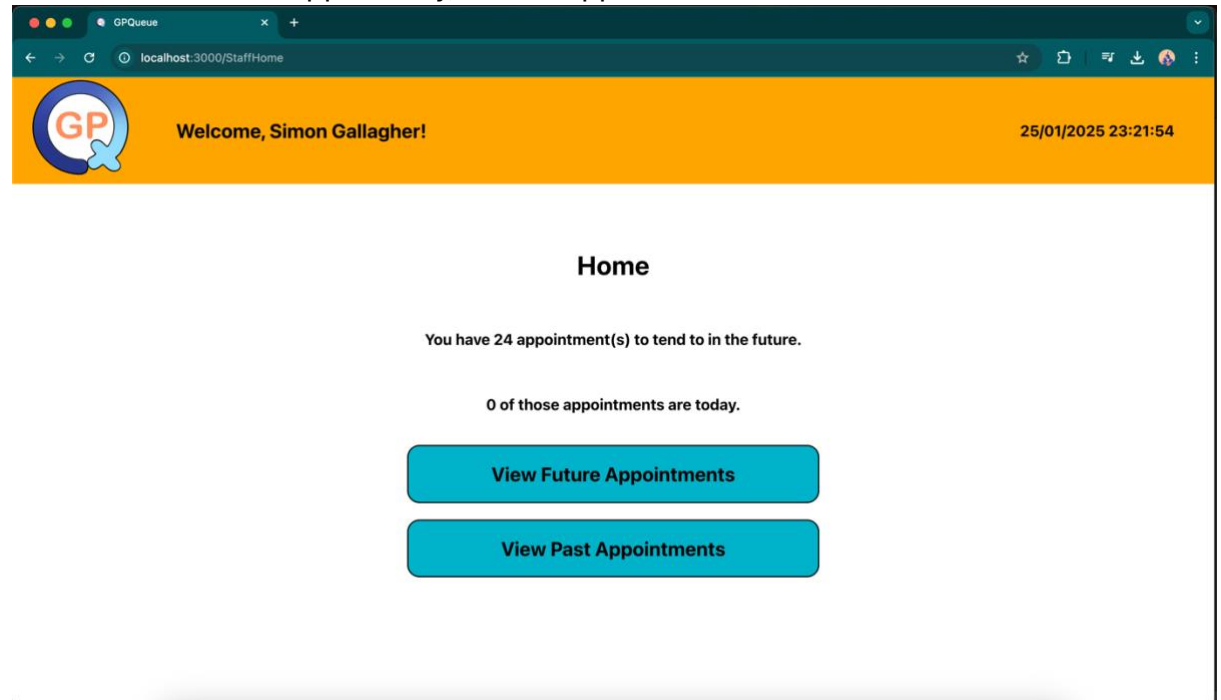
After:



The screenshot shows a web browser window with the URL `localhost:3000/FutureAppointments`. The page has an orange header with a 'GP' logo, the text 'Welcome, Simon Gallagher!', and a timestamp '25/01/2025 23:18:32'. Below the header is a 'Back' button. The main section is titled 'Future Appointments' and contains three appointment cards. Each card displays 'Appointment Time', 'Appointment Date', 'Patient', and 'Details'. The appointments are for 09:00, 09:30, and 10:00 on 26/01/2025, all for 'John Doe'. At the bottom are 'Previous' and 'Next' buttons.

Appointment Time	Appointment Date	Patient	Details
09:00	26/01/2025	John Doe	
09:30	26/01/2025	John Doe	
10:00	26/01/2025	John Doe	

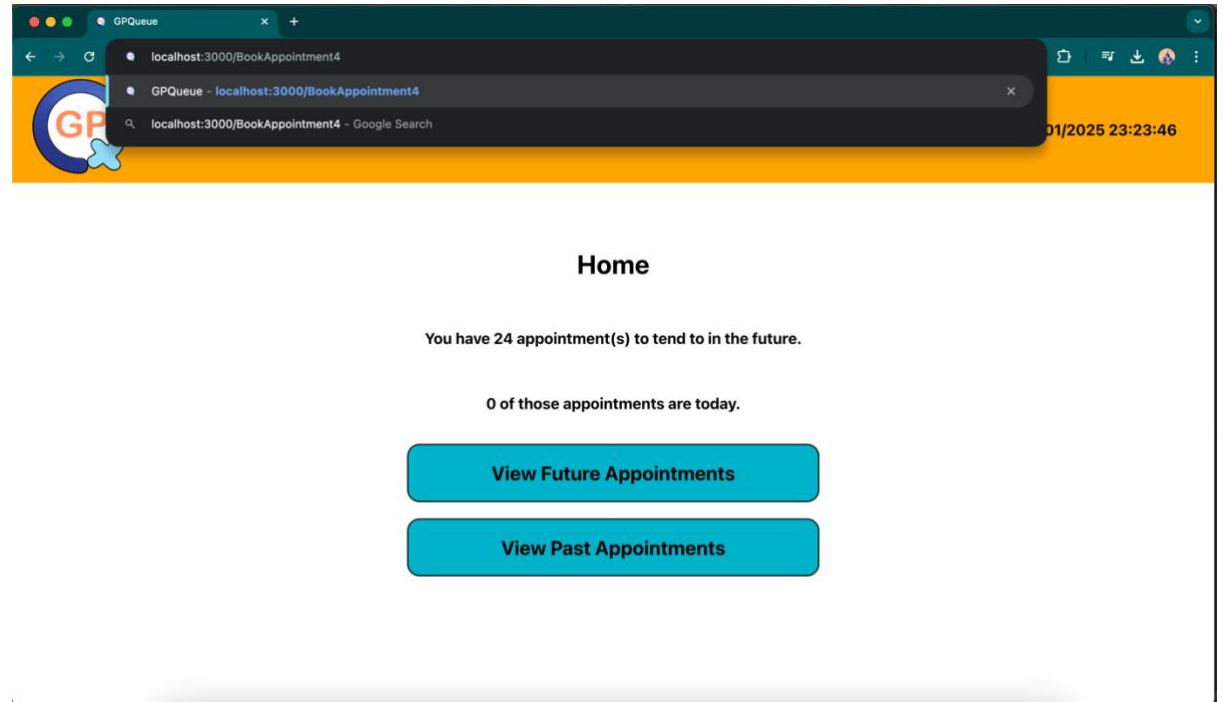
13c Staff member has 0 appts today, and 24 appts in the future:



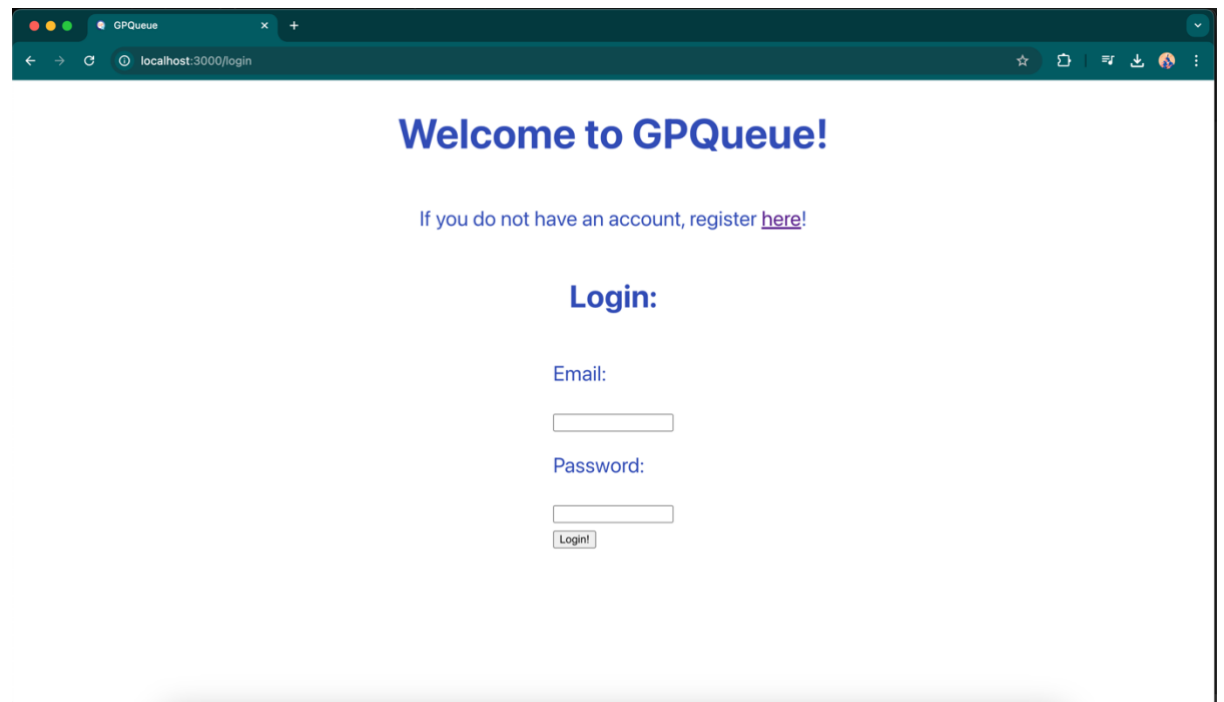
The screenshot shows a web browser window with the URL `localhost:3000/StaffHome`. The page has an orange header with a 'GP' logo, the text 'Welcome, Simon Gallagher!', and a timestamp '25/01/2025 23:21:54'. The main section is titled 'Home' and contains the text 'You have 24 appointment(s) to tend to in the future.' and '0 of those appointments are today.' Below this are two buttons: 'View Future Appointments' and 'View Past Appointments'.

14

Before:

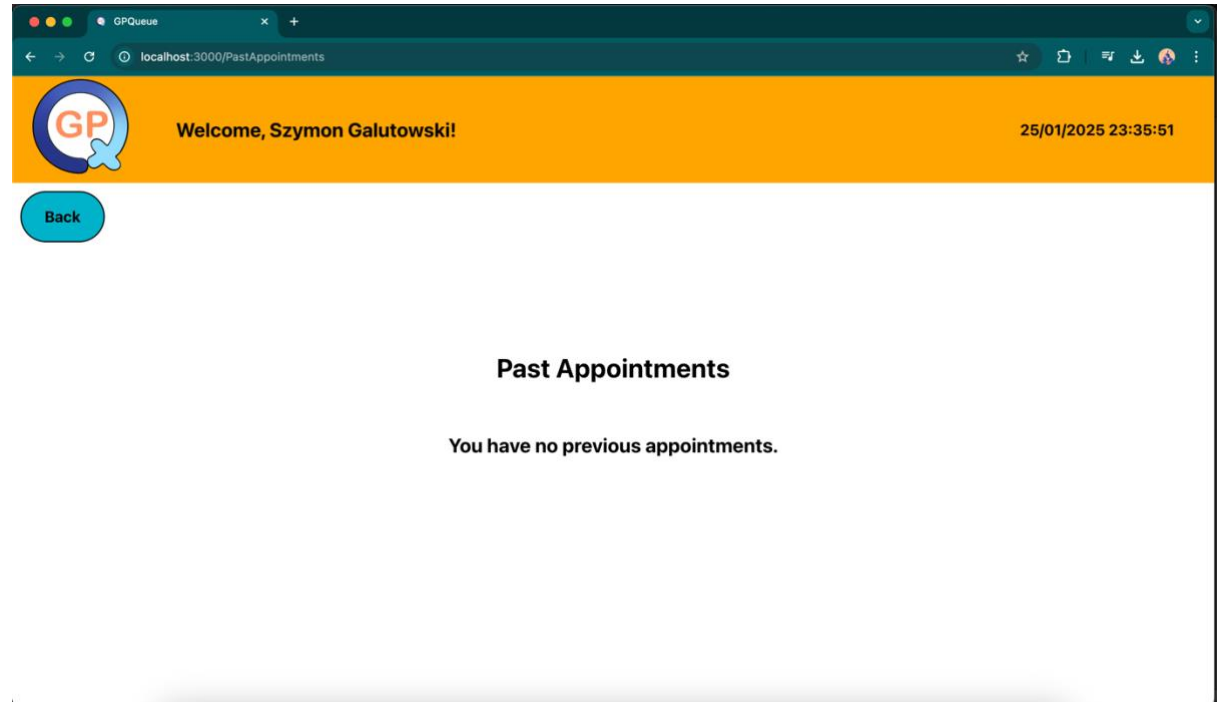


After:

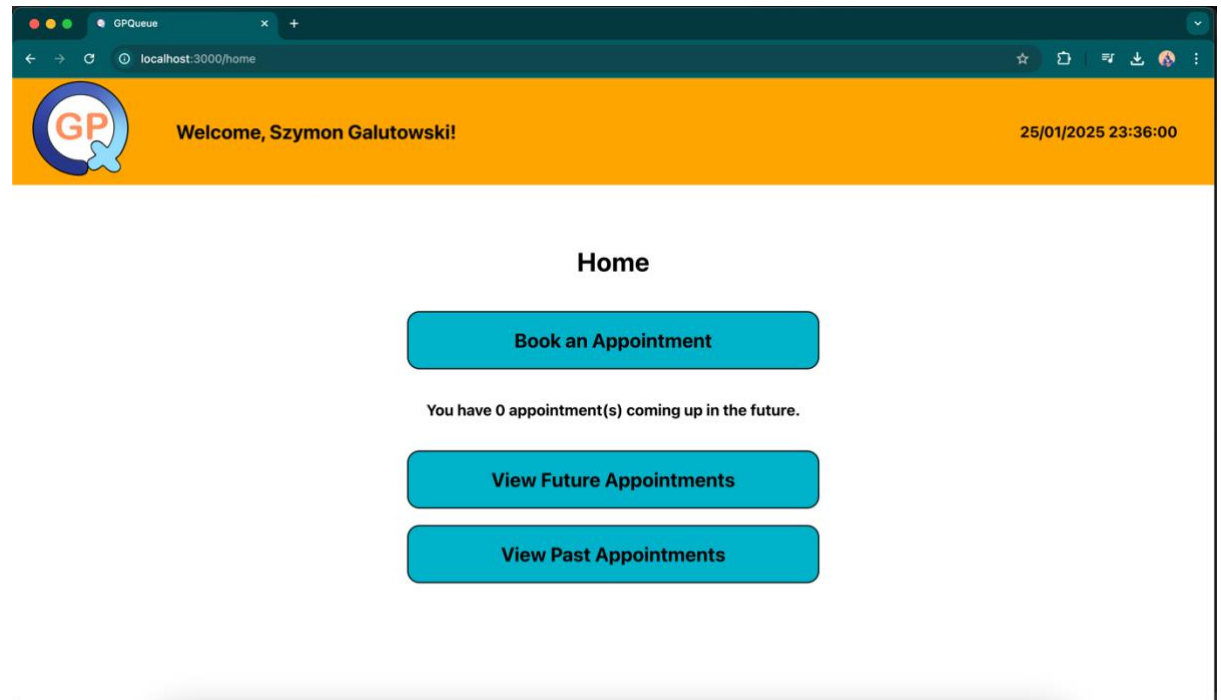


15

Before:

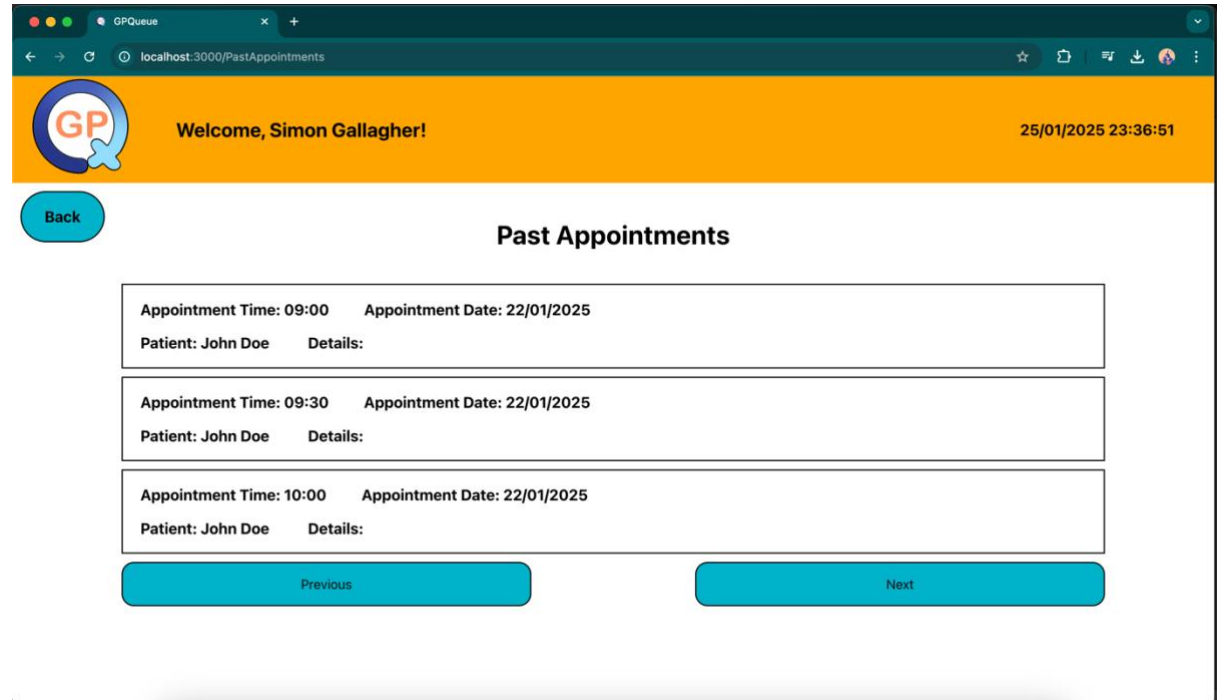


After:

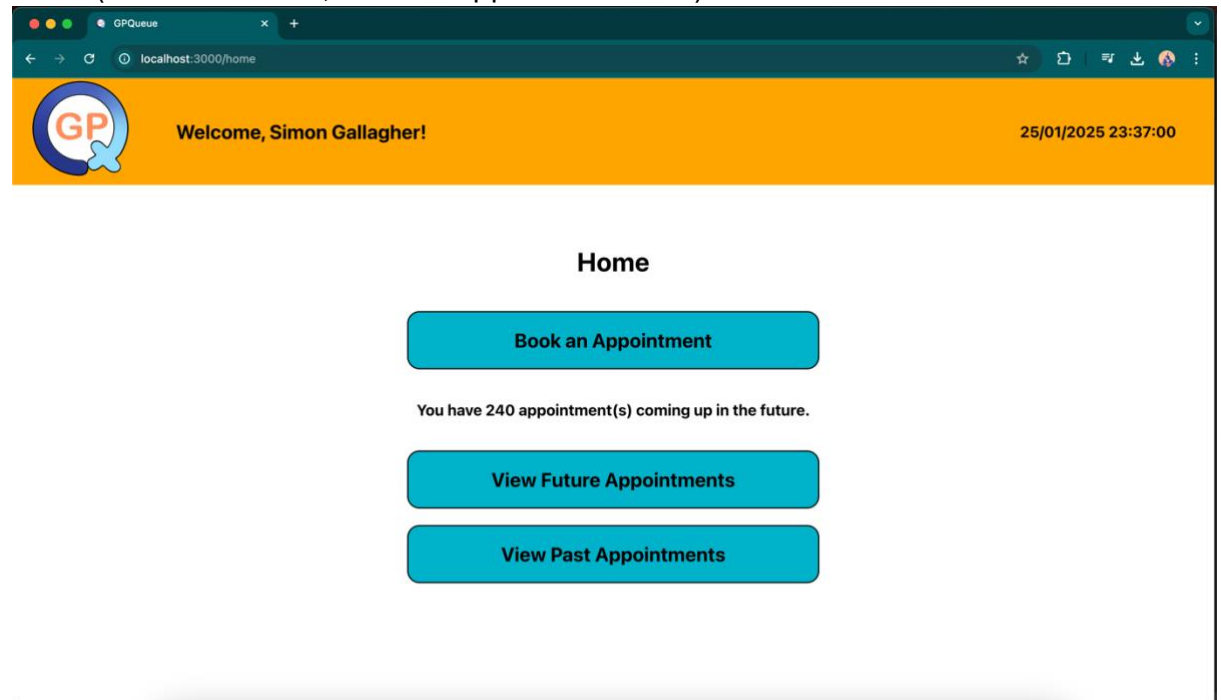


16a

Before:

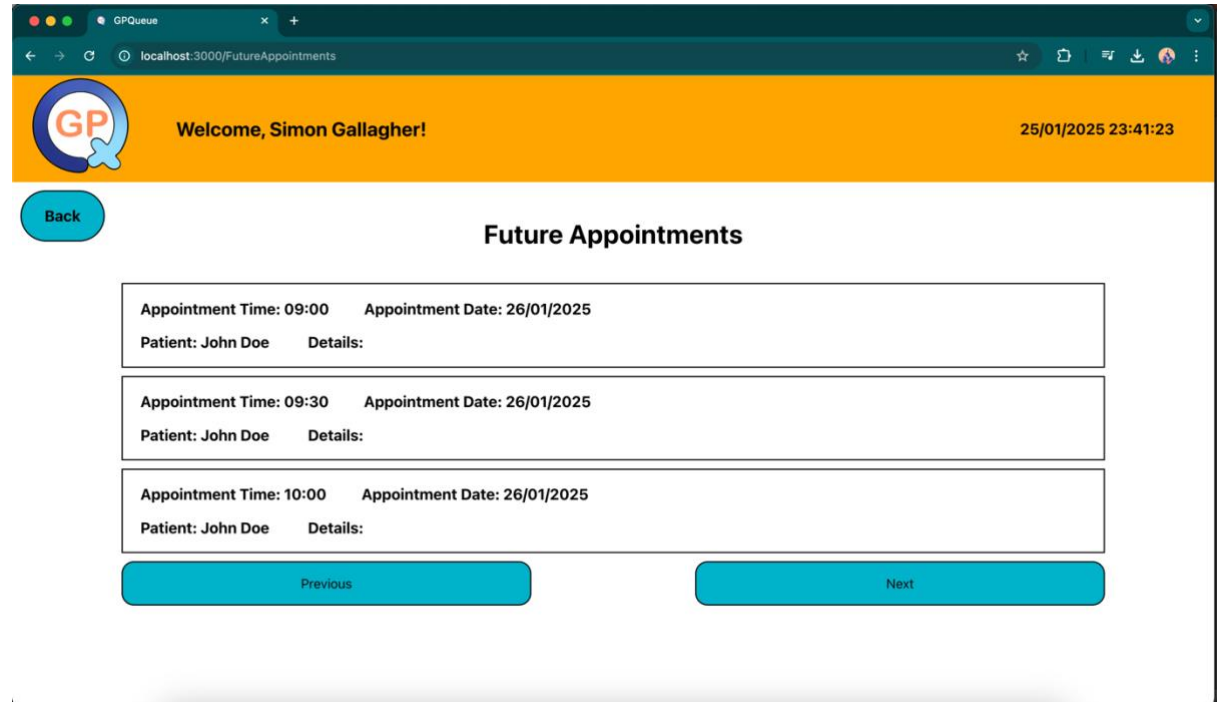


After: (URL is incorrect, and 240 appts is incorrect)

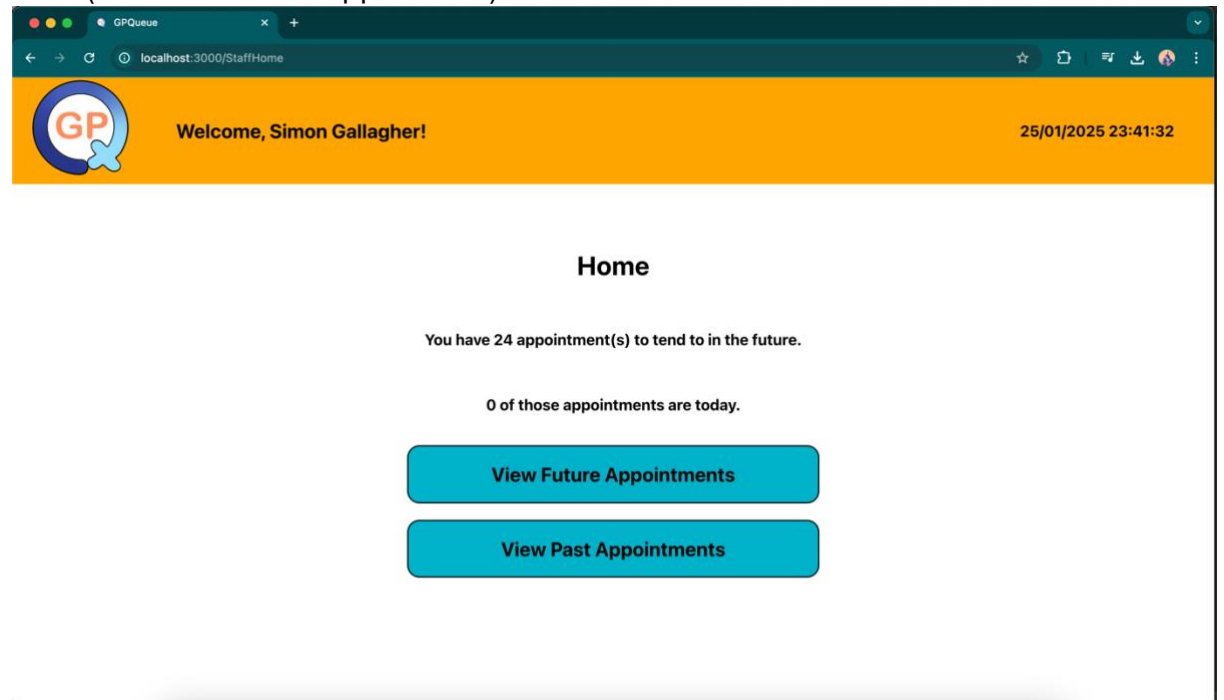


16b

Before:



After (correct URL and appt counts):



Fixes

Test No.	Broken code	New code
1b	<pre> uponLogin(email, password, (callback) => { console.log('User name received:', callback['name']); setCookies({ type: callback['type'], name: callback['name'], prevPageStack: [], currentPage: null, authBool: true }); if (callback['type'] === 'patient') { navigate('/home') } else if (callback['type'] === 'staff') { navigate('/StaffHome') } }); </pre>	<pre> uponLogin(email, password, (callback) => { if (callback === 'failed') { setOpacity(1) } else { setOpacity(0) console.log('User name received:', callback['name']); setCookies({ type: callback['type'], name: callback['name'], prevPageStack: [], currentPage: null, authBool: true }); if (callback['type'] === 'patient') { navigate('/home') } else if (callback['type'] === 'staff') { navigate('/StaffHome') } } }); </pre>
5d	<pre> // Fetch data in useEffect fetchDates(bookingData.priority, (callback) => { if (callback === 'error'){ setBookingData({ priority: null, reason: '', dates: '', times: '' }) } else { setFullDates(callback['dates']); setCurrentDates(callback['dates'][0]); // Set the first month's data as default setLoading(false); // Mark loading as complete } }); </pre>	<pre> fetchDates(bookingData.priority, (callback) => { console.log(callback) if (callback === 'error'){ setBookingData({ priority: null, reason: '', dates: '', times: '' }) navigate('/home') } else { setFullDates(callback['dates']); setCurrentDates(callback['dates'][0]); setLoading(false); } }); </pre>
8b	Frontend bug: <pre> // Fetch data in useEffect fetchAppointments(tempData.priority, tempData.dates, tempData.times, (callback) => { console.log(callback) if (!callback.length){ setNoAppts(true) } else { setApptsList(callback) setCurrentAppt(callback[0]) setLoading(false) try { // eslint-disable-next-line const temp = currentAppt['appt_id'] setOpacity(1) } catch(error) { console.log(error) setOpacity(0) } } }); </pre> Backend bugs: <pre> 63 cur.execute('SELECT appt_id, staff_id, appt_time, appt_date, priority, status, patient_id 64 FROM appointments 65 WHERE appt_time IN %s AND appt_date IN %s AND (status != 'completed')', 66 tuple([time.strftime('%Y-%m-%d') for time in times]), tuple(dates)) 67 result = cur.fetchall() 68 existing_appts = [] </pre> <pre> 86 all_appts_final.append(appt) 87 repeated_appts_list.append([appt['date'], appt['timeslot']]) 88 89 90 for appt in existing_appts: 91 if [appt['date'], appt['timeslot']] not in repeated_appts_list: 92 if appt['priority'] > priority and appt['status'] == 'scheduled': 93 all_appts_final.append(appt) 94 repeated_appts_list.append([appt['date'], appt['timeslot']]) 95 next 96 97 insert = [] 98 update = [] </pre>	Frontend Fix: <pre> // Fetch data in useEffect fetchAppointments(tempData.priority, tempData.dates, tempData.times, (callback) => { console.log(callback) if (!callback.length){ setNoAppts(true) } else { setApptsList(callback) setCurrentAppt(callback[0]) setLoading(false) try { if (callback[index]['appt_id']){ setOpacity(1) } } catch(error) { console.log(error) setOpacity(0) } } }); </pre> Backend Fixes: <pre> 63 cur.execute('SELECT appt_id, staff_id, appt_time, appt_date, priority, status, patient_id 64 FROM appointments 65 WHERE appt_time IN %s AND appt_date IN %s AND (status != 'completed') 66 ORDER BY appt_date, appt_time', 67 tuple([time.strftime('%Y-%m-%d') for time in times]), tuple(dates)) 68 result = cur.fetchall() 69 existing_appts = [] 70 </pre> <pre> for appt in existing_appts: if [appt['date'], appt['timeslot']] not in repeated_appts_list: if appt['priority'] > priority and appt['status'] == 'scheduled': all_appts_final.append(appt) repeated_appts_list.append([appt['date'], appt['timeslot']]) next </pre>
8c	<pre> 177 cur.execute('UPDATE appointments 178 SET patient_id = %s, priority = %s, status = 'scheduled', appt_details = %s 179 WHERE appt_id = %s', (id, priority, appt_id, appt_details)) 180 conn.commit() 181 182 183 184 185 186 187 188 189 190 191 192 193 timechange = datetime2.timeDelta(days=1) 194 appt_date = datetime.strptime(appt_date, '%d/%m/%Y') 195 196 flag = False </pre>	<pre> cur.execute('UPDATE appointments SET patient_id = %s, priority = %s, status = 'scheduled', appt_details = %s WHERE appt_id = %s', (id, priority, appt_id, appt_details)) conn.commit() timechange = datetime2.timeDelta(days=1) appt_date = datetime.strptime(appt_date, '%d/%m/%Y') flag = False </pre>

		Also added this after the rescheduling loop to fix the DB:
16	<pre> 17 const navigate = useNavigate(); 18 19 const sendHome = (event) => { 20 navigate('/home'); 21 } 22 23 return(</pre>	<pre> 226 cur.execute('UPDATE appointments 227 SET status = 'scheduled' 228 WHERE status = %s', (f"scheduled + reserved by {id}",)) 229 conn.commit() 230 231 const navigate = useNavigate(); 232 233 const sendHome = (event) => { 234 if (cookies.type === 'patient'){ 235 navigate('/home'); 236 } else { 237 navigate('/StaffHome') 238 } 239 } </pre>

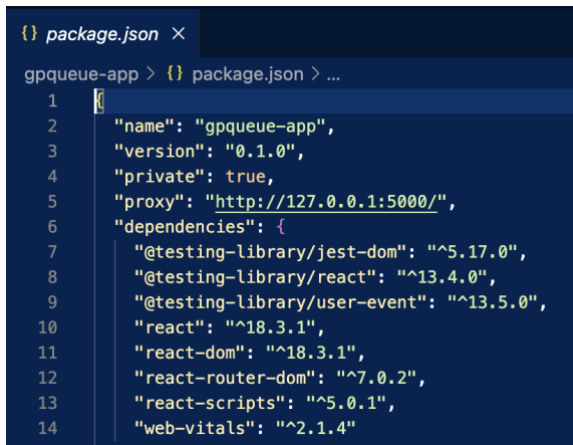
Acceptance Testing

This is to test if my objectives have been met. For the relevant objectives, I have provided timestamps for the following videos:

[GPQueue Website Walkthrough](#) [13]

[GPQueue Cronjob Demonstration](#) [14]

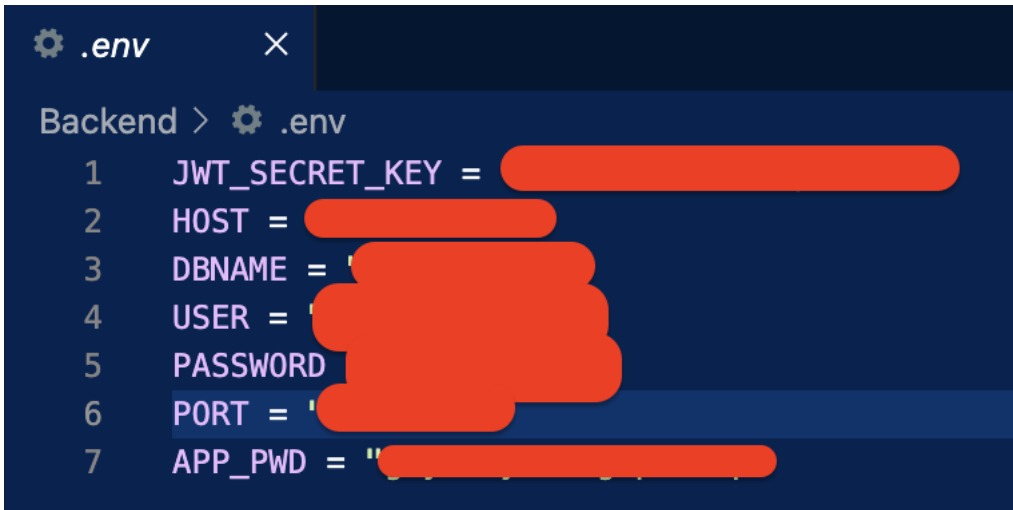
These videos are walkthroughs of parts of my project which I deem necessary to the user experience.

Objective Number	Evidence (if necessary)	Pass/Fail
1	GPQueue Website Walkthrough, timestamp 00:14	Pass
2	GPQueue Website Walkthrough, timestamp 00:00	Pass
3	As you can see, the React is one of the dependencies in my Package.json file.  Also, I am consistently importing and using React Hooks. For more evidence, go to my Technical Solution for the Frontend.	Pass
4	GPQueue Website Walkthrough, timestamp 04:01	Pass
5	GPQueue Cronjob Demonstration 01:11	Pass

6	This is based on opinion, but I think it is very simple: Video	Pass																																																																																																																																																																
7	Staff can only see their own, timestamp 06:16	Pass																																																																																																																																																																
8	GPQueue Website Walkthrough, timestamp 10:51	Pass																																																																																																																																																																
9	GPQueue Website Walkthrough, timestamp 03:33	Pass																																																																																																																																																																
10	GPQueue Website Walkthrough, timestamp 06:09	Pass																																																																																																																																																																
11	GPQueue Website Walkthrough, timestamp 01:45	Pass																																																																																																																																																																
12	GPQueue Website Walkthrough, timestamp 02:00	Pass																																																																																																																																																																
13	The Appointments table in the database contains a priority field, and it is determined by the reason input by the patient. <table><thead><tr><th></th><th>appt_id integer</th><th>patient_id integer</th><th>staff_id integer</th><th>priority integer</th><th>appt_date date</th><th>appt_time character varying</th><th>appt_details text</th><th>creation_date timestamp with time zone</th><th>status text</th></tr></thead><tbody><tr><td>1</td><td>78</td><td>1</td><td>2</td><td>2</td><td>2025-01-20</td><td>11:30</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>2</td><td>74</td><td>1</td><td>2</td><td>2</td><td>2025-01-20</td><td>09:30</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>3</td><td>75</td><td>1</td><td>2</td><td>2</td><td>2025-01-20</td><td>10:00</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>4</td><td>76</td><td>1</td><td>2</td><td>2</td><td>2025-01-20</td><td>10:30</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>5</td><td>77</td><td>1</td><td>2</td><td>2</td><td>2025-01-20</td><td>11:00</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>6</td><td>80</td><td>1</td><td>51</td><td>2</td><td>2025-01-19</td><td>09:30</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>7</td><td>81</td><td>1</td><td>51</td><td>2</td><td>2025-01-19</td><td>10:00</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>8</td><td>82</td><td>1</td><td>51</td><td>2</td><td>2025-01-19</td><td>10:30</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>9</td><td>73</td><td>1</td><td>2</td><td>2</td><td>2025-01-20</td><td>09:00</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>10</td><td>79</td><td>1</td><td>51</td><td>2</td><td>2025-01-19</td><td>09:00</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>completed</td></tr><tr><td>11</td><td>19</td><td>1</td><td>2</td><td>3</td><td>2025-02-12</td><td>10:00</td><td>Mild joint pain</td><td>2025-01-26 17:35:51.94864+00</td><td>scheduled</td></tr><tr><td>12</td><td>25</td><td>3</td><td>2</td><td>3</td><td>2025-01-29</td><td>09:00</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>scheduled</td></tr><tr><td>13</td><td>26</td><td>4</td><td>2</td><td>3</td><td>2025-01-29</td><td>09:30</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>scheduled</td></tr><tr><td>14</td><td>27</td><td>5</td><td>2</td><td>3</td><td>2025-01-29</td><td>10:00</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>scheduled</td></tr><tr><td>15</td><td>28</td><td>6</td><td>2</td><td>3</td><td>2025-01-29</td><td>10:30</td><td>placeholder symptom</td><td>2025-01-26 17:38:11.2267+00</td><td>scheduled</td></tr></tbody></table> Depending on the priority, the user has a higher/lower chance of being rescheduled, which is determined by this logic in the backend (the logic is part of the fetch_appointments function in the backend): <pre>for appt in existing_appts: if [appt['date'], appt['timeslot']] not in repeated_appts_list: if appt['priority'] > priority and appt['status'] == 'scheduled': all_appts_final.append(appt) repeated_appts_list.append([appt['date'], appt['timeslot']]) next</pre>		appt_id integer	patient_id integer	staff_id integer	priority integer	appt_date date	appt_time character varying	appt_details text	creation_date timestamp with time zone	status text	1	78	1	2	2	2025-01-20	11:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	2	74	1	2	2	2025-01-20	09:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	3	75	1	2	2	2025-01-20	10:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	4	76	1	2	2	2025-01-20	10:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	5	77	1	2	2	2025-01-20	11:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	6	80	1	51	2	2025-01-19	09:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	7	81	1	51	2	2025-01-19	10:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	8	82	1	51	2	2025-01-19	10:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	9	73	1	2	2	2025-01-20	09:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	10	79	1	51	2	2025-01-19	09:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed	11	19	1	2	3	2025-02-12	10:00	Mild joint pain	2025-01-26 17:35:51.94864+00	scheduled	12	25	3	2	3	2025-01-29	09:00	placeholder symptom	2025-01-26 17:38:11.2267+00	scheduled	13	26	4	2	3	2025-01-29	09:30	placeholder symptom	2025-01-26 17:38:11.2267+00	scheduled	14	27	5	2	3	2025-01-29	10:00	placeholder symptom	2025-01-26 17:38:11.2267+00	scheduled	15	28	6	2	3	2025-01-29	10:30	placeholder symptom	2025-01-26 17:38:11.2267+00	scheduled	Pass
	appt_id integer	patient_id integer	staff_id integer	priority integer	appt_date date	appt_time character varying	appt_details text	creation_date timestamp with time zone	status text																																																																																																																																																									
1	78	1	2	2	2025-01-20	11:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
2	74	1	2	2	2025-01-20	09:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
3	75	1	2	2	2025-01-20	10:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
4	76	1	2	2	2025-01-20	10:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
5	77	1	2	2	2025-01-20	11:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
6	80	1	51	2	2025-01-19	09:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
7	81	1	51	2	2025-01-19	10:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
8	82	1	51	2	2025-01-19	10:30	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
9	73	1	2	2	2025-01-20	09:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
10	79	1	51	2	2025-01-19	09:00	placeholder symptom	2025-01-26 17:38:11.2267+00	completed																																																																																																																																																									
11	19	1	2	3	2025-02-12	10:00	Mild joint pain	2025-01-26 17:35:51.94864+00	scheduled																																																																																																																																																									
12	25	3	2	3	2025-01-29	09:00	placeholder symptom	2025-01-26 17:38:11.2267+00	scheduled																																																																																																																																																									
13	26	4	2	3	2025-01-29	09:30	placeholder symptom	2025-01-26 17:38:11.2267+00	scheduled																																																																																																																																																									
14	27	5	2	3	2025-01-29	10:00	placeholder symptom	2025-01-26 17:38:11.2267+00	scheduled																																																																																																																																																									
15	28	6	2	3	2025-01-29	10:30	placeholder symptom	2025-01-26 17:38:11.2267+00	scheduled																																																																																																																																																									
14	Other_priority is the priority of the appointment being rescheduled, and as you can see, it is being decremented. Since the lowest priority is 3, the highest is 1, and only appointments with a priority less than yours can be replaced, the maximum amount of reschedules is 2 (3 -> 2 -> 1) since a person with a priority level of 1 cannot replace an appointment with a priority of 1 either. <pre>cur.execute('''SELECT patient_id, priority, appt_date, appt_details FROM appointments WHERE appt_id = %s''', (appt_id,)) other_id, other_priority, appt_date, other_appt_details = cur.fetchone() other_priority -= 1</pre>	Pass																																																																																																																																																																
15	This arrow function is in the frontend, and checks if the appointment level is less than 3. If it isn't then the user cannot go to the first booking page, and since this button is the only method, the maximum number of appointments that can be booked is 3.	Pass																																																																																																																																																																

	<pre>const sendToBook = (event) => { if (appointmentAmount < 3){ console.log('sendToBook') navigate('/BookAppointment1');} }</pre> More evidence: GPQueue Website Walkthrough, timestamp 13:36	
16	GPQueue Website Walkthrough, timestamp 04:35	Pass
17	This is the beginning of a function in ApiCalls.js which creates a Fetch API call, and sends a JWT token in the header (Under Authorization). <pre>function fetchAppointments(priority, dates, times, callback, retried=false) { const token = sessionStorage.getItem('jwt'); fetch(domain + 'fetchAppointments', { method: 'PUT', headers: { 'Content-Type': 'application/json', 'Authorization': `Bearer \${token}` }, body: JSON.stringify({ priority, dates, times }), credentials: 'include' }) }</pre> This is a section from the backend, in particular a part of the Login function which creates a JWT Access token, and a Refresh token, and sends them back to the frontend for it to store. All the endpoints which need to be secure use a @jwt_required() statement, which can be seen in the Technical Solution, Backend.py file. <pre>if fetched == None: return jsonify({'message': 'Invalid credentials'}), 401 id = fetched[0] token = create_access_token(identity=id) refresh_token = create_refresh_token(identity=id) response = jsonify({'message': 'Login successful', 'token':token, 'refresh_token':refresh_token, 'error':None}) return response</pre>	Pass
18	Refresh Token Rotation is in use, with a dedicated function which swaps out the old refresh token for a new one, given that the provided refresh token is the most recently provided one.	Pass

	<pre>@app.route('/refreshJWT', methods=['POST']) @jwt_required(refresh=True) def refreshJWT(): id = get_jwt_identity() header = request.headers.get('Authorization') old_refresh_token = header[7:] conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3], po cur = conn.cursor() cur.execute('SELECT refresh_token_salt FROM "Users" WHERE user_id = %s', (id,)) refresh_token_salt = cur.fetchone()[0] token_hash, refresh_token_salt = custom_hash(password=old_refresh_token, salt=refresh_token_salt) cur.execute('SELECT user_id FROM "Users" WHERE user_id=%s AND refresh_token_hash=%s', (id, str(token_hash))) fetched = cur.fetchone() if fetched == None: return jsonify({'error':'invalid token'}), 401 new_access_token = create_access_token(identity=id) new_refresh_token = create_refresh_token(identity=id) refresh_token_hash, refresh_token_salt = custom_hash(password=new_refresh_token, salt=refresh_token_salt) cur.execute('UPDATE "Users" SET refresh_token_hash = %s, refresh_token_salt = %s WHERE user_id = %s', (str(refresh_token_hash), refresh_token_salt, id)) conn.commit() cur.close() conn.close() return jsonify({'message':'token sent', 'token':new_access_token, 'refresh_token':new_refresh_token, 'error':None})</pre>							
19	Every single function in ApiCalls.js uses Fetch API to make an API call to the backend, which can be seen in the Technical Solution, ApiCalls.js file.	Pass						
20	This is a section of the Backend.py file, showing that I am indeed using Flask for my web server. For more evidence, go to the Technical Solution, Backend.py file. <pre>app = Flask(__name__) app.config['JWT_SECRET_KEY'] = os.getenv("JWT_SECRET_KEY") app.config['JWT_TOKEN_LOCATION'] = ['headers'] app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15) app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(days=30) jwt = JWTManager(app) CORS(app, supports_credentials=True, origins=['http://localhost:3000'], expose_headers=["Content-Type", "Authorization"])</pre>	Pass						
21	This section of code shows a connection to a PostgreSQL database being made, using the python library psycopg2 which is the best way for interacting with a Postgres Database in python. All the parameters necessary for making the connecting are extracted from the .env and passed to a conn_config array at the beginning of every file. This can be seen in the Technical Solution, Backend.py file. <pre>try: conn = psycopg2.connect(host=conn_config[0], dbname=conn_config[1], user=conn_config[2], password=conn_config[3], port=conn_config[4]) cur = conn.cursor() except Exception as error: return jsonify({'error':error})</pre>	Pass						
22	The image below is a screenshot of how my password hashes are stored in my database: <table><thead><tr><th>password_hash text</th><th>salt bytea</th></tr></thead><tbody><tr><td>60648448980547377823517610185372284571564450611415475465860943787125901476874</td><td>\x1B476EF77677DB36EC957B824648F681</td></tr><tr><td>101617360683472177811911394728143060004553082520284654379756989875476602220783</td><td>\xFB04DD48BBEEAFD719DF11D2FFE986FA</td></tr></tbody></table> Here is also the file in which my hashing process occurs.	password_hash text	salt bytea	60648448980547377823517610185372284571564450611415475465860943787125901476874	\x1B476EF77677DB36EC957B824648F681	101617360683472177811911394728143060004553082520284654379756989875476602220783	\xFB04DD48BBEEAFD719DF11D2FFE986FA	Pass
password_hash text	salt bytea							
60648448980547377823517610185372284571564450611415475465860943787125901476874	\x1B476EF77677DB36EC957B824648F681							
101617360683472177811911394728143060004553082520284654379756989875476602220783	\xFB04DD48BBEEAFD719DF11D2FFE986FA							

23	The image below is a screenshot of my env file, showing all the variables I have in there, however they are blurred in this document as some of them are sensitive. I recall these variables using the os and dotenv python libraries.  <pre>Backend > .env 1 JWT_SECRET_KEY = 2 HOST = 3 DBNAME = 4 USER = 5 PASSWORD = 6 PORT = 7 APP_PWD = </pre>	Pass
----	---	------

Evaluation

Second interview

This is the transcript of the follow up interview I conducted with the GP staff member. The staff member was given access to the website, with which they successfully created a staff account and a patient account for purposes of testing. I then asked them to provide qualitative feedback by answering the following questions:

Question	Answer
Are you satisfied with the method of registration and logging in?	Yes, all the instructions were easy to understand, and I was able to log in just fine.
Do you think the website is simple to navigate?	Yes, there wasn't anything confusing on the website. The buttons were all clear and I was able to use it easily.
Are the symptoms on the first page of booking assigned to the correct priorities, in your opinion?	Yes, nothing wrong with it in my opinion.
Is the calendar-like display of the dates to your liking?	Yes, it's great.
Is the booking process missing anything vital?	Not that I could tell, I was able to book an appointment quite easily.
What do you think about the method for fetching available appointments slots?	I like how the website shows me which slots are available rather than making me check them all, it made things a lot faster.
Do you think there were any unnecessary features?	Not really, but maybe for booking part 4 you could have had arrows for backwards & forwards instead of the big buttons.
Do you think the method of rescheduling is a good one?	Yes, I think it is a fair way to do it.
Do you think this system could replace the current system in use?	For sure, this is way simpler to use, and patients will finally stop complaining about the booking.
Would this system help to reduce administrative workload?	Yes, it would be nice for staff members to use, everything is nicely structured.
Did any part of the program feel unsecure?	No.
Did the email reminders help you stay on top of your schedule?	Yes, I really liked this feature.
Would you like to see any extra features added?	Maybe the ability to change the font sizes and colours?
Do you have any extra feedback?	No.

Completeness of Objectives

During my acceptance testing phase, I tested all my objectives and found that all the requirements were met. Here, I will go into deeper analysis and examine how exactly they passed, and perhaps what else I could improve on to go above and beyond the requirements.

Objectives (The project should:)	What Went Well	Even Better If
Use a graphical interface	The project ended up having a simple to use GUI, which fulfilled its functional requirements, and did so while being aesthetically pleasing.	If I had more time, I would put more work into the component styles and colours to make it pop, and so that it would like most modern websites.
Be a website, accessible through a web browser.	The project was a website hosted on the URL localhost:3000, which is accessible by a browser. Therefore it is a website.	If I had more time, I would see if I could properly deploy the web on something like Heroku, and make it accessible on any computer, as it is currently only accessible on my own.
Use React for creation of all the website components and content.	The website was built using React components and React hooks, and many other methods only available to React. Also, the page structure was built using JSX, which is a React language similar to HTML.	Nothing to add.
Make sure that the list of appointment dates is given in a calendar-like (grid instead of list) format.	The list of appointment dates was given as a grid, which is sufficient to pass the objective.	However, to make it even more calendar-like, I could've organised the dates so that each column held the same weekdays, and each row was a different week. This would've been very complicated however, and I deemed it less important than the more functional aspects.
Send email reminders about the appointment a day before and a week before to patients, and 30 minutes before for doctors/nurses.	The email reminders for patients are sent at 9:01am, and the email reminders for staff are sent 30 minutes before their appointment. Therefore, the	To make the reminders better, I could've put some work into trying to make the reminders more aesthetic, like the emails from big companies are nowadays. E.g. incorporating the GPQueue logo into the email, or other images.

	requirement has been met.	
Be very simple to use and understand.	Since the staff member could use it without any trouble, and I didn't add too much complexity in the UI, I think it is simple enough to use and understand, even for less technologically adept people.	Nothing to add.
Make sure staff cannot see other staff information/appointments.	Staff can only see their own appointments; it is impossible for them to see another staff member's appointments. There is no way for even viewing another staff members information.	This was not a requirement, but it perhaps would've been useful to have a settings page where users could view their account details.
Make sure there is an appointment log associated with the patient and staff that were present.	There is an appointment log for future appointments, and an appointment log for past appointments.	Nothing to add.
Give patients the ability to create appointments.	Patients can book an appointment from the homepage.	Nothing to add.
Make sure that staff cannot create appointments.	Staff do not have access to any of the booking pages.	Nothing to add.
Make sure that patients should be able to register with their full name, email address, and their date of birth.	To register, patients enter all these credentials.	Nothing to add.
Make sure that staff should be able to register the same way, but before they gain access an IT admin has to manually verify the member.	To register, staff enter their credentials and are then taken to the verification page.	If I had more time, I could perhaps add a 3 rd role (such as an Admin) who would be able to manage staff members, however this would not be in line with other objectives and would also take up too much time, and is unimportant.
Make sure that the scheduling system should use a priority-based method.	The scheduling system does use a priority-based method: 1 for high urgency, 2 for medium	If I had more time, I would add more priority levels, so that there would be more levels of urgency, so that the

	urgency, 3 for low urgency.	symptoms would be ranked more specifically.
Make sure that patients should be rescheduled a maximum of 2 times and gain a priority level every time they are.	Every time a patient is rescheduled, they gain a priority level, and since level 1s cannot be rescheduled, and level 3 is the lowest priority, the maximum amount of reschedules is 2. (3 -> 2 -> 1)	Maybe the rescheduling system could take into account the user's preferences for dates and times, and use them to reschedule, although that would take a lot of extra time, and the current process works fine.
Make sure that patients should have a maximum of 3 appointments booked in the future at once.	The button to book an appointment only works if the current number of booked appointments is less than 3, so it is impossible to book more than 3.	Nothing to add.
Make sure that appointment timeslots must start at 9:00 and end at 21:00, and each appointment must be 30min long.	The timeslots are hardcoded into the appointment choices, and it is impossible to book an appointment for any time not between 9:00 and 21:00, and all appointment times are in the format hh:00 or hh:30.	Nothing to add.
Use JWT Access Tokens and Refresh Tokens.	JWT Access Tokens are in use, along with Refresh Tokens.	As written earlier in my design, the tokens were stored in SessionStorage. However, SessionStorage is very unsecure, and while I did attempt to implement cookies after finishing the main project, I failed in doing so. This is because it required setting up an HTTPS connection, with valid certificates, but I received errors saying that the certificates had invalid details, which I could not fix.
Use Refresh Token Rotation.	Every time an access token expires, a new access token and refresh token are provided, and the new refresh token is stored in the database, ensuring	Nothing to add.

	that an old refresh token cannot be used to maliciously access data.	
Use Fetch API in conjunction with async/await.	Fetch API is used for all API requests to the web server, and async/await is in use.	Nothing to add.
Use a Flask web server for the backend.	A Flask web server is in use in the backend.	Nothing to add.
Have an established connection with a dedicated PostgreSQL database.	The credentials for connecting to the database are stored in a .env, so this is fulfilled as it will only connect to the specific one.	Nothing to add.
Utilise password hashing	A password hashing algorithm from hashlib is in use.	If I had more time, I could perhaps research how to build a strong password hashing algorithm, although it probably would've been weaker and less secure than what hashlib already offers.
Utilise a .env file for secure storage of important and confidential values.	The .env file is used to store the app password for the reminder Gmail account, credentials for signing into the database, and the key for the JWT encryption.	Nothing to add.

Final Thoughts

I very much enjoyed making this project. Although there were many times when I was confounded by large amounts of syntax and logic errors, eventually I got everything to work as intended. I'm also very happy with the amount I learnt during this project; this was my first time using React.js, coding properly in JavaScript, using CSS, working with an API, running a web server, using JWT, and even using a .env file. So much of my time was put into researching and creating this project, and even though I know it could be improved, I am proud of how it turned out.

I found the researching and coding processes the most interesting. Reading all those articles about web development technologies was enlightening, and retrieving the key information to explain it in my own words was a fun challenge, which I think I succeeded in. One feature that stood out to me was implementing the use of Refresh tokens, because I had to find a way to detect when the access token expired, then retry the previous API call. One solution took me a long time to code and ended up not working (it

was to do with re-rendering the page, so that it would redo the entire request), but since I used GitHub to track my progress, I was able to revert my changes and even incorporate asynchronous computation into the working solution.

If I could revisit this project, I would like to incorporate a searching system. Something that I think could be very useful would be giving staff the ability to search for a patient by entering their name, or parts of their name, or their ID. This would pose an additional problem for me to solve and allow me to conduct more research on searching algorithms utilising partial search terms.

In conclusion, I am glad that I tried to work with something new instead of creating a generic Python command line app, since throwing myself into the deep end has enabled me to gain many new skills, which will be applicable in the future.

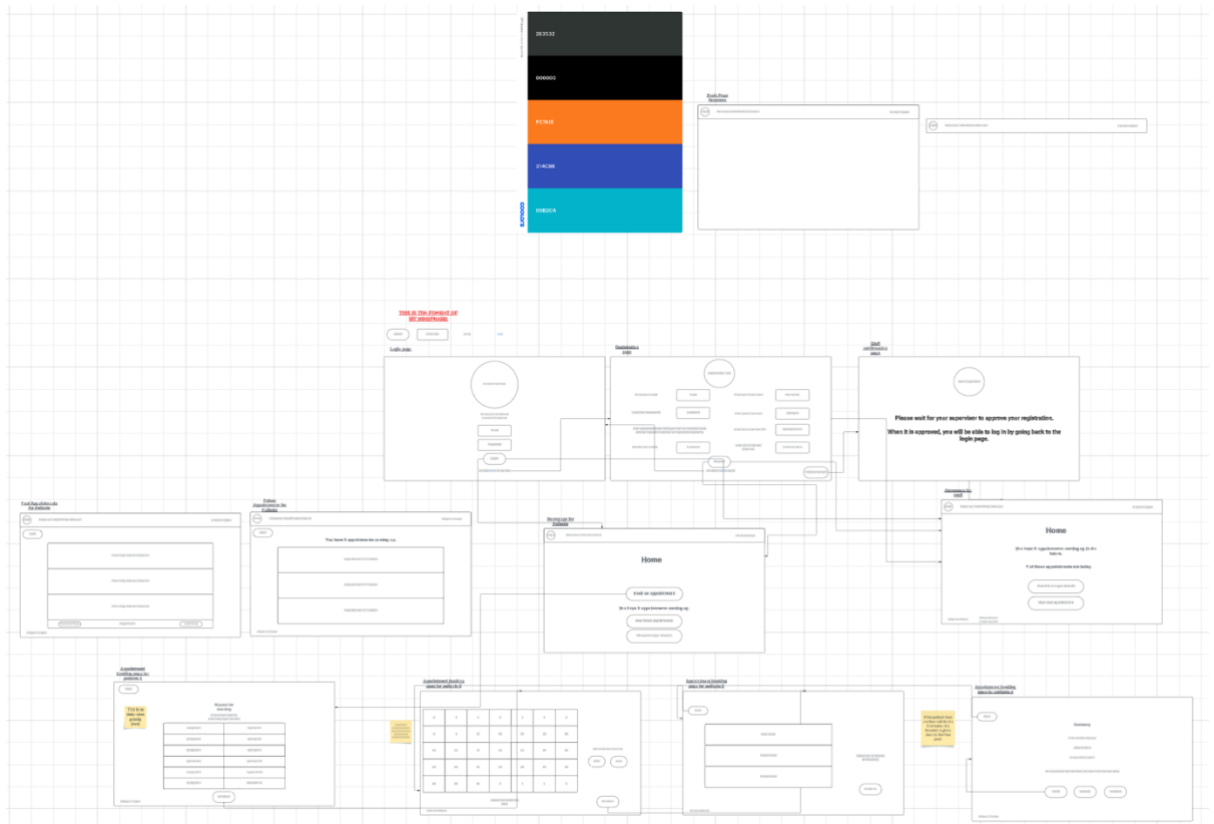
Bibliography

- [1] NHS, “NHS Inform - A to Z,” [Online]. Available: <https://www.nhsinform.scot/illnesses-and-conditions/a-to-z/>. [Accessed 6 9 2024].
- [2] Anthropic, “Claude,” [Online]. Available: <https://claude.ai/new>. [Accessed 7 9 2024].
- [3] M. S.-P. Bartłomiej Klimek, “Comparison of the performance of relational databases PostgreSQL and MySQL for desktop application,” *Journal of Computer Sciences Institute*, vol. 18, pp. 61-66, 2021.
- [4] A. Joshi, “pgAdmin,” [Online]. Available: <https://www.pgadmin.org/download/>. [Accessed 9 2024].
- [5] Facebook, “Create React App Docs,” 2022. [Online]. Available: <https://create-react-app.dev/docs/getting-started>. [Accessed September 2024].
- [6] D. A. Mochammad Fariz Syah Lazuardy, “Modern Front End Web Architectures with React.Js and Next.Js,” *International Research Journal of Advanced Engineering and Science*, vol. 7, no. 1, pp. 132-141, 2022.
- [7] D. V. Kornienko, “Principles of securing RESTful API web services developed with python frameworks,” *Journal of Physics: Conference Series*, vol. 2094, p. 032016, 2021.
- [8] J. O, “Choosing the Right API Interaction Method: A Comparison of Axios and Fetch for Web Applications,” 10 July 2023. [Online]. Available: <https://medium.com/@jorniks/choosing-the-right-api-interaction-method-a-comparison-of-axios-and-fetch-for-web-applications-8921507f371a>. [Accessed January 2025].
- [9] M. D. S. M. W. A. F. T. Alexi Turcotte, “DrAsync: Identifying and Visualizing Anti-Patterns in Asynchronous JavaScript,” in *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, 2022, Pittsburgh, Pennsylvania .
- [10] MDN, “Cross-Origin Resource Sharing (CORS),” MDN, [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>. [Accessed 2025].

- [11] Python Software Foundation, “hashlib — Secure hashes and message digests,” [Online]. Available: <https://docs.python.org/3/library/hashlib.html>. [Accessed January 2025].
- [12] Lucid, “Lucidchart,” [Online]. Available: <https://www.lucidchart.com/pages>. [Accessed 10 2024].
- [13] Szymonator, “GPQueue NEA Walkthrough Video (Website),” Youtube, 26 January 2025. [Online]. Available: <https://www.youtube.com/watch?v=TjHlycdQf4c>. [Accessed 26 January 2025].
- [14] Szymonator, “GPQueue NEA Walkthrough (Cronjob),” Youtube, 26 January 2025. [Online]. Available: <https://www.youtube.com/watch?v=au14Xi0UztQ>. [Accessed 26 January 2025].

Appendix

- All of the wireframes:



- All of the UML Diagrams + Flowcharts:

